

ESWIN

ESSDK OpenCV 开发者手册

Rev 1.1

2026-07-28

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
v1.1	2026-07-28	更新 OpenCV 与示例包交付说明
v1.0	2026-02-11	初始版本

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	6
1.1. 核心特性	6
1.1.1. 传统 vs ESSDK 零拷贝对比	6
2. 环境部署与验证	7
2.1. 交付形态	7
2.1.1. 安装目录	7
2.2. 安装	7
2.2.1. Debian	7
2.2.2. Python wheel	8
2.3. 验证	8
2.3.1. C/C++ 版本与编译参数	8
2.3.2. Python cv2 版本与加载路径	9
2.3.3. 示例与数据是否齐全	9
2.4. 运行示例	9
2.4.1. 快速跑通一个 C++ 示例	9
2.4.2. 快速跑通一个 Python 示例	9
2.5. 故障排除	10
2.5.1. apt install 找不到包或依赖	10
2.5.2. Python import cv2 失败或模块缺失	10
2.5.3. 示例提示模型/数据文件不存在	10
2.5.4. 提示找不到动态库	10
2.6. 信息收集	10
3. C++ 开发指南	11
3.1. 快速入门	11
3.1.1. 环境要求	11
3.1.2. 安装与配置	11
3.1.3. 第一个程序	12
3.1.4. 编译与运行	14
3.2. 硬件加速架构	14
3.2.1. 系统架构概述	14
3.2.2. 硬件加速单元	16
3.2.3. 零拷贝数据路径	17
3.2.4. FFmpeg 后端集成	18
3.3. escore 模块: 核心数据结构	18
3.3.1. esMat 类设计	18
3.3.2. 设备内存管理	19
3.3.3. CPU 与设备内存交互	19
3.3.4. 像素格式体系	21
3.3.5. ROI 与视图操作	22
3.3.6. API 参考表	23
3.4. 视频解码	27
3.4.1. VideoReader: 硬件解码器	27
3.4.2. 支持的解码格式	29
3.4.3. 解码器 API 参考表	31
3.5. 视频编码	32
3.5.1. VideoWriter: 硬件编码器	32
3.5.2. 编码参数配置	34

3.5.3. 回调式编码	36
3.5.4. 支持的编码格式	38
3.5.5. 编码器 API 参考表	40
3.6. esimgproc 模块: 图像处理	41
3.6.1. 图像缩放与裁剪	41
3.6.2. 格式转换	43
3.6.3. 图形绘制	45
3.6.4. 批量处理	47
3.6.5. API 参考表	48
3.7. 故障排查	52
3.7.1. 日志系统	52
3.7.2. 常见错误	53
3.7.3. 调试技巧	54
3.8. 附录	54
3.8.1. 完整示例代码	54
3.8.2. API 参考索引	55
3.8.3. 常用配置表	55
4. Python 开发指南	57
4.1. 快速入门	57
4.1.1. 环境要求	57
4.1.2. 安装与配置	57
4.1.3. 第一个程序	58
4.1.4. 虚拟环境管理	60
4.2. 硬件加速架构	60
4.2.1. 系统架构概述	60
4.2.2. 硬件加速单元	62
4.2.3. 零拷贝数据路径	63
4.2.4. FFmpeg 后端集成	64
4.3. cv2.es 模块: 核心数据结构	65
4.3.1. 模块组织	65
4.3.2. esMat 设计原理	66
4.3.3. 性能关键路径	67
4.3.4. esMat 完整 API 参考	69
4.4. 硬件视频解码	74
4.4.1. 功能概述	75
4.4.2. 支持的解码格式	75
4.4.3. 创建解码器	78
4.4.4. 解码参数配置	79
4.4.5. 解码视频帧	80
4.4.6. 视频格式查询	81
4.4.7. 解码器 API 参考	82
4.5. 硬件视频编码	83
4.5.1. 功能概述	83
4.5.2. 创建编码器	87
4.5.3. 编码参数配置	88
4.5.4. 高级特性	91
4.5.5. 编码器 API 参考	92
4.6. 硬件图像处理	98
4.6.1. 功能概述	98
4.6.2. 图像缩放与格式转换	98
4.6.3. process: 通用前处理	99
4.6.4. OSD 绘制	101
4.6.5. 图像处理 API 快速查阅	103
4.7. 故障排查	104
4.7.1. 日志系统	104
4.7.2. 常见错误	105

- 4.7.3. 调试技巧 106
- 4.8. 附录 106
 - 4.8.1. 完整示例代码 106
 - 4.8.2. API 参考索引 107
 - 4.8.3. 常用编码配置表 107
 - 4.8.4. Python API 设计说明 108

1. 概述

ESSDK OpenCV 是基于 OpenCV 标准接口扩展的硬件加速库，专为 EIC7700 SoC 打造。它通过引入 esMat 数据结构，打通了 VDEC（解码）、VPS（图像处理）、VENC（编码）及 NPU（AI 推理）之间的零拷贝（Zero-Copy）数据通路。

1.1. 核心特性

- **零拷贝流水线:** 数据在硬件单元间通过 DMA-BUF 传递，消除 CPU 内存拷贝瓶颈。
- **统一接口:** 提供与 OpenCV 风格一致的 C++ 和 Python API。
- **硬件透明化:** 应用层无需关心底层硬件调度，仅需调用标准接口。

1.1.1. 传统 vs ESSDK 零拷贝对比

特性	传统 OpenCV 流程	ESSDK 硬件加速流程
解码	CPU 软解 / GPU 解码 (数据回拷)	VDEC 硬解 (输出至设备内存)
处理	CPU (NumPy/OpenCV)	VPS (缩放/裁剪/格式转换)
编码	CPU 上传数据	VENC 直接编码设备内存
性能	受限于内存带宽与 CPU 算力	极低延迟，高吞吐量

2. 环境部署与验证

本章节旨在为交付场景提供最简明的环境搭建指南，帮助用户在目标平台上快速完成**安装** → **验证** → **示例运行** 的全流程。

2.1. 交付形态

- **OpenCV Debian 包 (推荐)**：系统级安装 C/C++ 开发与运行库，不包含示例可执行、示例源码或示例数据。
- **OpenCV 示例 Debian 包 (可选)**：安装预编译示例可执行和示例数据，包名为 `es-sdk-sample-opencv`。
- **Python wheel (可选)**：用于虚拟环境 (venv) 安装 `cv2` (推荐 Python 开发者使用)。

2.1.1. 安装目录

- OpenCV 库与头文件： `/usr/lib/`、 `/usr/include/opencv4/`
- C++ 示例包装脚本： `/usr/bin/opencv/opencv_sample_eswin_*`
- C++ 示例真实可执行： `/usr/lib/opencv-eswin-sample/bin/opencv_eswin_*`
- C++ 示例数据： `/usr/share/opencv-eswin-sample/data/`
- C++ 示例说明： `/usr/share/doc/es-sdk-sample-opencv/README.md`
- Python 示例模块与数据：当前 Python 环境的 `site-packages/opencv_eswin_sample/`，其中数据位于 `opencv_eswin_sample/data/`

2.2. 安装

2.2.1. Debian

2.2.1.1. 方式 A：通过 apt 源安装

```
sudo apt update
sudo apt install opencv4

# 如需运行 C++ 示例，单独安装示例包
sudo apt install es-sdk-sample-opencv
```

2.2.1.2. 方式 B：通过本地 .deb 安装

推荐用 `apt` 安装本地文件（仍可自动处理依赖）：

```
sudo apt update

# 安装 OpenCV deb
sudo apt install ./opencv4_1.12.2_riscv64.deb

# 如需运行 C++ 示例，另行安装示例 deb
sudo apt install ./es-sdk-sample-opencv_1.12.1_riscv64.deb
```

2.2.1.3. 关于“依赖是否会从源上拉取”

- `apt` 会自动解析并安装依赖包；
- 前提是：**依赖包在已配置的软件源中可用**（交付方 apt 源 / 本地离线仓库 / 或依赖已在系统镜像预装）。

2.2.2. Python wheel

适用于 Python 开发者，希望在 venv 中隔离安装 `cv2`。

```
sudo apt update
sudo apt install python3-venv python3-numpy

python3 -c "import numpy; print('numpy OK:', numpy.__version__)"

# 可选：检查系统里是否已有其它来源的 cv2
python3 - <<'PY'
try:
    import cv2
    print('System cv2:', cv2.__file__)
except ModuleNotFoundError:
    print('System cv2: not installed')
PY

# RISC-V 环境下为复用系统 numpy，推荐使用 --system-site-packages
# 如果上面检查发现系统已有其它来源 cv2，请改用不带 --system-site-packages 的隔离 venv
python3 -m venv --system-site-packages ~/opencv_env
source ~/opencv_env/bin/activate

python3 -m pip install ./opencv4-1.12.0-cp312-cp312-linux_riscv64.whl

# 如需运行 Python 示例，安装 Python 示例 wheel
python3 -m pip install ./opencv_eswin_sample-1.0.0-py3-none-any.whl

# 安装完成后再验证 cv2
python3 -c "import cv2; print(cv2.__version__); print(cv2.__file__)"
python3 -c "import cv2; import cv2.escodec; print('escodec OK!)"
python3 -m pip show opencv4
python3 -m pip show opencv-eswin-sample

# YOLOv5s 示例需要（若交付了 ESSDK Python 绑定 wheel）
# python3 -m pip install ./es_py-*.whl
```

提示

- `--system-site-packages` 用于在 RISC-V 环境下复用系统已安装的 `numpy`，避免在 `pip` 中单独安装 `numpy`。若可选检查发现系统中已存在其它来源的 `cv2`（例如 `python3-opencv`），则不要使用 `--system-site-packages`，应创建隔离 `venv`，并按交付方提供的离线 `wheel` 安装 `numpy`，或使用交付方提供的包含 `numpy` 的镜像/仓库。
- 当前构建 `wheel` 文件名形如 `opencv4-1.12.0-cp312-cp312-linux_riscv64.whl`，`pip` 包名为 `opencv4`，但 `Python` 中仍通过 `import cv2` 使用，`cv2.__version__` 输出的是 `OpenCV` 版本，例如 `4.12.0`。

2.3. 验证

2.3.1. C/C++ 版本与编译参数

```
pkg-config --modversion opencv4
pkg-config --cflags --libs opencv4
```

2.3.2. Python cv2 版本与加载路径

Wheel 安装 (venv 激活后):

```
source ~/opencv_env/bin/activate
python3 -c "import cv2; print(cv2.__version__); print(cv2.__file__)"
```

提示: 不建议在同一个 Python 环境中混装多个来源的 cv2 (例如系统 `python3-opencv`、`pip install opencv-python`、以及本项目提供的 ESWIN wheel)。

原因: 这些包都会提供同名模块 `cv2` (以及配套的二进制扩展/动态库)。如果它们同时出现在同一个 Python 的导入搜索路径中, 可能出现“导入 A 的 `cv2`, 却加载到 B 的库”, 从而导致模块缺失 (如 `cv2.escodec`)、符号不匹配 (`undefined symbol`) 或版本不符合预期。

2.3.3. 示例与数据是否齐全

```
ls -lh /usr/bin/opencv/opencv_sample_eswin_*
ls -lh /usr/lib/opencv-eswin-sample/bin/opencv_eswin_*
ls -lh /usr/share/opencv-eswin-sample/data/

source ~/opencv_env/bin/activate
python3 -m pip show opencv-eswin-sample
python3 -c "from opencv_eswin_sample.common.paths import data_dir; print(data_dir())"
```

2.4. 运行示例

示例的 **运行/参数/默认数据路径** 由示例 README 维护:

- C++ 示例说明: `/usr/share/doc/es-sdk-sample-opencv/README.md`
- Python 示例说明: Python 示例 wheel 安装后可通过 `python3 -m opencv_eswin_sample.<module> -h` 查看; 默认数据位于当前 Python 环境的 `opencv_eswin_sample/data/`。

2.4.1. 快速跑通一个 C++ 示例

```
ls /usr/bin/opencv/opencv_sample_eswin_*

# 例如 (以实际安装到板卡的可执行为准)
/usr/bin/opencv/opencv_sample_eswin_hw_transcode -h
/usr/bin/opencv/opencv_sample_eswin_hw_transcode
```

2.4.2. 快速跑通一个 Python 示例

```
source ~/opencv_env/bin/activate

# 例如: 硬件转码
python3 -m opencv_eswin_sample.hwTranscode.hw_transcode -h
python3 -m opencv_eswin_sample.hwTranscode.hw_transcode
```

若您在设备上未找到 C++ 示例 README 或 `/usr/bin/opencv/opencv_sample_eswin_*`, 通常表示尚未安装 `es-sdk-sample-opencv`。若找不到 `opencv_eswin_sample` 模块, 通常表示尚未安装 Python 示例 wheel。

2.5. 故障排除

2.5.1. apt install 找不到包或依赖

```
apt-cache policy opencv4  
sudo apt update
```

若依赖包无法解析：请确认已配置交付方提供的软件源，或依赖已随镜像/离线包交付。

2.5.2. Python import cv2 失败或模块缺失

```
python3 -c "import sys; print(sys.version)"  
python3 -c "import cv2; print(cv2.__version__); print(cv2.__file__)"
```

Wheel 场景请确认 venv 已激活，并检查：

```
python3 -m pip show opencv4  
python3 -c "import cv2; import cv2.error; print('error OK!')"
```

2.5.3. 示例提示模型/数据文件不存在

```
# C++ 示例数据  
ls -la /usr/share/opencv-eswin-sample/data/models/yolov5s/  
ls -la /usr/share/opencv-eswin-sample/data/video/  
  
# Python 示例数据（来自当前 Python 环境中的 opencv_eswin_sample wheel）  
source ~/opencv_env/bin/activate  
python3 -m pip show opencv-eswin-sample  
python3 -c "from opencv_eswin_sample.common.paths import data_dir; print(data_dir())"
```

2.5.4. 提示找不到动态库

```
sudo ldconfig  
  
command -v /usr/bin/opencv/opencv_sample_eswin_hw_transcode  
ldd /usr/lib/opencv-eswin-sample/bin/opencv_eswin_hw_transcode | head
```

2.6. 信息收集

```
uname -a  
cat /etc/os-release  
  
dpkg -l | grep -E "opencv4|ffmpeg|essdk-|escl-"  
  
pkg-config --modversion opencv4  
python3 -c "import cv2; print(cv2.__version__); print(cv2.__file__)"
```

3. C++ 开发指南

3.1. 快速入门

3.1.1. 环境要求

3.1.1.1. 硬件要求

组件	要求
SoC 平台	EIC7700 (RISC-V 64)
内存	>= 2GB RAM
存储	>= 500MB 可用空间

3.1.1.2. 软件依赖

组件	版本要求	说明
Linux 内核	>= 5.10	需要 EIC7700 驱动支持
FFmpeg	7.0-eswin1	必须使用 ESWIN 定制版本
OpenCV	4.12.0+eswin1	本版本
GCC	>= 9.0	C++14 标准支持
CMake	>= 3.16	项目构建工具
pkg-config	任意版本	依赖查找工具

注意：FFmpeg 必须使用 ESWIN 定制版本（包含 ESMPP 硬件编解码器），标准 FFmpeg 版本无法使用硬件加速功能。

3.1.2. 安装与配置

3.1.2.1. 安装 deb 包

```
# 安装 OpenCV 及其依赖
sudo dpkg -i opencv4_1.12.2_riscv64.deb

# 验证安装
dpkg -l | grep opencv
#ii opencv4 1.12.2 riscv64

# 检查 es 模块
pkg-config --cflags opencv4 | grep -o 'escodec\|escore\|esimgproc'
```

安装后的文件结构：

```
/usr/
├── lib/
│   ├── libopencv_core.so.412      # OpenCV 标准模块
│   ├── libopencv_imgproc.so.412
│   └── libopencv_videoio.so.412
```

```

|   |—— libopencv_escore.so.412    # ESWIN 扩展模块
|   |—— libopencv_escodec.so.412
|   |—— libopencv_esimgproc.so.412
|   |—— libopencv_*.so            # 开发符号链接
|   |—— pkgconfig/opencv4.pc      # pkg-config 配置
|   |—— cmake/opencv4/           # CMake 配置文件
|   |—— python3.X/dist-packages/cv2/ # Python 绑定 (可选)
|—— include/opencv4/opencv2/
|   |—— core/                     # OpenCV 标准头文件
|   |—— imgproc/
|   |—— videoio/
|   |—— escore/                   # ESWIN 扩展头文件
|   |—— escodec/
|   |—— esimgproc/

```

OpenCV `opencv4` deb 包不安装示例可执行、示例源码或示例数据。C++ 示例由独立的 `es-sdk-sample-opencv` 包提供。

3.1.2.2. CMake 项目集成

创建 `CMakeLists.txt`：

```

cmake_minimum_required(VERSION 3.16)
project(my_opencv_app)

# C++14 标准 (必需)
set(CMAKE_CXX_STANDARD 14)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 查找 OpenCV 包
find_package(OpenCV 4.12 REQUIRED
  COMPONENTS core escore escodec esimgproc)

# 创建可执行文件
add_executable(my_app main.cpp)

# 链接 OpenCV 库
target_link_libraries(my_app ${OpenCV_LIBS})

# 输出版本信息 (可选)
message(STATUS "OpenCV version: ${OpenCV_VERSION}")
message(STATUS "OpenCV libraries: ${OpenCV_LIBS}")

```

3.1.3. 第一个程序

创建 `main.cpp`，实现简单的视频转码：

```

#include <iostream>
#include <opencv2/escodec.hpp>
#include <opencv2/escore.hpp>

int main(int argc, char** argv)
{
    if (argc != 3) {
        std::cerr << "用法: " << argv[0] << " <输入视频> <输出视频>" << std::endl;
        return -1;
    }
}

```

```
try {
    // 1. 创建硬件解码器
    cv::Ptr<cv::escodec::VideoReader> reader =
        cv::escodec::createVideoReader(argv[1]);

    if (!reader) {
        std::cerr << "无法打开输入视频" << std::endl;
        return -1;
    }

    // 2. 获取视频信息
    cv::escodec::ESFormatInfo fmt = reader->format();
    std::cout << "输入视频: "
        << fmt.width << "x" << fmt.height
        << " @ " << fmt.avgFrameRate << " fps" << std::endl;

    // 3. 创建硬件编码器 (H.264, 保持原分辨率和帧率)
    cv::escodec::EncoderParams encParams;
    encParams.codec = cv::escodec::CODEC_H264;
    encParams.pixelFormat = cv::es::ES_PIXFMT_I420; // 与默认解码输出格式保持一致
    encParams.useHardwareInput = true; // 输入帧来自硬件解码器, 保持零拷贝路径

    cv::Ptr<cv::escodec::VideoWriter> writer =
        cv::escodec::createVideoWriter(
            argv[2],
            cv::Size(fmt.width, fmt.height),
            fmt.avgFrameRate,
            encParams
        );

    if (!writer) {
        std::cerr << "无法创建输出编码器" << std::endl;
        return -1;
    }

    // 4. 零拷贝转码循环
    cv::es::esMat frame;
    int frameCount = 0;

    while (reader->nextFrame(frame)) {
        writer->write(frame);
        frameCount++;

        if (frameCount % 30 == 0) {
            std::cout << "已处理 " << frameCount << " 帧" << std::endl;
        }
    }

    // 5. 完成编码
    writer->release();

    std::cout << "转码完成, 共 " << frameCount << " 帧" << std::endl;
    return 0;

} catch (const cv::Exception& e) {
    std::cerr << "OpenCV 错误: " << e.what() << std::endl;
    return -1;
}
```

代码说明:

1. **零拷贝路径**: `esMat` 从解码器输出后，直接传递给编码器，整个过程无 CPU 内存拷贝
2. **硬件加速**: 解码和编码均使用 VDEC/VENC 硬件单元
3. **异常处理**: 使用 try-catch 捕获 OpenCV 异常
4. **资源管理**: `cv::Ptr` 智能指针自动管理资源生命周期

3.1.4. 编译与运行

3.1.4.1. 方式 1: 使用 CMake

```
# 创建构建目录
mkdir build && cd build

# 生成 Makefile
cmake ..

# 编译
make

# 运行
./my_app input.mp4 output.mp4
```

3.1.4.2. 方式 2: 直接使用 g++

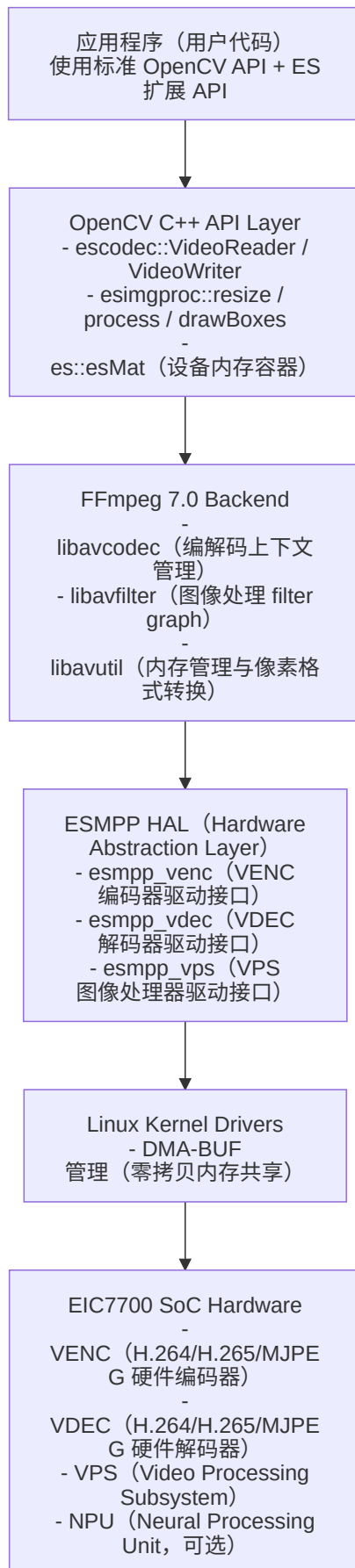
```
# 单文件编译
g++ -std=c++14 main.cpp -o my_app \
    $(pkg-config --cflags --libs opencv4)

# 运行
./my_app input.mp4 output.mp4
```

3.2. 硬件加速架构

3.2.1. 系统架构概述

EIC7700 OpenCV 采用分层架构设计，通过 OpenCV 风格的 ES 扩展 API 暴露硬件加速能力：



关键设计原则：

- 1. **API 兼容性：**escodec API 设计对标 NVIDIA cudacodec，降低迁移成本
- 2. **零拷贝优化：**esMat 基于 FFmpeg AVFrame，支持 DMA-BUF 设备内存
- 3. **后端资源调度：**应用层通过参数选择处理路径，具体硬件资源由 FFmpeg/ESMPP/驱动层协同管理
- 4. **灵活后端：**基于 FFmpeg，易于扩展新的编解码格式和处理算子

3.2.2. 硬件加速单元

3.2.2.1. VDEC: 视频解码器

规格项	参数
支持格式	H.264 (Baseline/Main/High/High10 Profile), H.265 (Main/Main10/Main Still Picture), MJPEG
最大分辨率	H.264/H.265: 8192x8192, JPEG/MJPEG: 32768x32768
最小分辨率	H.264: 48x48, H.265: 144x144, JPEG/MJPEG: 48x48
支持通道数	最多 128 个解码通道（分时复用硬件）
输出格式	NV12/NV21/I420/YV12/YUV400/P010LE, RGB(A/X), BGR(A/X)
特性	硬件去块滤波、错误掩盖、解码序/显示序输出

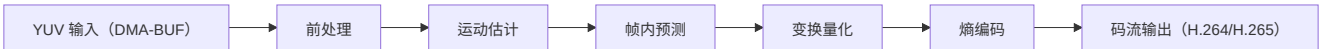
硬件解码流程：



3.2.2.2. VENC: 视频编码器

规格项	参数
支持格式	H.264 (BP/MP/HP/HP10), H.265 (Main/Main10/Main Still Picture), MJPEG
最大分辨率	H.264/H.265: 8192x8192, JPEG/MJPEG: 32768x32768
最小分辨率	H.264: 144(宽)x128(高), H.265: 136(宽)x128(高), JPEG/MJPEG: 32x32
对齐要求	宽高最小对齐 2 像素
输入格式	NV12/ NV21/ I420/ YV12/ YUY2/ UYVY/ P010LE/ P010BE (8bit/10bit)
码率控制	CBR/VBR/FixQP/QPMAP (H.264/H.265)，CBR/FixQP (MJPEG)
GOP 结构	NormalP/DualRef/SmartRef/AdvSmartP/BiPredB/LowDelayB
帧率范围	可配置，支持帧率控制

硬件编码流程：



3.2.2.3. VPS: 图像处理器

VPS 基于 DWE/VSE/HAE 三个硬件实现，各硬件规格如下：

硬件	最大分辨率	最小分辨率	主要功能
DWE	输入 4096x3072/输出 3840x2160	640x480	鱼眼矫正/畸形矫正/旋转
VSE	输入 4096x2160/输出 3840x2160	64x32	缩放/裁剪/格式转换/分流
HAE	7680x3840	64x16	缩放/旋转/镜像/归一化/绘制/合流

功能支持：

功能	硬件	备注
缩放	VSE/HAE	DWE 仅支持放大至 3840x2160
裁剪	DWE/VSE/HAE	组裁剪/通道裁剪
旋转	DWE/HAE	90/180/270 度，建议优先 HAE
镜像/翻转	DWE/HAE	水平/垂直，可组合实现 180° 旋转
格式转换	DWE/VSE/HAE	YUV ⇌ RGB, packed ⇌ planar
归一化	HAE	Z-score/Min-Max
绘制	HAE	矩形/线段/文本/Alpha 混合
分流	VSE	一进多出（最多 3 路输出）
合流	HAE	多进一出/Alpha Blend

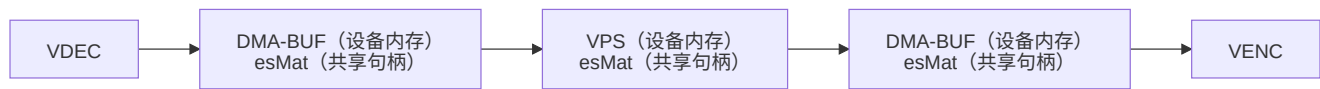
3.2.3. 零拷贝数据路径

3.2.3.1. 传统 CPU 拷贝 vs 零拷贝

传统方式（存在 CPU 拷贝）：



零拷贝方式（EIC7x OpenCV）：



零拷贝实现机制：

1. **DMA-BUF 共享**：兼容路径中通过 fd 共享/传递同一 DMA-BUF 引用，避免 CPU 侧整帧拷贝
2. **AVFrame 传递**：esMat 内部封装 FFmpeg AVFrame (DRM_PRIME 类型)
3. **文件描述符传递**：通过 fd 在硬件单元间传递内存引用
4. **引用计数**：AVBufferRef 机制管理内存生命周期

3.2.3.2. 零拷贝路径要求

场景	是否支持零拷贝	说明
VDEC → VENC	支持	直接传递 DMA-BUF
VDEC → VPS → VENC	支持	输入/输出格式与后端兼容时，图像处理链路保持 DMA-BUF
CPU 创建 → VENC	不支持	需要 upload 到设备内存
VDEC → CPU 处理 → VENC	不支持	download/upload 破坏零拷贝
esMat → esMat（浅拷贝）	支持	赋值操作共享底层存储，引用计数

3.2.4. FFmpeg 后端集成

EIC7x OpenCV 基于 FFmpeg 7.0 实现，充分利用其成熟的编解码和容器处理能力。

3.2.4.1. 容器格式支持

通过 FFmpeg 的 libavformat，escodec 支持多种容器格式：

容器格式	扩展名	VideoReader	VideoWriter	备注
MP4	.mp4	支持	支持	最常用，iOS 设备友好
Matroska	.mkv	支持	支持	开源格式，支持多音轨
AVI	.avi	支持	支持	兼容性好
FLV	.flv	支持	支持	Flash 流媒体
MPEG-TS	.ts	支持	支持	广播标准
Raw H.264	.h264	支持	支持	无容器，纯码流
Raw H.265	.h265/hevc	支持	支持	无容器，纯码流
RTMP	rtmp://	支持	支持	实时流媒体协议

3.3. escore 模块: 核心数据结构

3.3.1. esMat 类设计

esMat 是 EIC7x OpenCV 的核心数据结构，设计目标是提供与 cv::Mat 类似的接口；底层可管理 DMA-BUF 设备内存，也可包装 FFmpeg AVFrame。esMat 是独立设计的类，不继承 cv::Mat；在 DMA-BUF 兼容路径下可与硬件后端零拷贝交互。

3.3.1.1. 与 cv::Mat 的差异

特性	cv::Mat	esMat
内存位置	CPU 内存 (malloc/new)	主要为设备内存 (DMA-BUF)，也可包装 AVFrame
拷贝语义	浅拷贝（共享 data 指针）	浅拷贝（共享底层存储/AVFrame 引用）
数据访问	直接指针访问 (mat.data)	需要 map 或 download
像素格式	OpenCV 类型 (CV_8UC3 等)	OpenCV 类型 + EsPixelFormat
ROI 操作	支持	支持（共享底层存储并调整视图元数据）

特性	cv::Mat	esMat
与硬件交互	需要显式上传/下载	DMA-BUF 兼容路径下可零拷贝传递

重要提示：不要混用 `esMat` 和 `cv::Mat` 进行直接赋值，应使用 `upload()` / `download()` 转换。

3.3.2. 设备内存管理

3.3.2.1. 缓存模式与对齐

```
struct esMat::AllocConfig {
    int cacheMode = ES_CACHE_NOCACHE; // 缓存策略
    int alignMode = ES_ALIGN_64;      // 对齐字节数
};

// 创建带配置的 esMat
esMat::AllocConfig config;
config.cacheMode = ES_CACHE_CACHED; // CPU 频繁访问时启用（需手动调用 flush 同步）
config.alignMode = ES_ALIGN_64;      // 对齐模式，VENC 要求至少 64 字节

// rows 在前、cols 在后；1920x1080 图像应传入 rows=1080, cols=1920
esMat frame = esMat::createWithConfig(1080, 1920, ES_PIXFMT_NV12, config);
// 也可以使用 cv::Size(width, height) 避免写反
esMat frame2 = esMat::createWithConfig(cv::Size(1920, 1080), ES_PIXFMT_NV12, config);
```

缓存模式说明：

模式	适用场景	CPU 访问性能	DMA 性能	注意事项
ES_CACHE_NO-CACHE	纯硬件处理（解码→编码）	慢	最优	无需 flush
ES_CACHE_CACHED	CPU 需要读取内容（分析/调试）	快	良好	必须手动 flush

重要：使用 `ES_CACHE_CACHED` 时，必须调用 `flush` 接口同步缓存：

- **CPU 写入后：**调用 `flush` 确保数据从 `cache` 写回内存（设备才能看到）
- **设备写入后：**CPU 读取前调用 `flush` 使 `cache` 无效（CPU 才能读到内存中的最新数据）
- 参考：`ES_SYS_MemFlushCache` 接口（详见 `Memory` 用户手册）

对齐要求：

- VPS 输入：至少 16 字节对齐
- VENC 输入：至少 64 字节对齐（支持 0/1/64/128/256/512/1024/2048/4096）
- VDEC 输出：驱动自动保证对齐（支持 1/8/16/32/64/128/256/512/1024/2048）

3.3.3. CPU 与设备内存交互

3.3.3.1. upload:CPU 到设备

```
void esMat::upload(InputArray src);
```

将 CPU 内存（`cv::Mat`、`std::vector` 等）上传到设备内存：

```
cv::Mat cpuImg = cv::imread("image.jpg"); // CPU 内存

cv::es::esMat hwImg;
hwImg.upload(cpuImg); // 上传到设备内存
```

3.3.3.2. download: 设备到 CPU

```
void esMat::download(OutputArray dst) const;
```

从设备内存下载到 CPU 内存：

```
cv::es::esMat hwImg; // 从硬件解码器获取
reader->nextFrame(hwImg);

cv::Mat cpuImg;
hwImg.download(cpuImg); // 拷贝数据到 CPU 内存

cv::imwrite("output.jpg", cpuImg);
```

说明：download 会将设备内存中的数据拷贝到 CPU 内存，存在数据传输开销。

优化建议：尽量避免 download，在设备侧完成处理。

3.3.3.3. mapToCpuMat: 映射访问

```
Mat esMat::mapToCpuMat();
```

DMA-backed `esMat` 会将设备内存映射到 CPU 地址空间并返回 `cv::Mat` 视图；software AVFrame 路径会导出 packed CPU 缓冲：

```
cv::es::esMat hwImg;
reader->nextFrame(hwImg);

// 映射到 CPU 视图（DMA-BUF 路径不执行数据拷贝）
cv::Mat cpuView = hwImg.mapToCpuMat();

// 可以直接访问像素数据
std::cout << "像素值: " << (int)cpuView.at<uchar>(0, 0) << std::endl;
```

重要说明：

分配模式	cache 同步	性能特点
ES_CACHE_NOCACHE	不需要	CPU 直接访问物理内存，无 cache 开销
ES_CACHE_CACHED	mapToCpuMat()/download 会调用 flushCache()	直接通过 ptr()/dataPtr() 访问时需按读写方向自行同步 cache

使用建议：

- 如果只需要偶尔读取少量数据，DMA-BUF 路径下可使用 `mapToCpuMat()` 避免整帧拷贝；software AVFrame 路径会返回 packed CPU 缓冲

- 对于需要频繁 CPU 访问的场景，使用 `ES_CACHE_CACHED` 分配模式
- 确保硬件操作（如解码、VPS 处理）完成后再访问映射的数据

```
// 读取像素（通过 mmap 访问设备内存）
for (int y = 0; y < cpuView.rows; y++) {
    uint8_t* row = cpuView.ptr(y);
    // 处理 row 数据（速度较慢，因为访问的是设备内存）
}

// 注意：DMA-BUF 路径下 cpuView 只是视图，不能超过 hwImg 及其底层存储的生命周期使用；software AVFrame
// 路径返回的是 packed CPU 缓冲
```

限制：

1. 仅适用于需要随机访问少量数据的场景
2. 大量数据访问建议使用 `download`
3. DMA-BUF 路径下映射后的 Mat 仅在 `esMat` 生命周期内有效；`software AVFrame` 路径返回 `packed CPU` 缓冲

3.3.4. 像素格式体系

3.3.4.1. OpenCV 标准类型

`esMat` 与 `cv::Mat` 兼容部分 OpenCV 标准类型：

```
// 支持的 OpenCV 类型
esMat gray(rows, cols, CV_8UC1); // 8 位灰度
esMat bgr(rows, cols, CV_8UC3); // 8 位 BGR
esMat bgra(rows, cols, CV_8UC4); // 8 位 BGRA
esMat float_img(rows, cols, CV_32FC3); // 32 位浮点 3 通道
```

3.3.4.2. EsPixelFormat 扩展类型

对于硬件原生支持的 YUV 格式，使用 `EsPixelFormat`：

```
namespace cv { namespace es {
    enum EsPixelFormat {
        // YUV 4:2:0 平面格式
        ES_PIXFMT_I420 = 10321, // YUV420P (Y 平面 + U 平面 + V 平面)
        ES_PIXFMT_YV12 = 10320, // YVU420P (Y 平面 + V 平面 + U 平面)
        ES_PIXFMT_NV12 = 10322, // YUV420SP (Y 平面 + UV 交织)
        ES_PIXFMT_NV21 = 10323, // YVU420SP (Y 平面 + VU 交织)

        // YUV 4:2:2 格式
        ES_PIXFMT_YUY2 = 10300, // YUYV packed
        ES_PIXFMT_UYVY = 10301, // UYVY packed
        ES_PIXFMT_NV16 = 10310, // YUV422SP

        // 10 位格式
        ES_PIXFMT_YUV420P010LE = 10330, // 10-bit YUV420P
        ES_PIXFMT_YUV420SP010LE = 10334, // P010 (10-bit NV12)

        // RGB 平面格式（VPS 输出）
        ES_PIXFMT_B8G8R8_PLANAR = 10221, // BGR planar (8-bit)
        ES_PIXFMT_B32G32R32F_PLANAR = 10609, // BGR planar (32-bit float)
    };
}; }
```

```

    // ... 更多格式见 es_types.hpp
};
}

```

创建特定格式的 esMat:

```

// 创建 NV12 格式
cv::es::esMat nv12Frame(1080, 1920, cv::es::ES_PIXFMT_NV12);

// 创建 P010 格式 (10 位 HDR)
cv::es::esMat hdrFrame(2160, 3840, cv::es::ES_PIXFMT_YUV420SP010LE);

// 创建 BGR float planar (AI 推理输入)
cv::es::esMat aiInput(640, 640, cv::es::ES_PIXFMT_B32G32R32F_PLANAR); // 方形图像 rows=cols=640

```

3.3.4.3. 格式转换

格式转换通过 esimgproc 模块后端实现:

```

#include <opencv2/esimgproc.hpp>

cv::es::esMat frame, bgr;
reader->nextFrame(frame); // 默认 I420 格式 (可通过 ESVideoReaderInitParams 配置输出格式)

// 格式转换 (通过 esimgproc 后端执行)
cv::esimgproc::ProcessParams params;
params.common.output_format = cv::es::ES_PIXFMT_B8G8R8; // 指定输出 BGR
cv::esimgproc::process(frame, bgr, params);

```

3.3.5. ROI 与视图操作

esMat 支持 ROI (Region of Interest) 操作, 类似 cv::Mat:

3.3.5.1. 创建 ROI 视图

```

cv::es::esMat fullFrame(1080, 1920, cv::es::ES_PIXFMT_NV12);

// 方式 1: 使用 Rect
cv::Rect roi(100, 100, 640, 480);
cv::es::esMat roiView = fullFrame(roi);

// 方式 2: 使用 Range
cv::es::esMat roiView2 = fullFrame(Range(100, 580), Range(100, 740));

// 方式 3: 行/列切片
cv::es::esMat topHalf = fullFrame.rowRange(0, 540);
cv::es::esMat leftHalf = fullFrame.colRange(0, 960);

```

ROI 特性:

1. **浅拷贝**: ROI 与原 esMat 共享底层存储和引用计数
2. **零拷贝视图**: ROI 通过调整 rows/cols/local_offset 形成子视图, 不复制像素数据
3. **生命周期**: 任一视图仍持有引用时, 底层存储不会释放; 不要在底层存储失效后继续使用 ROI

3.3.5.2. ROI 编码示例

```
cv::es::esMat fullFrame;
reader->nextFrame(fullFrame); //1920x1080

// 仅编码右下角区域
cv::Rect roi(960, 540, 960, 540);
cv::es::esMat roiFrame = fullFrame(roi);

// 创建编码器 (960x540)
cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H265;
params.pixelFormat = cv::es::ES_PIXFMT_I420;
params.useHardwareInput = true;

cv::Ptr<cv::escodec::VideoWriter> writer =
    cv::escodec::createESVideoWriter("output.mp4", cv::Size(960, 540), 30.0, params);

writer->write(roiFrame); // 硬件编码器处理 ROI
```

内部实现：ROI 由 rows/cols/step/local_offset 描述子区域，当前 esMat 视图操作本身不等同于设置 AVFrame 裁剪字段或执行裁剪滤镜；需要实际硬件裁剪时，建议使用 esimgproc::crop() 或 process() 中的 crop_rect。

3.3.6. API 参考表

3.3.6.1. 构造与创建类 API

API	功能	是否拷贝	关键点
esMat()	空构造函数	-	创建空对象，不分配内存
esMat(rows, cols, type)	尺寸构造	-	自动分配 DMA-BUF 设备内存
esMat(Size size, type)	Size 构造	-	同上，使用 cv::Size 指定尺寸
esMat(rows, cols, type, Scalars)	初始化构造	-	分配内存并填充初始值
esMat(InputArray arr)	从数组构造	✓	从 cv::Mat/esMat 构造，涉及数据拷贝
esMat(rows, cols, type, fd, step, offset)	从 DMA-FD 构造	-	包装现有 DMA-BUF，零拷贝
esMat(AVFrame* frame, bool ownership)	从 AVFrame 构造	-	包装 AVFrame，可选接管所有权
esMat(Allocator* allocator)	自定义分配器构造	-	使用指定内存分配器
create(rows, cols, type)	创建设备内存	-	重新分配 DMA-BUF，释放旧内存
createWithConfig(size, type, config)	带配置创建	-	指定缓存模式 (NOCACHE/CACHED) 和对齐模式 (ES_ALIGN_1 到 ES_ALIGN_2048)
clone()	深拷贝	✓	创建独立副本，分配新 DMA-BUF，完整数据拷贝

3.3.6.2. 数据传输类 API

API	功能	是否拷贝	关键点
upload(src)	CPU → 设备上传	✓	从 cv::Mat 拷贝到 esMat 设备内存
download(dst)	设备 → CPU 下载	✓	从 esMat 拷贝到 cv::Mat 主机内存
copyTo(dst)	拷贝到目标	✓	支持 esMat → esMat、esMat → Mat，完整拷贝
mapToCpuMat()	映射为 CPU 可访问	DMA-BUF 路径否	返回 cv::Mat 视图；DMA-BUF 路径共享内存，software AVFrame 路径返回 packed CPU 缓冲
ptr(y)	获取行指针	-	返回 CPU 虚拟地址 (mmap)，不拷贝；读取设备写入数据前需保证 cache 同步
dataPtr()	获取数据指针	-	返回数据起始地址 (CPU 虚拟地址)，不自动做额外拷贝

3.3.6.3. ROI 与视图类 API

API	功能	是否拷贝	关键点
row(y)	获取单行	-	浅拷贝，共享底层存储
col(x)	获取单列	-	浅拷贝，共享底层存储
rowRange(start, end)	行范围	-	浅拷贝，ROI 引用
colRange(start, end)	列范围	-	浅拷贝，ROI 引用
operator()(Range, Range)	Range 索引	-	浅拷贝，ROI 引用
operator()(Rect roi)	Rect 索引	-	浅拷贝，ROI 引用
reshape(cn, rows)	重塑形状	-	改变通道数/行数，不改变数据
adjustROI(dtop, dbottom, dleft, dright)	调整 ROI 边界	-	扩展/收缩 ROI 区域
locateROI(wholeSize, ofs)	定位 ROI	-	查询 ROI 在完整图像中的位置

3.3.6.4. 平面操作类 API

API	功能	是否拷贝	关键点
getPlane(idx)	获取单个平面	-	DMA-backed esMat 返回平面视图；software AVFrame 路径会导出并拷贝该平面
planeCount()	平面数量	-	返回像素格式的平面数 (I420=3, NV12=2)
planeHeights()	各平面高度	-	返回每个平面的高度数组
planeWidths()	各平面宽度	-	返回每个平面的宽度数组
planeStrides()	各平面 stride	-	返回每个平面的行字节数
planeOffsets()	各平面偏移	-	返回每个平面的起始偏移量

API	功能	是否拷贝	关键点
setPlaneLayout(steps, offsets)	设置平面布局	-	手动配置平面内存布局

3.3.6.5. 属性查询类 API

API	功能	返回值	说明
empty()	是否为空	bool	检查是否已分配内存
size()	图像尺寸	Size	返回宽高
rows / cols	行数/列数	int	公共成员变量
type()	OpenCV 类型	int	CV_8UC3 等类型常量
depth()	数据深度	int	CV_8U/CV_16U/CV_32F 等
channels()	通道数	int	1/3/4 等
elemSize()	元素字节数	size_t	单个像素字节数（含所有通道）
elemSize1()	单通道字节数	size_t	单通道元素字节数
step	行字节数	size_t	公共成员变量，一行的字节数
step1(i)	以元素为单位的 step	size_t	step/elemSize1()
total()	元素总数	int	rows × cols
dims()	维度数	int	通常为 2（二维图像）
isContinuous()	是否连续	bool	内存是否连续无 padding
pixelFormat()	像素格式	int	ES_PIXFMT_XXX 常量
cacheMode()	缓存模式	int	ES_CACHE_NOCACHE/ES_CACHE_CACHED
alignMode()	对齐模式	int	ES_ALIGN_64/ES_ALIGN_256
dmaFd()	DMA 文件描述符	int64	用于传递给硬件单元
offset()	数据偏移量	size_t	ROI 场景下的起始偏移
getStep()	获取 step	size_t	同 step 成员

3.3.6.6. 缓存与同步类 API

API	功能	关键点
flushCache()	刷新缓存	仅 CACHED 模式需要，NOCACHE 模式自动跳过
setTo(s)	填充值	将所有元素设置为指定值

3.3.6.7. AVFrame 互操作类 API

API	功能	是否拷贝	关键点
setAvFrame(frame, ownership)	绑定 AVFrame	-	将 AVFrame 绑定到 esMat，可选接管所有权

API	功能	是否拷贝	关键点
getAvFrame(add_ref)	导出 AVFrame	-	导出 AVFrame 指针，add_ref=true 增加引用计数
getAvFrameType()	获取帧类型	-	返回硬件/软件帧类型枚举

3.3.6.8. 内存管理类 API

API	功能	关键点
release()	释放内存	手动释放 DMA-BUF，通常由析构自动调用
swap(b)	交换内容	交换两个 esMat 的内部指针，零拷贝
defaultAllocator()	获取默认分配器	静态方法，返回全局默认分配器
setDefaultAllocator(allocator)	设置默认分配器	静态方法，配置全局分配器
defaultConfig()	获取默认配置	静态方法，返回缓存/对齐默认配置
setDefaultConfig(config)	设置默认配置	静态方法，配置全局默认参数

3.3.6.9. 全局工具函数

API	功能	是否拷贝	关键点
toCvMatView(esmat)	esMat → Mat 视图	-	零拷贝包装，返回 cv::Mat 视图
fromCvMatView(mat)	Mat 视图→ esMat	-	仅接受由 toCvMatView() 创建且带有 esMat 元数据的 Mat 视图
toCvMatWithEsBackend(esmat)	使用 es 后端的 Mat	-	创建使用 esMat 作为后端的 cv::Mat
fromCvMatWithEsBackend(mat)	从 es 后端提取 esMat	-	从特殊 cv::Mat 提取 esMat
createContinuous(rows, cols, type, arr)	创建连续内存	-	确保 esMat 内存连续（无 padding）
ensureSizeEnough(rows, cols, type, arr)	确保尺寸足够	-	当前实现仅在尺寸/类型满足时直接返回；尺寸不足场景建议显式调用 create() / createWithConfig() 重新分配
cvTypeToEsPixelFormat(cv_type)	CV 类型→像素格式	-	类型转换工具
esPixelFormatToCvType(fmt)	像素格式→ CV 类型	-	类型转换工具
getPlaneDepth(fmt, plane_idx)	获取平面深度	-	返回指定平面的位深度

3.3.6.10. 数据拷贝总结

涉及拷贝的操作（性能开销较大）：

- upload() / download() - CPU 与设备间数据传输

- `copyTo()` - 显式拷贝到另一个容器
- `clone()` - 深拷贝创建独立副本
- `esMat(InputArray arr)` - 从数组构造时拷贝

零拷贝操作（推荐优先使用）：

- ROI 操作: `row()`, `col()`, `rowRange()`, `colRange()`, `operator()(Rect)`
- 视图操作: `reshape()`, `adjustROI()`；DMA-backed 路径下 `getPlane()` 返回平面视图
- 映射操作: DMA-backed 路径下的 `mapToCpuMat()`, `ptr()`, `toCvMatView()`
- 包装操作: `esMat(fd, ...)`, `esMat(AVFrame*)`, `setAvFrame()`
- 引用操作: `swap()`, `operator=`（浅拷贝）

3.3.6.11. 像素格式常量

格式常量	说明	典型用途
ES_PIXFMT_I420	YUV420P planar	编解码默认格式，3 平面 (Y/U/V 分离)
ES_PIXFMT_NV12	YUV420SP	2 平面 (Y 单独/UV 交错)，VPS 常用
ES_PIXFMT_NV21	YUV420SP 变种	Android 相机常用
ES_PIXFMT_YUY2	YUV422 packed	相机输入常用
ES_PIXFMT_B8G8R8A8	BGRA8888	OpenGL 纹理、显示
ES_PIXFMT_B8G8R8	BGR24	OpenCV 标准格式
ES_PIXFMT_GRAY8	灰度 8 位	单通道图像
ES_PIXFMT_YUV420SP010LE	YUV420SP 10 位 (P010)	HDR 内容

3.3.6.12. 缓存模式与对齐

模式常量	说明	使用场景
ES_CACHE_NOCACHE	不使用 CPU 缓存	硬件间零拷贝（推荐）
ES_CACHE_CACHED	使用 CPU 缓存	CPU 频繁读写场景
ES_ALIGN_64	64 字节对齐	标准对齐（默认）
ES_ALIGN_256	256 字节对齐	某些硬件单元要求

关键设计原则：

1. **零拷贝优先**: 硬件链路中尽量使用 ROI 和 DMA-BUF 路径传递，避免不必要的 upload/download；CPU 密集处理仍建议 download 到普通 CPU 内存
2. **缓存模式选择**: 硬件间传递用 NOCACHE，CPU 密集计算用 CACHED
3. **生命周期管理**: 通过引用计数自动管理，类似 `std::shared_ptr`
4. **格式一致性**: 编解码链路保持格式一致，避免隐式转换

3.4. 视频解码

3.4.1. VideoReader: 硬件解码器

`VideoReader` 提供硬件加速的视频解码功能，支持本地文件和网络流。

3.4.1.1. 创建解码器

```
namespace cv { namespace escodec {

// 工厂函数
CV_EXPORTS_W Ptr<VideoReader> createVideoReader(
    const std::string& src,
    const ESVideoReaderInitParams& params = ESVideoReaderInitParams()
);

}}
```

基础用法：

```
#include <opencv2/escodec.hpp>

// 本地文件
cv::Ptr<cv::escodec::VideoReader> reader =
    cv::escodec::createVideoReader("video.mp4");

// RTSP 网络流
cv::Ptr<cv::escodec::VideoReader> rtspReader =
    cv::escodec::createVideoReader("rtsp://192.168.1.100:8554/stream");

// RTMP 流
cv::Ptr<cv::escodec::VideoReader> rtmpReader =
    cv::escodec::createVideoReader("rtmp://server/live/stream");
```

3.4.1.2. 解码参数配置

```
struct ESVideoReaderInitParams {
    int minNumDecodeSurfaces = 2;    // 最小解码表面数
    bool allowFrameDrop = false;    // 是否允许丢帧
    bool udpSource = false;    // UDP 源（实时流优化）
    int strideAlign = 64;    // 行步长对齐
    int bufCacheMode = 1;    // 缓冲区缓存模式
    int decodeSurfaceFormat = cv::es::ES_PIXFMT_I420; // 解码输出 surface 布局
    bool enableHistogram = false;    // 启用直方图统计
    int firstFrameIdx = 0;    // 起始帧索引
    bool rawMode = false;    // 原始模式
};
```

关键参数说明：

- `decodeSurfaceFormat`：支持 I420/NV12/NV21/YUV420SP010LE/GRAY8/BGR/RGB/BGRA/RGBA/B-GRx/RGBx（详见[解码器输出格式](#)）
- `strideAlign`：YUV 行步长对齐，范围 [8, 2048]，默认 64 字节
- `bufCacheMode`：缓冲区缓存策略，0=无缓存，1=默认缓存
- `udpSource` + `allowFrameDrop`：实时流场景设置为 true 可降低延迟
- 当前 `VideoReader` 不在初始化阶段执行缩放或 ROI 裁剪；如需缩放或裁剪，应使用 `esimgproc` 模块处理解码后的 `esMat`。

3.4.1.3. 解码循环

```
cv::es::esMat frame;

// 方式 1: nextFrame (推荐)
while (reader->nextFrame(frame)) {
    // 处理 frame (硬件内存)
    processFrame(frame);
}

// 方式 2: grab/retrieve (需要细粒度控制时)
while (reader->grab()) {
    if (someCondition) {
        reader->retrieve(frame);
        processFrame(frame);
    }
}
```

nextFrame vs grab/retrieve:

方法	适用场景	延迟	灵活性
nextFrame	通用场景	低	简单
grab + retrieve	需要选择性解码时	稍高	灵活

3.4.1.4. 查询视频信息

```
cv::escodec::ESFormatInfo fmt = reader->format();

std::cout << "分辨率: " << fmt.width << "x" << fmt.height << std::endl;
std::cout << "帧率: " << fmt.avgFrameRate << " fps" << std::endl;
std::cout << "编码格式: " << fmt.codec << std::endl; // ESCODEC_H264/H265/MJPEG 等
std::cout << "像素格式: " << fmt.surfaceFormat << std::endl;
std::cout << "位深度: " << fmt.bitDepth << std::endl;
```

注意：ESFormatInfo 结构体的详细定义请参考[解码器 API 参考表](#)中的说明。

3.4.2. 支持的解码格式

格式	解码支持	Profile	分辨率范围	备注
H.264	支持	Baseline/Main/High	48x48 ~ 8192x8192	主流格式
H.265	支持	Main	144x144 ~ 8192x8192	高压缩率
MJPEG	支持	-	48x48 ~ 32768x32768	帧内编码

其他格式解码：

除 H.264/H.265/MJPEG 硬件解码路径外，其他编码格式可由 FFmpeg 后端尝试使用软件解码器处理。具体是否可用取决于当前系统中的 FFmpeg 编译配置、解码器插件和输入码流特性。

注意：

软件解码性能低于硬件解码，建议生产环境优先使用 H.264/H.265/MJPEG 硬件解码路径。对于 VP8/VP9/AV1/MPEG-2/MPEG-4 等格式，应以目标系统的 FFmpeg 能力为准。

3.4.2.1. 容器格式支持

容器	文件扩展名	支持的编码格式	备注
MP4	.mp4	H.264/H.265/MJPEG	推荐，兼容性最好
Matroska	.mkv	H.264/H.265/MJPEG	开源格式
AVI	.avi	H.264/MJPEG	老格式，兼容性好
FLV	.flv	H.264	Flash 流媒体
MPEG-TS	.ts	H.264/H.265	广播标准
Raw	.h264/.h265	H.264/H.265	无容器

3.4.2.2. 解码器输出格式

通过 `ESVideoReaderInitParams::decodeSurfaceFormat` 配置：

```
cv::escodec::ESVideoReaderInitParams params;
params.decodeSurfaceFormat = cv::es::ES_PIXFMT_NV12; // 显式设置为 NV12（默认是 I420）

cv::Ptr<cv::escodec::VideoReader> reader =
    cv::escodec::createVideoReader("input.mp4", params);
```

支持的输出格式：

格式	零拷贝支持	常量名	备注
YUV 格式			
I420	支持	ES_PIXFMT_I420	FFmpeg 标准，解码器默认格式
NV12	支持	ES_PIXFMT_NV12	推荐，Android 标准
NV21	支持	ES_PIXFMT_NV21	Android 常用变种
YUV420SP010LE	支持	ES_PIXFMT_YUV420SP010LE	10-bit NV12，HDR 解码
灰度格式			
GRAY8	支持	ES_PIXFMT_GRAY8	8 位灰度
RGB 格式			
BGR	支持	ES_PIXFMT_B8G8R8	24 位 BGR
RGB	支持	ES_PIXFMT_R8G8B8	24 位 RGB
BGRA	支持	ES_PIXFMT_B8G8R8A8	32 位 BGRA
RGBA	支持	ES_PIXFMT_R8G8B8A8	32 位 RGBA
BGRx	支持	ES_PIXFMT_B8G8R8X8	32 位 BGR+填充
RGBx	支持	ES_PIXFMT_R8G8B8X8	32 位 RGB+填充

注意：

- YUV 格式零拷贝性能最佳，推荐使用 NV12
- `decodeSurfaceFormat` 仅请求解码器输出 surface 布局，不执行 YUV 到 RGB/BGR 的后处理色彩空间转换

- RGB/BGR/BGRA/RGBA 等 packed 格式依赖解码器 `dec_pixfmt` 能力；如需明确的颜色空间转换，应在解码后使用 `esimgproc` 处理
- 默认格式为 I420，可通过 `decodeSurfaceFormat` 参数设置其他格式

3.4.3. 解码器 API 参考表

3.4.3.1. VideoReader API

API	功能	关键点
<code>createVideoReader(src)</code>	创建解码器（简化版）	<code>src</code> 为本地文件路径或流 URI，使用默认参数
<code>createVideoReader(src, params)</code>	创建解码器（完整版）	支持自定义解码输出格式、缓存、起始帧和实时流参数
<code>nextFrame(frame)</code>	读取下一帧	返回 bool，false 表示流结束，frame 为 esMat 输出
<code>format()</code>	获取视频信息	返回 ESFormatInfo(宽高/帧率/编码格式/像素格式)
<code>get(propId)</code>	查询属性	支持 PROP_FRAME_WIDTH/HEIGHT/FPS/COUNT/POS 等
<code>set(propId, value)</code>	设置属性	支持部分属性（如跳帧）
<code>grab()</code>	抓取帧（不解码）	快速跳帧场景
<code>retrieve(frame)</code>	解码已抓取帧	配合 grab 使用

3.4.3.2. ESVideoReaderInitParams

解码器初始化参数结构体。

参数	类型	默认值	说明
<code>decodeSurfaceFormat</code>	int	ES_PIXFMT_I420	请求解码输出 surface 布局；不执行后处理色彩空间转换
<code>strideAlign</code>	int	64	YUV stride 对齐，范围 [8, 2048]
<code>bufCacheMode</code>	int	1	解码 buffer 缓存模式（0=无缓存, 1=默认缓存）
<code>allowFrameDrop</code>	bool	false	是否允许丢帧（实时流场景）
<code>udpSource</code>	bool	false	UDP 源标识（实时流优化）
<code>minNumDecodeSurfaces</code>	int	2	最小解码表面数（帧缓冲数量）
<code>enableHistogram</code>	bool	false	是否启用直方图统计
<code>firstFrameIdx</code>	int	0	起始帧索引
<code>rawMode</code>	bool	false	原始模式（跳过某些后处理）

3.4.3.3. ESFormatInfo

解码器格式信息结构体，由 `VideoReader::format()` 返回。

字段	类型	默认值	说明
codec	int	ESCODEC_UNKNOWN	编码格式 (ESCODEC_H264/ H265/ MJPEG 等)
width / height	int	0	视频分辨率
avgFrameRate	double	0.0	平均帧率
bitDepth	int	0	位深度 (8/10/12 位)
surfaceFormat	int	ES_PIXFMT_UNKNOWN	解码输出像素格式
videoFullRangeFlag	bool	false	是否全范围 YUV (true=0-255, false=16-235)
enableHistogram	bool	false	是否启用直方图统计
nMaxHistogramBins	int	0	直方图最大 bin 数量

关键设计原则：

- 1. **零拷贝输出**：解码器输出 `esMat` 位于设备内存，可直接送入 VPS 或编码器
- 2. **格式选择**：推荐使用 NV12 格式，兼容性和性能最佳。默认为 I420，需通过 `decodeSurfaceFormat` 显式配置
- 3. **Stride 对齐**：`strideAlign` 默认 64 字节，某些硬件模块可能需要更大对齐（如 128/256）
- 4. **实时流优化**：设置 `udpSource=true` + `allowFrameDrop=true` 可降低网络流延迟

3.5. 视频编码

3.5.1. VideoWriter: 硬件编码器

`VideoWriter` 提供硬件加速的视频编码功能，支持文件输出和流媒体推送。C++ 中 `cv::escodec::VideoWriter` 是 `cv::escodec::ESVideoWriter` 的别名，用于区别标准 OpenCV 的 `cv::VideoWriter`。

3.5.1.1. 创建编码器

```
namespace cv { namespace escodec {  
  
    // 基础工厂函数  
    CV_EXPORTS Ptr<ESVideoWriter> createVideoWriter(  
        const String& filename,  
        int codec,           // 编码格式 (CODEC_H264/CODEC_H265/CODEC_MJPEG)  
        Size frameSize,  
        double fps  
    );  
  
    // 简化工厂函数 (默认 H.265)  
    CV_EXPORTS_W Ptr<ESVideoWriter> createESVideoWriter(  
        const String& filename,  
        Size frameSize,  
        double fps,  
        int codec = CODEC_H265 // 默认使用 H.265  
    );  
  
    // EncoderParams 方式 (推荐)
```

```

CV_EXPORTS Ptr<ESVideoWriter> createVideoWriter(
    const String& filename,
    Size frameSize,
    double fps,
    const EncoderParams& params = EncoderParams()
);

// EncoderParams 方式 (Python 兼容命名)
CV_EXPORTS_W Ptr<ESVideoWriter> createESVideoWriter(
    const String& filename,
    Size frameSize,
    double fps,
    const EncoderParams& params
);

// 回调式编码器
CV_EXPORTS Ptr<ESVideoWriter> createVideoWriter(
    const Ptr<EncoderCallback>& encoderCallback,
    Size frameSize,
    int codec,
    double fps,
    const EncoderParams& params
);
}

```

说明： `createESVideoWriter(filename, frameSize, fps, codec)` 简化接口默认按硬件输入帧路径创建编码器，适用于解码器输出的硬件 `esMat`。如输入为软件帧，或需要显式配置 `pixelFormat`、码率、GOP 等参数，应使用 `EncoderParams` 重载。

基础用法：

```

// H.264 编码到 MP4 (推荐使用 EncoderParams 重载)
cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H264;
params.pixelFormat = cv::es::ES_PIXFMT_I420; // 与输入 esMat 像素格式保持一致
params.useHardwareInput = true;

cv::Ptr<cv::escodec::VideoWriter> writer =
    cv::escodec::createVideoWriter(
        "output.mp4",
        cv::Size(1920, 1080),
        30.0,
        params
    );

// H.265 编码到 MKV (使用简化接口，默认 H.265)
cv::Ptr<cv::escodec::VideoWriter> hevcWriter =
    cv::escodec::createESVideoWriter(
        "output.mkv",
        cv::Size(3840, 2160),
        25.0
    );

// MJPEG 编码
params = cv::escodec::EncoderParams();
params.codec = cv::escodec::CODEC_MJPEG;
params.pixelFormat = cv::es::ES_PIXFMT_I420;
params.useHardwareInput = true;

```

```
cv::Ptr<cv::escodec::VideoWriter> mjpegWriter =
    cv::escodec::createVideoWriter(
        "output.avi",
        cv::Size(1280, 720),
        30.0,
        params
    );
```

3.5.1.2. 编码循环

```
cv::es::esMat frame;

while (reader->nextFrame(frame)) {
    // 零拷贝编码 (frame 来自硬件解码器)
    writer->write(frame);
}

// 完成编码并关闭文件
writer->release();
```

注意事项：

- 1. write() 会校验输入帧并将当前帧送入编码器；编码器内部可能缓存延迟帧
- 2. release() 会 flush 编码器内部缓存并写入文件尾，可能耗时较长
- 3. 输入帧必须与创建时指定的分辨率一致

3.5.2. 编码参数配置

编码器参数详解：

EncoderParams 结构提供完整的编码配置选项。应用通常只需要设置编码格式、码率控制、GOP、输入像素格式和硬件输入方式，其余字段可保留默认值。

3.5.2.1. 基础编码参数

参数	类型	默认值	说明
codec	int	CODEC_H264	编码格式 (CODEC_H264/CODEC_H265/CODEC_MJPEG)
bitrate	int	2000	目标码率 (kbps)
rcMode	int	RC_VBR	码率控制模式 (RC_CBR/RC_VBR/RC_CQP)
gopSize	int	75	GOP 大小 (I 帧间隔)
qp	int	-1	CQP 模式的固定 QP 值, -1 表示自动
enableLowLatency	bool	false	低延迟模式
pixelFormat	int	ES_PIXFMT_I420	输入像素格式 (必须与 esMat 一致)
bitDepth	int	8	位深 (8/10)
profile	int	-1	Codec profile (H264/HEVC), -1 表示自动
level	int	0	Codec level (H264/HEVC), 0 表示自动
useHardwareInput	bool	false	使用 DRM_PRIME 硬件帧输入

说明： pixelFormat 必须与输入 esMat 的像素格式一致。使用硬件零拷贝输入时， useHardwareInput 应设置为 true ，且输入帧必须是硬件 AVFrame。

3.5.2.2. MJPEG 特定参数

参数	类型	默认值	说明
qfactor	int	50	MJPEG 质量因子 (1-99, 越小质量越好)
qfactorMin	int	-1	MJPEG 最小质量因子
qfactorMax	int	-1	MJPEG 最大质量因子

3.5.2.3. 高级码率控制参数

参数	类型	默认值	说明
iqp / pqp / bqp	int	30/32/32	I/P/B 帧量化参数 (0-51)
qpMin / qpMax	int	-1	QP 最小/最大值
qpIMin / qpIMax	int	-1	I 帧 QP 最小/最大值
iprop	int	-1	I 帧比例 [50,100], -1 表示自动
cpbSize	int	-1	CPB 缓冲区大小 (kbps), -1 表示自动
statTime	int	1	码率统计时间窗口 (秒), 范围 [1,60]

3.5.2.4. GOP 和帧结构参数

参数	类型	默认值	说明
gopMode	int	0	GOP 模式
bFrmNum	int	2	B 帧数量 (1-3)
ipQpDelta	int	2	IP 帧 QP 差值
iQpDelta	int	2	I 帧 QP 增量
sbInterval	int	0	特殊 B 帧间隔
spQpDelta	int	0	特殊 B 帧 QP 差值
bgInterval	int	-1	长参考帧间隔
bgQpDelta	int	5	长参考帧 QP 差值

3.5.2.5. 编码工具参数

参数	类型	默认值	说明
enableCabac	int	0	H264 CABAC 开关 (0=CAVLC, 1=CABAC)

参数	类型	默认值	说明
enableDeblock	int	1	去块滤波器 (0=禁用, 1=启用)
tier	int	0	HEVC 层级 (0=Main Tier, 1=High Tier)

3.5.2.6. 其他参数

参数	类型	默认值	说明
strideAlign	int	1	YUV stride 对齐
outFps	int	25	输出帧率覆盖 (≤ 0 使用构造时的 fps)
timestampMode	int	0	PTS 生成模式: 0=AUTO(帧计数), 1=PASSTHROUGH(透传输入 PTS)

编码参数示例:

```
cv::escodec::EncoderParams params;

// 配置 1: 高质量 H.264
params.codec = cv::escodec::CODEC_H264;
params.bitrate = 6000;    // 6 Mbps (kbps)
params.rcMode = cv::escodec::RC_VBR;
params.gopSize = 60;     // 2 秒一个 I 帧 (30fps)

cv::Ptr<cv::escodec::ESVideoWriter> hqWriter =
    cv::escodec::createVideoWriter("hq_output.mp4",
        cv::Size(1920, 1080), 30.0, params);

// 配置 2: 低码率 CBR (适合网络直播)
params.codec = cv::escodec::CODEC_H264;
params.bitrate = 2000;    // 2 Mbps (kbps)
params.rcMode = cv::escodec::RC_CBR;
params.gopSize = 30;     // 1 秒一个 I 帧 (30fps)

cv::Ptr<cv::escodec::ESVideoWriter> streamWriter =
    cv::escodec::createVideoWriter("rtmp://server/live/stream",
        cv::Size(1280, 720), 30.0, params);

// 配置 3: 固定 QP (质量优先)
params.codec = cv::escodec::CODEC_H264;
params.qp = 23;          // 固定 QP=23 (高质量, 范围 0-51)
params.rcMode = cv::escodec::RC_CQP;
cv::Ptr<cv::escodec::ESVideoWriter> fixQpWriter =
    cv::escodec::createVideoWriter("fixqp_output.mp4",
        cv::Size(1920, 1080), 30.0, params);
```

3.5.3. 回调式编码

回调式编码允许自定义码流处理逻辑, 适用于:

- RTMP/RTP 实时推流
- 自定义容器格式
- 原始码流分析

- 多路并发编码

3.5.3.1. EncoderCallback 接口

```
class EncoderCallback {
public:
    /** @brief 编码完成回调
     * @param vPacket 编码后的数据包（一个或多个 NAL 单元）
     * @param pts 每个数据包的呈现时间戳（微秒）
     *
     * 对于 H.264/H.265，每个数据包可能包含：
     * - SPS/PPS/VPS（序列/图像/视频参数集）
     * - IDR/I 帧
     * - P 帧
     * - NAL 起始码
     */
    virtual void onEncoded(
        const std::vector<std::vector<uint8_t>>& vPacket,
        const std::vector<uint64_t>& pts
    ) = 0;

    /** @brief 编码结束回调
     *
     * 在 flush() 完成后调用，用于最终清理和资源释放。
     */
    virtual void onEncodingFinished() = 0;

    /** @brief 动态调整 GOP 结构
     * @param frameIntervalP P 帧间隔
     * - 0: 全 I 帧模式 (IPPP...)
     * - 1: IP 模式 (IPIP...)
     * - N: 每个 I 帧后有 N 个 P 帧 (IPPPPIPPPP...)
     * @return true 表示成功，false 表示不支持
     *
     * @note 可在编码过程中动态调用
     */
    virtual bool setFrameIntervalP(const int frameIntervalP) = 0;

    virtual ~EncoderCallback() {}
};
```

3.5.3.2. 使用回调式编码器

回调接口使用示例：

```
//1. 实现回调接口
class MyEncoderCallback : public cv::escodec::EncoderCallback {
public:
    void onEncoded(const std::vector<std::vector<uint8_t>>& vPacket,
        const std::vector<uint64_t>& pts) override {
        // 处理编码数据：推流、保存、网络传输等
        for (size_t i = 0; i < vPacket.size(); i++) {
            const auto& packet = vPacket[i];
            uint64_t timestamp = pts[i];
            // 自定义处理逻辑
            // 例如：发送到 RTMP 服务器、写入文件等
        }
    }
};
```

```
}

void onEncodingFinished() override {
    // 编码结束清理（关闭连接、释放资源等）
}

bool setFrameIntervalP(const int frameIntervalP) override {
    // 如果需要动态调整 GOP，实现此方法
    return true; // 返回 true 表示支持
}

};

// 2. 创建回调式编码器
auto callback = cv::makePtr<MyEncoderCallback>();

cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H264;
params.bitrate = 3000;
params.rcMode = cv::escodec::RC_CBR;

cv::Ptr<cv::escodec::ESVideoWriter> writer =
    cv::escodec::createVideoWriter(
        callback,
        cv::Size(1920, 1080),
        cv::escodec::CODEC_H264,
        30.0,
        params
    );

// 3. 编码循环
cv::es::esMat frame;
while (reader->nextFrame(frame)) {
    writer->write(frame); // 编码完成后自动调用 callback
}
```

完整回调实现 (RTMP 推流、文件保存、网络传输): 参见 `/usr/share/opencv4/samples/eswin/cpp/callback_encoder.cpp`

3.5.4. 支持的编码格式

3.5.4.1. 编码格式支持

格式	编码支持	Profile	分辨率范围	备注
H.264	硬件加速	Baseline/Main/High	144x128 ~ 8192x8192 (对齐 2)	主流格式，推荐
H.265	硬件加速	Main	136x128 ~ 8192x8192 (对齐 2)	高压缩率，推荐
MJPEG	硬件加速	-	32x32 ~ 32768x32768 (对齐 2)	帧内编码
MPEG-2	软件编码	-	取决于 FFmpeg	已过时，性能低
MPEG-4	软件编码	-	取决于 FFmpeg	已过时，性能低

提示

- 硬件加速: H.264/H.265/MJPEG 使用 ESMPP 硬件编码器 (`h264_esmpp_encoder` / `hevc_esmpp_encoder` / `mjpeg_esmpp_encoder`)

- **软件编码：**MPEG-2/MPEG-4 使用 FFmpeg 软件编码器，性能远低于硬件编码，不推荐生产使用
- 不支持其他软件编码格式（如 AV1、VP9 等）

3.5.4.2. 容器格式支持

容器	文件扩展名	支持的编码格式	备注
MP4	.mp4	H.264/H.265/MJPEG	推荐，兼容性最好
Matroska	.mkv	H.264/H.265/MJPEG	开源格式
AVI	.avi	H.264/MJPEG	老格式，兼容性好
FLV	.flv	H.264	Flash 流媒体
MPEG-TS	.ts	H.264/H.265	广播标准
Raw	.h264/h265	H.264/H.265	无容器

3.5.4.3. 编码器输入格式兼容性

支持的输入格式：

格式	常量名	零拷贝支持	来源	备注
8-bit YUV 格式				
I420	ES_PIXFMT_I420	支持	解码器默认输出	YUV420 planar, 推荐
NV12	ES_PIXFMT_NV12	支持	解码器/VPS 输出	YUV420 semi-planar, 推荐
NV21	ES_PIXFMT_NV21	支持	VPS 输出	Android 常用
YV12	ES_PIXFMT_YV12	支持	软件处理	I420 变种 (UV 平面交换)
YUYV	ES_PIXFMT_YUY2	支持	相机输入	YUV422 packed
UYVY	ES_PIXFMT_UYVY	支持	相机输入	YUV422 packed
10-bit YUV 格式				
P010LE	ES_PIXFMT_YUV420SP010LE	支持	HDR 解码器	10-bit NV12, HDR 内容
YUV420P010LE	ES_PIXFMT_YUV420P010LE	支持	HDR 解码器	10-bit I420 planar
灰度格式				
GRAY8	ES_PIXFMT_GRAY8	支持	解码器	8-bit 单通道

提示

- 格式选择建议：**
- **H.264/H.265:** 推荐 NV12 或 I420
 - **MJPEG:** 推荐 I420
 - **HDR 内容:** 使用 P010LE 或 YUV420P010LE
 - 零拷贝硬件编码要求输入为硬件 AVFrame，并且 EncoderParams::pixelFormat 与输入 esMat 像

素格式一致

3.5.5. 编码器 API 参考表

3.5.5.1. ESVideoWriter 类 API

API	功能	关键点
write(frame)	写入一帧	frame 为 esMat，零拷贝编码
release()	结束编码	刷新缓冲区，写入文件尾
getParams()	获取编码参数	返回当前 EncoderParams 配置

3.5.5.2. VideoWriter 工厂函数

工厂函数	功能	关键点
createVideoWriter(filename, codec, size, fps)	创建编码器（基础版）	使用默认编码参数
createVideoWriter(filename, size, fps, params)	创建编码器（推荐）	自定义编码参数 (codec/bitrate/GOP/RC 等)
createESVideoWriter(filename, size, fps, codec)	创建编码器（简化）	默认 codec 为 H.265
createESVideoWriter(filename, size, fps, params)	创建编码器（Python 兼容命名）	参数顺序与推荐重载一致
createVideoWriter(callback, size, codec, fps, params)	回调式编码器	用于 RTMP 推流/自定义 muxer

3.5.5.3. EncoderParams

参数	类型	默认值	说明
codec	int	CODEC_H264	编 码 格 式 (H264/H265/MJPEG/M- PEG2/MPEG4)
bitrate	int	2000	码率 (kbps)，越大质量越高
rcMode	int	RC_VBR	码 率 控 制 模 式 (VBR/CBR/CQP)
gopSize	int	75	GOP 大小，I 帧间隔
iqp / pqp / bqp	int	30/32/32	I/P/B 帧 QP 值，越小质量越高
pixelFormat	int	ES_PIXFMT_I420	输入像素格式
profile	int	-1	编码 profile，-1 表示自动
level	int	0	编码 level，0 表示自动
useHardwareInput	bool	false	是否硬件输入（零拷贝标识）

3.5.5.4. EncoderCallback 接口

方法	功能	使用场景
onEncoded(vPacket, pts)	编码完成回调	接收 NAL 单元，pts 为时间戳 (微秒)
onEncodingFinished()	编码结束回调	清理资源、关闭连接
setFrameIntervalP(interval)	动态调整 GOP 结构	P 帧间隔控制 (0=全 I 帧, 1=IP 模式, N=N 个 P 帧)

3.5.5.5. 编码格式常量

常量	值	说明
CODEC_H264	0	H.264/AVC 编码（兼容性好）
CODEC_H265	1	H.265/HEVC 编码（推荐，压缩率高）
CODEC_MJPEG	2	Motion JPEG 编码（帧内编码）
CODEC_MPEG4	3	MPEG-4 Part 2 编码（已过时）
CODEC_MPEG2	4	MPEG-2 编码（已过时）

3.5.5.6. 码率控制模式常量

模式	值	说明	适用场景
RC_CBR	0	恒定码率	直播、网络传输（带宽稳定）
RC_VBR	1	可变码率	离线转码、归档（质量优先）
RC_CQP	2	恒定 QP	质量测试、对比实验
RC_FIXQP	2	固定 QP 别名	兼容旧代码，等价于 RC_CQP

编码器关键设计原则：

- 1. **零拷贝编码**：接受 esMat 作为输入，直接从设备内存编码
- 2. **格式匹配**：输入像素格式应与解码器输出格式一致（推荐 NV12）
- 3. **GOP 配置**：gopSize=fps 实现 1 秒 I 帧间隔（通用场景）
- 4. **码率选择**：1080p 推荐 4-6Mbps(VBR) 或 2-3Mbps(CBR)
- 5. **回调模式**：RTMP 推流、自定义容器需实现 EncoderCallback 接口
- 6. **编码完成**：write() 将当前帧送入编码器，release() 负责 flush 编码器内部缓存并完成封装输出

3.6. esimgproc 模块: 图像处理

esimgproc 模块提供面向 cv::es::esMat 的图像处理接口。resize/crop/format/normalize/blend/stack 等处理通过 FFmpeg libavfilter/ESMPP 后端构建硬件滤镜图执行；绘制接口优先使用 Native 后端，必要时回退到 FFmpeg 滤镜，因此不应简单理解为所有操作都由 VPS 执行。

3.6.1. 图像缩放与裁剪

3.6.1.1. resize: 硬件缩放

```
namespace cv { namespace esimgproc {

int resize(
    const cv::es::esMat& src,
    cv::es::esMat& dst,
    cv::Size dsize,
    double fx = 0,
    double fy = 0,
    int interpolation = 0
);

}}
```

基础缩放：

```
cv::es::esMat src, dst;
reader->nextFrame(src); // 1920x1080

// 缩放到 640x480
int ret = cv::esimgproc::resize(src, dst, cv::Size(640, 480));
if (ret != cv::esimgproc::ESIMGPROC_SUCCESS) {
    // 处理错误
}
```

Letterbox 缩放（保持宽高比）：

```
// makeLetterboxClip 是库提供的辅助函数
CV_EXPORTS_W cv::esimgproc::ClipParams makeLetterboxClip(
    cv::Size src_size,
    cv::Size target_size,
    bool long_edge_fit = true,
    const std::string& fill = "black",
    cv::esimgproc::LetterboxAlignment alignment = cv::esimgproc::LETTERBOX_ALIGN_CENTER,
    int stride = 2
);

// 使用示例：YOLOv5 预处理（居中对齐，黑边填充）
cv::esimgproc::ClipParams clip_params = cv::esimgproc::makeLetterboxClip(
    src.size(),           // 源尺寸 1920x1080
    cv::Size(640, 640),   // 目标尺寸
    true,                 // 长边适配
    "black"               // 黑边填充
);

// 使用 process 函数应用 letterbox
cv::esimgproc::ProcessParams params;
params.common.clip_params = clip_params;
params.common.output_format = cv::es::ES_PIXFMT_NV12;
cv::esimgproc::process(src, dst, params);
// 输出：640x640，黑边填充，内容保持宽高比

// YOLOX 预处理（左上对齐，灰色填充）
clip_params = cv::esimgproc::makeLetterboxClip(
    cv::Size(1920, 1080),
    cv::Size(640, 640),
    true,
    "0x727272",          // 灰色填充
    cv::esimgproc::LETTERBOX_ALIGN_TOP_LEFT // 左上对齐
);
```

3.6.1.2. ClipParams 裁剪参数详解

```
struct ClipParams {
    int x, y;           // 内容在画布上的偏移位置
    int width, height; // 缩放后内容的尺寸
    int o_w, o_h;       // 画布尺寸 (0 表示使用输入尺寸)
    std::string fill;   // 背景填充颜色 ("black"/"0xFF0000"/"red@0.5")
};
```

示例 1: PiP (画中画) 效果:

```
cv::es::esMat main_video(1080, 1920, cv::es::ES_PIXFMT_NV12);
cv::es::esMat small_video(480, 640, cv::es::ES_PIXFMT_NV12);
cv::es::esMat result;

// 方式 1: 使用 processDualInput (推荐)
cv::esimgproc::DualInputParams params;
// 小视频预处理: 缩放到 320x180
params.overlay_ops.enable = true;
params.overlay_ops.common.dsize = cv::Size(320, 180);
// Logo 模式: 放置到右下角
params.overlay_mode.logo.enable = true;
params.overlay_mode.logo.x = 1600; // 右下角 X
params.overlay_mode.logo.y = 900;  // 右下角 Y
params.overlay_mode.logo.x_dir = 1; // 从右边算起
params.overlay_mode.logo.y_dir = 1; // 从下边算起

cv::esimgproc::processDualInput(main_video, small_video, result, params);
```

示例 2: 中心裁剪 + 缩放:

```
// 使用 CommonParams 裁剪
cv::esimgproc::ProcessParams params;
params.common.crop_rect = cv::Rect(640, 220, 640, 640); // 裁剪中心 640x640
params.common.dsize = cv::Size(320, 320);              // 缩放到 320x320
params.common.output_format = cv::es::ES_PIXFMT_NV12;

cv::esimgproc::process(src, dst, params);
```

3.6.2. 格式转换

3.6.2.1. process: 统一处理接口

```
int process(const cv::es::esMat& src, cv::es::esMat& dst, const ProcessParams& params);

struct ProcessParams {
    CommonParams common;           // 通用参数 (resize/crop/flip/rotate/clip/format)
    NormalizeModeParams normalize_mode; // 归一化模式 (AI 预处理)
};

struct CommonParams {
    cv::Size dsize;                // 目标尺寸 (0,0=使用 fx/fy)
    double fx, fy;                 // 缩放因子
    cv::Rect crop_rect;            // 裁剪区域
    int flip_code;                 // 翻转: 0=水平, 1=垂直, -1 或 -100=跳过
    int rotate_code;               // 旋转: 0=90° 顺时针, 1=180°, 2=逆时针 90°, -1 或 -100=跳过
```

```

ClipParams clip_params; // Letterbox 裁剪
int output_format;      // 输出像素格式
int snapshot_interval;  // 快照间隔 (帧数, 0=禁用)
bool shortest;          // 最短输入结束时终止 (多输入)
bool repeatlast;        // 重复最后一帧 (多输入)
};

struct NormalizeModeParams {
    bool enable;          // 启用归一化
    NormalizeParams params; // 归一化配置
};

```

YUV 到 RGB/BGR 转换:

```

cv::es::esMat frame, bgr;
reader->nextFrame(frame); // 默认 I420 格式

cv::esimgproc::ProcessParams params;
params.common.output_format = cv::es::ES_PIXFMT_B8G8R8; // BGR packed

cv::esimgproc::process(frame, bgr, params);

```

YUV 到 BGR planar (AI 输入):

```

cv::es::esMat frame, bgr_planar;
reader->nextFrame(frame); // 默认 I420 格式

cv::esimgproc::ProcessParams params;
params.common.output_format = cv::es::ES_PIXFMT_B8G8R8_PLANAR; // BGR planar

cv::esimgproc::process(frame, bgr_planar, params);

// 输出内存布局:
// [B 平面: 640x640] [G 平面: 640x640] [R 平面: 640x640]

```

3.6.2.2. 归一化

```

enum NormalizeMode {
    MIN_MAX = 0, // Min-max 归一化
    Z_SCORE = 1 // Z-score 归一化
};

struct NormalizeParams {
    int mode; // NormalizeMode 枚举
    int output_format; // 输出格式 (通常为 Float planar)
    float step_reciprocal; // 量化步长倒数 (1/scale_factor), 0=使用默认 1.0

    // Min-max 模式参数
    std::vector<float> min_val; // 每通道最小值 (用于缩放前)
    std::vector<float> mm_reciprocal; // (Max-Min) 倒数 (缩放到 [0,1] 或 [0,255])

    // Z-score 模式参数
    std::vector<float> mean_val; // 每通道均值
    std::vector<float> std_reciprocal; // 每通道标准差倒数 (1/stddev)
};

```

YOLOv5 预处理示例:

```

cv::es::esMat frame, preprocessed;
reader->nextFrame(frame); //1920x1080 1420 (默认格式)

cv::esimgproc::ProcessParams params;

//1. Letterbox 缩放到 640x640
params.common.clip_params = cv::esimgproc::makeLetterboxClip(
    frame.size(), cv::Size(640, 640));

//2. 归一化: [0, 255] → [0, 1] (Z-score 模式)
params.normalize_mode.enable = true;
params.normalize_mode.params.mode = cv::esimgproc::Z_SCORE;
params.normalize_mode.params.output_format = cv::es::ES_PIXFMT_R32G32B32F_PLANAR; //Float32 RGB
    ↪ planar
params.normalize_mode.params.mean_val = {0.0f, 0.0f, 0.0f}; //均值
params.normalize_mode.params.std_reciprocal = {1.0f/255.0f, 1.0f/255.0f, 1.0f/255.0f}; //1/std

//一次 process 调用完成: 预处理阶段执行 YUV → RGB + Letterbox, 随后执行归一化
int ret = cv::esimgproc::process(frame, preprocessed, params);
if (ret == cv::esimgproc::ESIMGPROC_SUCCESS) {
    //preprocessed 可直接送入 NPU 推理
    //内存布局: [R 平面: 640x640][G 平面: 640x640][B 平面: 640x640], Float32 格式
}

```

3.6.3. 图形绘制

esimgproc 提供原地修改 esMat 的图形绘制接口。矩形、线段和网格优先使用 Native 后端，格式或特性不满足时回退到 FFmpeg 滤镜；文本绘制优先使用 Native FreeType 后端，复杂特性或不支持格式会回退到 FFmpeg drawtext。

3.6.3.1. drawBoxes: 绘制矩形框

```

int drawBoxes(cv::es::esMat& img, const DrawBoxesParams& params);

struct DrawBoxesParams {
    std::vector<Rectangle> boxes; //矩形列表
};

struct Rectangle {
    int x, y, width, height; //位置和尺寸
    std::string color; //颜色 (默认"red")
    int thickness; //线宽 (-1=填充, 默认 2)
    bool replace; //替换颜色&alpha (默认 false)
    std::string box_source; //从 side data 读取 (默认空)
};

```

检测框绘制示例:

```

cv::es::esMat frame;
reader->nextFrame(frame);

//创建绘制参数
cv::esimgproc::DrawBoxesParams params;

//添加红色边框
cv::esimgproc::Rectangle box1;
box1.x = 100; box1.y = 100;
box1.width = 200; box1.height = 150;

```

```

box1.color = "red";
box1.thickness = 3;
params.bboxes.push_back(box1);

// 添加半透明蓝色填充
cv::esimgproc::Rectangle box2;
box2.x = 400; box2.y = 300;
box2.width = 150; box2.height = 100;
box2.color = "blue@0.5"; // 50%透明度
box2.thickness = -1; // 填充
params.bboxes.push_back(box2);

// 绘制 (原地修改 frame, 后端自动选择 Native 或 FFmpeg)
int ret = cv::esimgproc::drawBoxes(frame, params);

```

颜色格式:

1. 预定义名称: red, green, blue, yellow, white, black
2. 十六进制 RGB/RGBA: #FF0000、0x00FF00、#FF000080
3. 带透明度: red@0.5、0xFF0000@0x80

3.6.3.2. drawTexts: 绘制文本

```

int drawTexts(cv::es::esMat& img, const DrawTextsParams& params);

struct DrawTextsParams {
    std::vector<TextLabel> labels; // 文本标签列表
};

struct TextLabel {
    // 核心参数
    std::string text;           // 文本内容
    std::string fontfile;       // 字体文件路径 (必须)
    int x, y;                   // 位置坐标
    int fontsize;               // 字体大小 (默认 16)

    // 颜色参数
    std::string fontcolor;       // 文本颜色 (默认 "white")
    std::string boxcolor;        // 背景框颜色 (默认 "black@0.5")
    std::string bordercolor;     // 文本边框颜色
    std::string shadowcolor;     // 阴影颜色

    // 背景框参数
    bool box;                   // 启用背景框 (默认 false)
    int boxborderw;              // 框边框宽度 (默认 0)
    int boxw, boxh;              // 框固定尺寸 (0=自动)

    // 布局参数
    int line_spacing;            // 行间距
    std::string text_align;      // 对齐方式
    int shadowx, shadowy;        // 阴影偏移
    int borderw;                 // 文本边框宽度
    // ... 更多参数见完整 API 文档
};

```

文本标注示例:

```

cv::esimgproc::DrawTextsParams params;

// 添加带背景的标签
cv::esimgproc::TextLabel label;
label.text = "Person: 0.95";
label.x = 100;
label.y = 80; // 位于检测框上方
label.fontfile = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf";
label.fontsize = 20;
label.fontcolor = "white";
label.box = true; // 启用背景框
label.boxcolor = "black@0.7"; // 半透明黑色背景
label.boxborderw = 2; // 背景框边框宽度
params.labels.push_back(label);

int ret = cv::esimgproc::drawTexts(frame, params);

```

字体处理：

```

# 板卡上安装字体
sudo apt-get install fonts-dejavu-core

# 列出可用字体
fc-list | grep ttf

# 常用字体路径
# 英文: /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
# 中文: /usr/share/fonts/truetype/wqy/wqy-zenhei.ttc

```

3.6.3.3. drawLines: 绘制线段

```

int drawLines(cv::es::esMat& img, const DrawLinesParams& params);

struct DrawLinesParams {
    std::vector<LineSegment> lines; // 线段列表
};

struct LineSegment {
    int x1, y1, x2, y2; // 起止点坐标
    std::string color; // 颜色 (默认 "white")
    int thickness; // 线宽 (默认 1)
    bool replace; // 替换颜色&alpha (默认 false)

    // 注意: 仅支持水平线 (y1==y2) 或垂直线 (x1==x2)
    // 对角线当前不支持
};

```

3.6.4. 批量处理

esimgproc 支持批量绘制，减少绘制/滤镜调用次数，提升检测框、标签等叠加场景的处理效率。

3.6.4.1. 批量绘制检测结果


```
// YOLOv5 检测结果
struct Detection {
    cv::Rect box;
    int classId;
    float score;
};

std::vector<Detection> detections = yolo_model.infer(frame);

// 批量创建绘制参数
cv::esimgproc::DrawBoxesParams boxParams;
cv::esimgproc::DrawTextsParams textParams;

for (const auto& det : detections) {
    // 添加边框
    cv::esimgproc::Rectangle rect;
    rect.x = det.box.x;
    rect.y = det.box.y;
    rect.width = det.box.width;
    rect.height = det.box.height;
    rect.color = getColorByClass(det.classId);
    rect.thickness = 2;
    boxParams.bboxes.push_back(rect);

    // 添加标签
    cv::esimgproc::TextLabel label;
    label.text = class_names[det.classId] + ": " +
        std::to_string(det.score).substr(0, 4);
    label.x = det.box.x;
    label.y = det.box.y - 5;
    label.fontfile = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf";
    label.fontsize = 16;
    label.fontcolor = "white";
    label.box = true;
    label.boxcolor = "black@0.7";
    textParams.labels.push_back(label);
}

// 一次性绘制所有边框和标签
int ret1 = cv::esimgproc::drawBoxes(frame, boxParams);
int ret2 = cv::esimgproc::drawTexts(frame, textParams);
if (ret1 != cv::esimgproc::ESIMGPROC_SUCCESS || ret2 != cv::esimgproc::ESIMGPROC_SUCCESS) {
    // 处理错误
}
```

3.6.5. API 参考表

3.6.5.1. 几何变换 API

API	功能	关键点
<code>resize(src, dst, dsize, fx, fy, interpolation)</code>	图像缩放	<code>dsize</code> 非空时按目标尺寸缩放，否则使用 <code>fx/fy</code> ； <code>interpolation</code> 当前实现会被忽略，硬件使用固定算法
<code>crop(src, dst, rect)</code>	图像裁剪	使用硬件滤镜图按 <code>Rect</code> 裁剪并输出到 <code>dst</code> ， <code>rect</code> 必须在输入图像范围内
<code>rotate(src, dst, rotateMode)</code>	图像旋转	支持 90°/180°/270° 硬件加速旋转

API	功能	关键点
flip(src, dst, flipMode)	图像翻转	支持水平/垂直翻转

3.6.5.2. 颜色空间转换 API

API	功能	关键点
cvtColor(src, dst, dst_outputFormat)	像素格式转换	第三个参数是目标 cv::es::EsPixelFormat，不是 OpenCV COLOR_* 转换码

常用目标格式：

- cv::es::ES_PIXFMT_B8G8R8 - 输出 BGR packed
- cv::es::ES_PIXFMT_R8G8B8 - 输出 RGB packed
- cv::es::ES_PIXFMT_GRAY8 - 输出灰度
- cv::es::ES_PIXFMT_NV12 / cv::es::ES_PIXFMT_I420 - 输出 YUV 格式

3.6.5.3. 图形绘制 API

API	功能	关键点
drawBoxes(frame, params)	批量绘制矩形	检测框、ROI 标注，支持颜色/线宽配置
drawTexts(frame, params)	批量绘制文本	支持 TrueType 字体、背景框、透明度
drawLines(frame, params)	批量绘制线段	仅支持水平/垂直线
drawGrids(frame, params)	批量绘制网格	网格可视化

3.6.5.4. 图像混合 API

API	功能	关键点
alphaBlend(base, overlay, dst, params)	Alpha 混合	水印、Logo 叠加，支持多种混合模式
processDualInput(base, overlay, dst, params)	双输入处理	支持 Blend/Logo 模式，可配合预处理
processMultiInput(inputs, dst, params)	多输入堆叠	用于画面拼接（2x2/1x4 等布局）

3.6.5.5. 统一处理接口

API	功能	关键点
process(src, dst, params)	统一处理接口	用一个参数结构统一描述 resize/crop/flip/rotate/clip/format/normalize; 启用 normalize 时实现分为预处理和归一化两个阶段

ProcessParams 主要参数:

- `common` - CommonParams (resize/crop/flip/rotate/clip/format)
- `normalize_mode` - NormalizeModeParams (AI 预处理)

3.6.5.6. 绘制参数结构

DrawBoxesParams:

```
struct Rectangle {
    int x, y, width, height; // 矩形位置和尺寸
    std::string color;       // 颜色 ("red"/"#FF0000"/"red@0.5")
    int thickness;           // 线宽 (像素)
};
std::vector<Rectangle> boxes; // 批量矩形
```

DrawTextsParams:

```
struct TextLabel {
    std::string text;        // 文本内容
    int x, y;               // 位置
    std::string fontfile;    // TTF 字体文件路径
    int fontsize;           // 字体大小
    std::string fontcolor;   // 字体颜色
    bool box;               // 是否绘制背景框
    std::string boxcolor;    // 背景框颜色 (支持透明度"black@0.7")
    int boxborderw;         // 背景框边框宽度
};
std::vector<TextLabel> labels; // 批量文本
```

3.6.5.7. 裁剪与 Letterbox 参数

参数	类型	说明
x, y	int	内容在画布上的偏移位置
width, height	int	缩放后内容的尺寸
o_w, o_h	int	画布尺寸 (0 表示使用输入尺寸)
fill	string	背景填充颜色 ("black"/"0xFF0000"/"red@0.5")

3.6.5.8. 归一化参数

参数	类型	说明
mode	int	归一化模式 (0=MIN_MAX, 1=Z_SCORE)
mean_val	vector<float>	均值, 少于 3 个值时实现会复用最后一个值
std_reciprocal	vector<float>	标准差倒数
step_reciprocal	float	步长倒数, 0 表示使用默认 1.0

参数	类型	说明
output_format	int	归一化输出格式，如 ES_PIXFMT_R32G32B32F_PLANAR 或 ES_PIXFMT_B32G32R32F_PLANAR

3.6.5.9. 旋转/翻转模式

模式常量	值	说明
ROTATEMODE_90_CLOCKWISE	0	顺时针 90°
ROTATEMODE_180	1	180°
ROTATEMODE_270_COUNTER-CLOCKWISE	2	逆时针 90°（等价于顺时针 270°）
FLIP_HORIZONTAL	0	水平翻转
FLIP_VERTICAL	1	垂直翻转

3.6.5.10. 混合模式

模式	说明	典型用途
BLEND_BASE	仅显示基础图	调试/旁路
BLEND_OVERLAY	仅显示覆盖图	调试/旁路
BLEND_BASE_OVER_OVERLAY	基础图覆盖在 overlay 之上	默认模式
BLEND_OVERLAY_OVER_BASE	overlay 覆盖在基础图之上	Logo/水印叠加
BLEND_BASE_IN_OVERLAY	基础图保留在 overlay alpha 区域内	遮罩合成
BLEND_OVERLAY_IN_BASE	overlay 保留在基础图 alpha 区域内	遮罩合成
BLEND_BASE_OUT_OVERLAY	基础图排除 overlay alpha 区域	遮罩合成
BLEND_OVERLAY_OUT_BASE	overlay 排除基础图 alpha 区域	遮罩合成
BLEND_BASE_ATOP_OVERLAY	基础图 atop overlay	高级合成
BLEND_OVERLAY_ATOP_BASE	overlay atop 基础图	高级合成
BLEND_XOR	异或混合	特殊效果

3.6.5.11. 状态码

状态码	值	说明
ESIMGPROC_SUCCESS	0	操作成功
ESIMGPROC_ERR_BACKEND_NOT_AVAILABLE	-1	后端不可用
ESIMGPROC_ERR_BACKEND_INIT_FAILED	-2	后端初始化失败
ESIMGPROC_ERR_HARDWARE_NOT_AVAILABLE	-3	硬件加速不可用
ESIMGPROC_ERR_INVALID_INPUT	-11	无效输入参数

状态码	值	说明
ESIMGPROC_ERR_EMPTY_INPUT	-12	输入 esMat 为空
ESIMGPROC_ERR_INVALID_SIZE	-13	无效尺寸
ESIMGPROC_ERR_INVALID_FORMAT	-14	不支持的格式
ESIMGPROC_ERR_INVALID_ROI	-15	无效 ROI
ESIMGPROC_ERR_INVALID_MODE	-16	无效模式/标志
ESIMGPROC_ERR_INVALID_PARAMS	-17	无效参数组合
ESIMGPROC_ERR_OUT_OF_MEMORY	-21	内存分配失败
ESIMGPROC_ERR_HARDWARE_FAILURE	-22	硬件处理失败
ESIMGPROC_ERR_FILTER_GRAPH_INIT	-23	滤镜图初始化失败
ESIMGPROC_ERR_FILTER_GRAPH_EXEC	-24	滤镜图执行失败
ESIMGPROC_ERR_FRAME_CONVERT	-25	帧转换失败
ESIMGPROC_ERR_UNSUPPORTED_FORMAT	-26	当前后端不支持该格式，可触发回退
ESIMGPROC_ERR_UNKNOWN	-99	未知错误

关键设计原则：

- 1. **批量优先**：绘制检测结果时使用批量 API(drawBoxes/drawTexts) 而非逐个绘制
- 2. **格式兼容**：输入输出格式应与编解码器一致 (I420/NV12)，避免转换开销
- 3. **process 接口**：AI 预处理推荐使用 process() 统一描述 resize/crop/format/normalize；启用 normalize 时实现会拆成预处理和归一化两个阶段
- 4. **硬件限制**：YUV 格式要求坐标/尺寸偶数对齐 (2 的倍数)
- 5. **字体路径**：绘制文本需指定 TTF 字体文件完整路径
- 6. **颜色格式**：支持 FFmpeg av_parse_color 语法，如名称 (“red”)、十六进制 (“#FF0000” / “0xFF0000”)、RGBA 十六进制 (“#FF000080”)、带透明度 (“black@0.7”)

3.7. 故障排查

3.7.1. 日志系统

3.7.1.1. 启用调试日志

```
# 方式 1: 环境变量
export OPENCV_LOG_LEVEL=DEBUG
export FFMPEG_LOG_LEVEL=debug

# 方式 2: 代码设置
cv::utils::logging::setLogLevel(cv::utils::logging::LOG_LEVEL_DEBUG);
```

3.7.1.2. 日志输出示例

```
#include <opencv2/escore/esLog.hpp>

// 使用便利宏
ES_LOGD("解码器初始化完成");
ES_LOGI("开始处理视频: %s", filename.c_str());
ES_LOGW("帧率异常: %.2f fps", actual_fps);
ES_LOGE("编码器打开失败: %s", error_msg.c_str());
```

```
// 或使用通用宏
ES_LOG(DEBUG, "解码器初始化完成");
ES_LOG(INFO, "开始处理视频: %s", filename.c_str());
```

3.7.2. 常见错误

3.7.2.1. 错误 1: 无法打开解码器

错误信息:

```
[ERROR] Unable to open decoder for file: input.mp4
```

可能原因:

1. 文件不存在或路径错误
2. 文件格式不支持
3. 解码器驱动未加载

排查步骤:

```
# 检查文件是否存在
ls -lh input.mp4

# 使用 ffprobe 查看文件信息
ffprobe input.mp4
```

3.7.2.2. 错误 2: 编码器创建失败

错误信息:

```
[ERROR] Failed to create encoder: No hardware encoder available
```

可能原因:

1. VENC 驱动未加载
2. 硬件实例数量已用尽
3. 分辨率不支持

排查步骤:

```
// 检查分辨率是否 16 像素对齐
// 1920x1080 支持
// 1921x1080 不支持 (宽度必须是 16 的倍数)
```

3.7.2.3. 错误 3: 零拷贝路径失败

错误信息:

```
[WARN] Zero-copy path not available, falling back to memcpy
```

可能原因:

1. 输入不是硬件内存
2. 格式不兼容

检查代码：

```
cv::es::esMat frame;
reader->nextFrame(frame);

// 检查是否为硬件内存
AVFrame* avf = frame.getAvFrame(); // 注意：方法名是 getAvFrame (小写 v)
if (avf && avf->format == AV_PIX_FMT_DRM_PRIME) {
    std::cout << "硬件内存 (支持零拷贝)" << std::endl;
} else {
    std::cout << "软件内存 (不支持零拷贝)" << std::endl;
}
```

3.7.3. 调试技巧

3.7.3.1. 验证数据路径

逐步测试每个环节，定位问题：

```
// 1. 测试解码
if (!reader->nextFrame(frame)) {
    std::cerr << "解码失败" << std::endl;
    return -1;
}

// 2. 验证数据有效性 (下载一帧到 CPU 查看)
cv::Mat cpu_frame;
frame.download(cpu_frame);
if (!cpu_frame.empty()) {
    cv::imwrite("debug_frame.jpg", cpu_frame); // 保存验证
}

// 3. 测试编码
if (!writer->write(frame)) {
    std::cerr << "编码失败" << std::endl;
    return -1;
}
```

3.7.3.2. 检查系统日志

```
# 检查内核错误日志
dmesg | grep -i "error\|fail"

# 查看 OpenCV 日志 (需设置环境变量 OPENCV_LOG_LEVEL=DEBUG)
```

3.8. 附录

3.8.1. 完整示例代码

完整可运行的示例代码参见：

- 使用手册： `/usr/share/doc/es-sdk-sample-opencv/README.md`

- **可执行程序：** `/usr/bin/opencv/opencv_sample_eswin_*`
- **示例数据：** `/usr/share/opencv-eswin-sample/data/`

`es-sdk-sample-opencv` 不安装示例源码。每个示例可执行支持 `-h` 或 `--help` 查看参数说明。

主要示例包括：

示例	功能	源文件
硬件转码	零拷贝视频转码	hwTranscode/hw_transcode.cpp
YOLOv5 检测	AI 推理 Pipeline	yolov5s/yolov5s.cpp
多线程推理	多线程 Pipeline	yolov5s/yolov5s_async.cpp
水印转码	水印叠加与转码	watermarkTranscode/watermark_transcode.cpp
混合转码	混合转码处理	hybridTranscode/hybrid_transcode.cpp

3.8.2. API 参考索引

模块	API 参考位置
escore	本手册 escore API 章节
escodec	本手册 escodec API 章节
esimgproc	本手册 esimgproc API 章节

3.8.3. 常用配置表

3.8.3.1. 编码预配置

1080p 高质量配置：

```
cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H264;
params.bitrate = 6000; // 6 Mbps (kbps)
params.rcMode = cv::escodec::RC_VBR;
params.iqp = 15;
params.pqp = 20;
params.gopSize = 30;
```

720p 直播配置：

```
cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H264;
params.bitrate = 2000; // 2 Mbps (kbps)
params.rcMode = cv::escodec::RC_CBR;
params.gopSize = 30;
```

4K 高清配置：

```
cv::escodec::EncoderParams params;
params.codec = cv::escodec::CODEC_H265; // 4K 推荐使用 H.265
params.bitrate = 20000; // 20 Mbps (kbps)
```



```
params.rcMode = cv::escodec::RC_VBR;  
params.iqp = 20;  
params.pqp = 23;  
params.gopSize = 60;
```

4. Python 开发指南

4.1. 快速入门

4.1.1. 环境要求

4.1.1.1. 硬件要求

组件	要求
SoC 平台	EIC7700 (RISC-V 64)
内存	>= 2GB RAM
存储	>= 1GB 可用空间

4.1.1.2. 软件依赖

组件	版本要求	说明
Python	3.12	已验证
Linux 内核	>= 5.10	需要 EIC7700 驱动支持
OpenCV wheel	4.12.0+eswin1	Python 绑定包
NumPy	>= 1.19	数组操作
es_py	>= 1.0	ESWIN SDK Python 绑定（可选，部分场景/示例需要）

注意：Python 3.12 默认强制使用虚拟环境，不允许直接 pip 安装到系统目录（PEP 668）。

4.1.2. 安装与配置

4.1.2.1. 方式 1：虚拟环境安装

```
# 1. 安装系统依赖
sudo apt-get update
sudo apt-get install python3.12-venv python3-numpy

# 2. 创建虚拟环境（继承系统 numpy）
python3 -m venv --system-site-packages ~/opencv_env

# 3. 激活虚拟环境
source ~/opencv_env/bin/activate

# 4. 安装 OpenCV wheel
pip install opencv4-1.12.0-cp312-cp312-linux_riscv64.whl

# 5. 安装 es_py（可选）
pip install es_py-1.0.0-cp312-cp312-linux_riscv64.whl

# 6. 安装 Python 示例 wheel（可选）
pip install opencv_eswin_sample-1.0.0-py3-none-any.whl

# 7. 验证安装
```

```
python -c "import cv2; print(cv2.__version__)"
# 输出: 4.12.0+eswin1

python -c "import cv2.escodec; print('escodec OK!)"
# 输出: escodec OK
```

虚拟环境优点：

- 隔离项目依赖，避免版本冲突
- 符合 Python 3.12 规范
- 保持系统环境干净

4.1.2.2. 安装验证

```
# 检查模块导入
python3 << EOF
import cv2
print(f"OpenCV 版本: {cv2.__version__}")
print(f"ESWIN 扩展模块:")
print(f"  - escodec: {hasattr(cv2, 'escodec')}")
print(f"  - esimgproc: {hasattr(cv2, 'esimgproc')}")
print(f"  - es: {hasattr(cv2, 'es')}")
EOF

# 输出示例:
# OpenCV 版本: 4.12.0+eswin1
# ESWIN 扩展模块:
#   - escodec: True
#   - esimgproc: True
#   - es: True
```

4.1.3. 第一个程序

创建 `transcode.py`，实现简单的视频转码：

```
#!/usr/bin/env python3
"""
简单的硬件加速视频转码示例
"""
import sys
import cv2

def main():
    if len(sys.argv) < 3:
        print(f"用法: {sys.argv[0]} <输入视频> <输出视频>")
        return 1

    input_file = sys.argv[1]
    output_file = sys.argv[2]

    try:
        # 1. 创建硬件解码器
        reader = cv2.escodec.createVideoReader(input_file)

        # 2. 获取视频信息
        fmt = reader.format()
        print(f"输入视频: {fmt.width}x{fmt.height} @ {fmt.avgFrameRate:.2f} fps")
```

```
# 3. 创建硬件编码器 (H.264)
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.pixelFormat = cv2.es.ES_PIXFMT_I420 # 与默认解码输出格式保持一致
enc_params.useHardwareInput = True           # 输入帧来自硬件解码器, 保持零拷贝路径

writer = cv2.escodec.createESVideoWriter(
    output_file,
    (fmt.width, fmt.height),
    fmt.avgFrameRate,
    enc_params
)

# 4. 零拷贝转码循环
frame = cv2.es.esMat()
frame_count = 0

while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame)
    frame_count += 1

    if frame_count % 30 == 0:
        print(f"已处理 {frame_count} 帧")

# 5. 完成编码
writer.release()
print(f"转码完成, 共 {frame_count} 帧")
return 0

except cv2.error as e:
    print(f"OpenCV 错误: {e}")
    return 1
except Exception as e:
    print(f"错误: {e}")
    return 1

if __name__ == "__main__":
    sys.exit(main())
```

运行程序：

```
# 激活虚拟环境
source ~/opencv_env/bin/activate

# 运行转码
python transcode.py input.mp4 output.mp4
```

代码说明：

1. **零拷贝路径**：esMat 从解码器到编码器，无 CPU 拷贝
2. **硬件加速**：VDEC 解码 + VENC 编码
3. **异常处理**：捕获 cv2.error 异常
4. **资源管理**：自动释放资源（Python 垃圾回收 + release()）

4.1.4. 虚拟环境管理

4.1.4.1. 激活与退出

```
# 激活虚拟环境
source ~/opencv_env/bin/activate

# 验证环境
which python
# 输出: /home/user/opencv_env/bin/python

# 退出虚拟环境
deactivate
```

4.1.4.2. 多项目管理

```
# 项目 A: 推理/视觉应用 (可选)
python3 -m venv --system-site-packages ~/yolo_env
source ~/yolo_env/bin/activate
pip install opencv4-1.12.0-cp312-cp312-linux_riscv64.whl es_py-1.0.0-cp312-cp312-linux_riscv64.whl
↪ opencv_eswin_sample-1.0.0-py3-none-any.whl

# 项目 B: 视频转码
python3 -m venv --system-site-packages ~/transcode_env
source ~/transcode_env/bin/activate
pip install opencv4-1.12.0-cp312-cp312-linux_riscv64.whl opencv_eswin_sample-1.0.0-py3-none-any.whl

# 切换项目
deactivate
source ~/yolo_env/bin/activate
```

4.1.4.3. requirements.txt

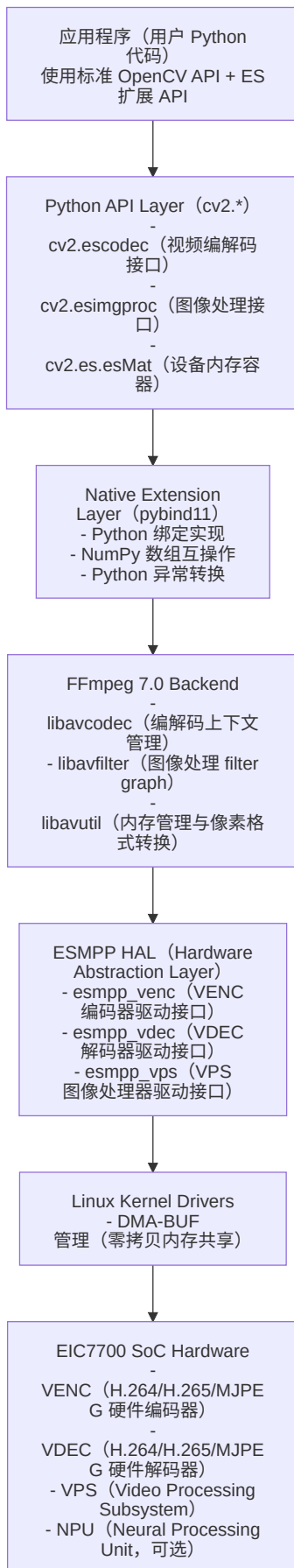
```
# 导出依赖
pip freeze > requirements.txt

# 在新环境安装
pip install -r requirements.txt
```

4.2. 硬件加速架构

4.2.1. 系统架构概述

EIC7700 OpenCV 采用分层架构设计，通过 `cv2.es*` ES 扩展模块暴露硬件加速能力，并与标准 OpenCV/NumPy API 协作：



关键设计原则：

- 1. **Pythonic 接口**：API 设计遵循 Python 习惯，与标准 OpenCV 风格一致
- 2. **零拷贝优化**：DMA-BUF 兼容路径下，esMat 在硬件后端间传递设备内存引用，避免 CPU 侧整帧拷贝
- 3. **后端资源调度**：应用层通过参数选择处理路径，具体硬件资源由 FFmpeg/ESMPP/驱动层协同管理
- 4. **NumPy 互操作**：通过 upload()/download() 与 NumPy 数组交互；这类操作涉及 CPU/设备内存拷贝
- 5. **异常机制**：使用 Python 标准异常处理，错误信息清晰明确

4.2.2. 硬件加速单元

4.2.2.1. VDEC: 视频解码器

规格项	参数
支持格式	H.264 (Baseline/Main/High/High10 Profile), H.265 (Main/Main10/Main Still Picture), MJPEG
最大分辨率	H.264/H.265: 8192x8192, JPEG/MJPEG: 32768x32768
最小分辨率	H.264: 48x48, H.265: 144x144, JPEG/MJPEG: 48x48
支持通道数	最多 128 个解码通道（分时复用硬件）
输出格式	NV12/ NV21/ I420/ YV12/ YUV400/ P010LE, RGB(A/X), BGR(A/X)
特性	硬件去块滤波、错误掩盖、解码序/显示序输出

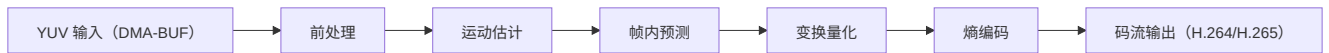
硬件解码流程：



4.2.2.2. VENC: 视频编码器

规格项	参数
支持格式	H.264 (BP/MP/HP/HP10), H.265 (Main/Main10/Main Still Picture), MJPEG
最大分辨率	H.264/H.265: 8192x8192, JPEG/MJPEG: 32768x32768
最小分辨率	H.264: 144(宽)x128(高), H.265: 136(宽)x128(高), JPEG/MJPEG: 32x32
对齐要求	宽高最小对齐 2 像素
输入格式	NV12/NV21/I420/YV12/YUY2/UYVY/P010LE/P010BE (8bit/10bit)
码率控制	CBR/VBR/FixQP/QPMAP (H.264/H.265), CBR/FixQP (MJPEG)
GOP 结构	NormalP/DualRef/SmartRef/AdvSmartP/BiPredB/LowDelayB
帧率范围	可配置，支持帧率控制

硬件编码流程：



4.2.2.3. VPS: 图像处理器

VPS 基于 DWE/VSE/HAE 三个硬件实现，各硬件规格如下：

硬件	最大分辨率	最小分辨率	主要功能
DWE	输入 4096x3072/输出 3840x2160	640x480	鱼眼矫正/畸形矫正/旋转
VSE	输入 4096x2160/输出 3840x2160	64x32	缩放/裁剪/格式转换/分流
HAE	7680x3840	64x16	缩放/旋转/镜像/归一化/绘制/合流

功能支持：

功能	硬件	备注
缩放	VSE/HAE	DWE 仅支持放大至 3840x2160
裁剪	DWE/VSE/HAE	组裁剪/通道裁剪
旋转	DWE/HAE	90/180/270 度，建议优先 HAE
镜像/翻转	DWE/HAE	水平/垂直，可组合实现 180° 旋转
格式转换	DWE/VSE/HAE	YUV ⇌ RGB, packed ⇌ planar
归一化	HAE	Z-score/Min-Max
绘制	HAE	矩形/线段/文本/Alpha 混合
分流	VSE	一进多出（最多 3 路输出）
合流	HAE	多进一出/Alpha Blend

4.2.3. 零拷贝数据路径

4.2.3.1. 传统 CPU 拷贝 vs 零拷贝

传统方式（存在 CPU 拷贝，如使用标准 OpenCV）：

```
# 传统方式（有 CPU 拷贝）
cap = cv2.VideoCapture("input.mp4") # CPU 软件解码
ret, frame = cap.read() # frame 是 numpy 数组（CPU 内存）

# 处理
processed = cv2.resize(frame, (640, 480)) # CPU 处理

# 编码
out.write(processed) # 需要 CPU 编码或上传到设备
```

零拷贝方式（EIC7700 OpenCV）：

```
# 零拷贝方式（DMA-BUF 兼容硬件路径）
reader = cv2.escodec.createVideoReader("input.mp4")
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.pixelFormat = cv2.es.ES_PIXFMT_I420
enc_params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("output.mp4", (640, 480), 30.0, enc_params)

frame = cv2.es.esMat() # 设备内存容器
while True:
```



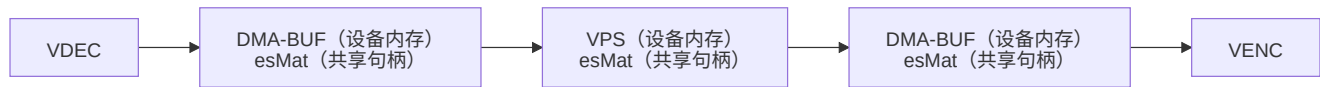
```
ok, frame = reader.nextFrame(frame) # VDEC 解码 → 设备内存
if not ok:
    break
# esimgproc 缩放 (兼容路径下设备内存 → 设备内存)
resized = cv2.es.esMat()
ret, resized = cv2.esimgproc.resize(frame, resized, (640, 480))
if ret != 0:
    raise RuntimeError("esimgproc.resize failed")

writer.write(resized) # VENC 编码 (设备内存 → 码流)
```

数据流对比：传统方式（存在 CPU 拷贝）：



零拷贝方式（EIC7700 OpenCV）：



零拷贝实现机制：

- 1. **DMA-BUF 共享**：兼容路径中通过 fd 共享/传递同一 DMA-BUF 引用，避免 CPU 侧整帧拷贝
- 2. **AVFrame 传递**：esMat 内部封装 FFmpeg AVFrame (DRM_PRIME 类型)
- 3. **文件描述符传递**：通过 fd 在硬件单元间传递内存引用
- 4. **引用计数**：AVBufferRef 机制管理内存生命周期
- 5. **Python 包装**：原生扩展封装底层对象，Python GC 自动管理 esMat 生命周期

4.2.3.2. 零拷贝路径要求

场景	是否支持零拷贝	说明
VDEC → VENC	支持	直接传递 DMA-BUF
VDEC → VPS → VENC	支持	输入/输出格式与后端兼容时，图像处理链路保持 DMA-BUF
NumPy 数组 → VENC	不支持	需要 upload 到设备内存
VDEC → CPU 处理 → VENC	不支持	download/upload 破坏零拷贝
esMat 赋值	支持	浅拷贝共享底层存储/AVFrame 引用

Python 特别说明：

- `frame.download()` 返回 NumPy 数组 → **有 CPU 拷贝**
- `frame.upload(numpy_array)` 上传 NumPy 数据 → **有 CPU 拷贝**
- DMA-BUF 兼容硬件链路中传递 esMat → **零拷贝**

4.2.4. FFmpeg 后端集成

EIC7700 OpenCV 基于 FFmpeg 7.0 实现，充分利用其成熟的编解码和容器处理能力。

4.2.4.1. 容器格式支持

通过 FFmpeg 的 libavformat，escodec 支持多种容器格式：

容器格式	扩展名	createVideoReader	createESVideoWriter	备注
MP4	.mp4	支持	支持	最常用，iOS 设备友好
Matroska	.mkv	支持	支持	开源格式，支持多音轨
AVI	.avi	支持	支持	兼容性好
FLV	.flv	支持	支持	Flash 流媒体
MPEG-TS	.ts	支持	支持	广播标准
Raw H.264	.h264	支持	支持	无容器，纯码流
Raw H.265	.h265/hevc	支持	支持	无容器，纯码流
RTMP	rtmp://	支持	支持	实时流媒体协议
RTSP	rtsp://	支持	不支持	实时流接收

Python 使用示例：

```
# 读取不同格式
reader_mp4 = cv2.escodec.createVideoReader("video.mp4")
reader_mkv = cv2.escodec.createVideoReader("video.mkv")
reader_rtsp = cv2.escodec.createVideoReader("rtsp://192.168.1.100/stream")

# 输出不同格式
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer_mp4 = cv2.escodec.createESVideoWriter("out.mp4", (1920, 1080), 30.0, params)

params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H265
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer_h265 = cv2.escodec.createESVideoWriter("out.h265", (1920, 1080), 30.0, params)
```

4.3. cv2.es 模块: 核心数据结构

4.3.1. 模块组织

EIC7700 OpenCV Python API 采用三层命名空间组织：

```
cv2                                # OpenCV 标准命名空间
├── imread()                       # 标准 OpenCV 函数
├── imwrite()
├── ... (标准 OpenCV API 省略)
├── escodec                        # 【硬件编解码模块】
│   ├── createVideoReader()        # 创建视频解码器
│   ├── createESVideoWriter()      # 创建视频编码器
│   ├── CODEC_H264 / CODEC_H265    # 编码格式常量
│   └── ... (更多编解码 API，见下方 tip)
├── esimgproc                      # 【硬件图像处理模块】
│   └── resize()                   # 硬件加速缩放
```

```

|   |—— process()           # 通用图像处理（格式转换/归一化）
|   |—— drawBoxes() / drawTexts() # 矩形框与文本绘制
|   |—— ...（更多图像处理 API，见下方 tip）
|
|—— es           # 【核心数据结构模块】
|   |—— esMat     # 设备内存容器类
|   |—— ES_PIXFMT_NV12 / NV21 # 像素格式常量（YUV/RGB/Gray）
|   |—— ...（更多格式常量，见下方 tip）

```

提示

对应 API 章节：

- `escodec`：见[硬件视频解码](#)和[硬件视频编码](#)。
- `esimgproc`：见[硬件图像处理](#)。
- `es`：见[esMat 完整 API 参考](#)和[像素格式常量](#)。

导入示例：

```

import cv2

# 方式 1：使用完整路径
reader = cv2.escodec.createVideoReader("video.mp4")
frame = cv2.es.esMat()

# 方式 2：导入子模块
from cv2 import escodec, esimgproc, es
reader = escodec.createVideoReader("video.mp4")
frame = es.esMat()

# 方式 3：导入特定函数（推荐）
from cv2.escodec import createVideoReader, createESVideoWriter
from cv2.es import esMat
reader = createVideoReader("video.mp4")
frame = esMat()

```

4.3.2. esMat 设计原理

`esMat` 是 EIC7700 OpenCV 的核心数据结构。设计目标是提供与 `cv::Mat` / `numpy.ndarray` 相近的尺寸、类型和 ROI 接口，但底层可管理硬件设备内存（DMA-BUF），便于与编解码、图像处理等硬件后端进行零拷贝交互。

4.3.2.1. 与 NumPy 数组的差异

`esMat` 是 EIC7700 OpenCV 的核心数据容器。与标准的 `numpy.ndarray` 不同，`esMat` 封装的是 EIC7700 SoC 的设备内存（DMA-BUF），而非 CPU 主内存。

三大核心特性：

1. **零拷贝传递** `esMat` 可在兼容硬件单元之间传递 DMA-BUF 引用，避免 CPU 侧整帧拷贝；是否保持零拷贝取决于输入/输出格式、后端能力和具体处理路径。
2. **平面访问** YUV 格式（如 NV12、I420）通过 `getPlane(idx)` 独立访问 Y/U/V 平面，方便 AI 预处理或自定义图像处理。
3. **NumPy 互操作** 通过 `upload()` / `download()` 与 NumPy 数组交互，但这两个操作涉及内存拷贝，应尽量在纯硬件加速路径中避免使用。

4.3.2.2. 基本用法

下面展示 esMat 的典型使用场景，完整 API 参考见 [esMat 完整 API 参考](#)。

```
import cv2
from cv2.es import esMat

# 场景 1: 作为硬件单元的输入输出容器
reader = cv2.escodec.createVideoReader("input.mp4")
frame = esMat() # 空构造, 由解码器填充
ok, frame = reader.nextFrame(frame) # 解码器输出到 esMat (设备内存)
if not ok:
    raise RuntimeError("decode failed")

# 场景 2: 与 NumPy 交互 (涉及拷贝)
numpy_array = frame.download() # 设备 → CPU
processed = process_on_cpu(numpy_array)
output_frame = esMat()
output_frame.upload(processed) # CPU → 设备

# 场景 3: 访问 YUV 平面
# rows=height, cols=width: 1920x1080 图像应传入 (1080, 1920)
nv12_frame = esMat(1080, 1920, cv2.es.ES_PIXFMT_NV12)
y_plane = nv12_frame.getPlane(0) # Y 平面 (rows=1080, cols=1920)
uv_plane = nv12_frame.getPlane(1) # NV12 UV 交织平面 (rows=540, cols=1920)

# 场景 4: 零拷贝切片 (ROI)
top_half = frame.rowRange(0, frame.rows // 2) # 行切片
left_half = frame.colRange(0, frame.cols // 2) # 列切片
```

内存模型说明:

- **设备内存池**: esMat 从 EIC7700 驱动维护的 DMA-BUF 内存池分配
- **引用计数**: Python 的垃圾回收机制自动管理 esMat 生命周期, 无需手动 `release()`
- **浅拷贝与深拷贝**: 切片操作 (`rowRange` 、 `colRange`) 创建浅拷贝 (共享底层内存), `clone()` 创建深拷贝

4.3.3. 性能关键路径

4.3.3.1. 数据传输路径对比

操作	是否拷贝	使用场景	性能特点
<code>upload()</code>	是	NumPy 数组 → esMat	CPU 到设备内存拷贝; 尺寸/类型需与目标匹配, 格式转换请显式调用 <code>cv2.esimgproc</code>
<code>download()</code>	是	esMat → NumPy 数组	按 esMat 当前类型导出到 CPU 内存, 返回标准 <code>numpy.ndarray</code>
<code>mapToCpuMat()</code>	DMA-BUF 路径否	少量数据读取	mmap 映射设备内存, CPU 访问延迟高; 非 DMA-BUF 后端以实现返回为准
硬件间传递	否	VDEC → VPS → VENC	兼容路径下保持设备内存与 DMA-BUF fd 传递, 性能最优

4.3.3.2. 性能优化策略

策略 1：优先使用硬件间零拷贝路径

```
# 推荐：全硬件路径
ok, frame = reader.nextFrame(frame)      # VDEC 解码 → esMat
if not ok:
    raise RuntimeError("decode failed")
ret, resized = cv2.esimgproc.resize(frame, ...) # VPS 处理
if ret != 0:
    raise RuntimeError("esimgproc.resize failed")
writer.write(resized)                    # VENC 编码
# 数据全程在设备内存，无 CPU 拷贝
```

策略 2：仅在必须时使用 upload/download

```
# 需要 CPU 处理时才使用
cpu_array = frame.download()             # 拷贝到 CPU
processed = cv2.cvtColor(cpu_array, cv2.COLOR_BGR2GRAY) # CPU 处理
result = esMat()
result.upload(processed)                  # 拷贝回设备
```

高分辨率图像的拷贝开销显著，1080p BGR 图像（ $1920 \times 1080 \times 3 \approx 6\text{MB}$ ）的上传/下载会影响实时性能。

策略 3：mapToCpuMat() 的适用场景

`mapToCpuMat()` 在 DMA-BUF 路径下将设备内存通过 mmap 映射到 CPU 地址空间，返回 numpy 视图。

适合场景：

- 读取图像元信息（尺寸、格式）
- 访问少量像素值（如 ROI 中心点）
- 调试时快速查看图像数据

不适合场景：

- CPU 密集计算（如逐像素遍历、卷积）
- 大量数据读写操作

技术背景：

- **UMA 架构**：EIC7700 的 CPU 和硬件加速单元共享 DDR 物理内存
- **内存属性优化**：设备内存针对 VDEC/VENC/VPS/NPU 的协同访问优化，确保硬件间高带宽和缓存一致性
- **CPU 访问代价**：CPU 访问设备内存的延迟高于访问 CPU 本地缓存或普通内存（如 numpy 分配的内存）
- **零拷贝优势**：DMA-BUF 路径下映射操作无拷贝开销；`mapToCpuMat()` / `download()` 返回前会做必要的 cache 同步，但低层指针或长期视图读写仍需注意生命周期和同步边界

4.3.3.3. 完整示例：三种数据路径对比

```
import cv2
import numpy as np

reader = cv2.escodec.createVideoReader("input.mp4")
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.pixelFormat = cv2.es.ES_PIXFMT_I420
enc_params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("output.mp4", (1920, 1080), 30.0, enc_params)
frame = cv2.es.esMat()

# ===== 方案 1：全硬件路径（最优性能） =====
```

```
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    # VDEC 解码 → esimgproc 缩放 → VENC 编码（兼容路径下全程设备内存）
    resized = cv2.es.esMat()
    ret, resized = cv2.esimgproc.resize(frame, resized, (1280, 720))
    if ret != 0:
        raise RuntimeError("esimgproc.resize failed")
    writer.write(resized) # 零拷贝传递

# ===== 方案 2: 需要 CPU 处理（有拷贝开销） =====
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    # 下载到 CPU 内存
    cpu_array = frame.download() # 拷贝: 设备 → CPU (~6MB for 1080p BGR)

    # CPU 处理（标准 OpenCV/NumPy 函数）
    gray = cv2.cvtColor(cpu_array, cv2.COLOR_BGR2GRAY)
    result = cv2.Canny(gray, 100, 200)

    # 上传回设备内存
    device_frame = cv2.es.esMat()
    device_frame.upload(result) # 拷贝: CPU → 设备 (~2MB for 1080p GRAY)
    writer.write(device_frame)

# ===== 方案 3: DMA-BUF 零拷贝映射（仅读取少量数据） =====
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    cpu_view = frame.mapToCpuMat() # DMA-BUF 路径下零拷贝 mmap

    # 适用场景: 读取元信息
    height, width = cpu_view.shape[:2]
    roi_pixel = cpu_view[540, 960] # 读取中心点像素

    # 不推荐: 密集计算（CPU 访问设备内存延迟高）
    # mean_val = cpu_view.mean() # 慢! 需要遍历全图
    # 建议改为: mean_val = frame.download().mean()

    writer.write(frame) # 原始帧直接编码
```

4.3.4. esMat 完整 API 参考

本节提供 esMat 类的全部 Python API，按功能分类列出。

4.3.4.1. 构造与创建

Python API	功能	是否拷贝	示例
esMat()	空构造	否	frame = cv2.es.esMat()
esMat(rows, cols, type)	尺寸构造	否	frame = cv2.es.esMat(1080, 1920, cv2.es.ES_PIXFMT_NV12)

Python API	功能	是否拷贝	示例
esMat(size, type)	从元组构造	否	frame = cv2.es.esMat((1920, 1080), cv2.es.ES_PIXFMT_NV12) # (width, height)
esMat(rows, cols, type, scalar)	初始化构造	否	frame = cv2.es.esMat(1080, 1920, cv2.CV_8UC3, (0, 0, 255))
esMat(array)	从 NumPy 构造	是	frame = cv2.es.esMat(numpy_array)
esMat(rows, cols, type, fd, step, offset)	从 DMA-FD 构造	否	frame = cv2.es.esMat(1080, 1920, type, fd, step)
create(rows, cols, type)	重新分配内存	否	frame.create(1080, 1920, cv2.es.ES_PIXFMT_NV12)
clone()	深拷贝	是	copy = frame.clone()

4.3.4.2. 数据传输

Python API	功能	是否拷贝	返回值	性能说明
upload(src)	CPU →设备上传	是	None	内存拷贝；按源数组类型创建/写入，需尺寸和类型匹配
download()	设备→ CPU 下载	是	numpy.ndarray	内存拷贝，返回标准 numpy 数组
copyTo(dst)	拷贝到目标	是	None	同类型 esMat 间直接拷贝
map-ToCpuMat()	映射为 CPU 可访问	DMA-BUF 路径否	numpy.ndarray 视图	mmap 映射设备内存，CPU 访问延迟高

4.3.4.3. 高级视图转换

Python API	功能	返回类型	说明
cv2.es.toCvMatView(esmat)	esMat → numpy 视图	numpy.ndarray	零拷贝包装，返回带 esMat 元数据的 numpy 数组
cv2.es.fromCvMatView(mat)	numpy 视图→ esMat	esMat	仅能从 toCvMatView() 返回且仍保留元数据的原始视图恢复 esMat
cv2.es.toCvMatWithEsBackend(esmat)	esMat 后端的 numpy	numpy.ndarray	创建 esMat 支持的 numpy 数组，可用于标准 OpenCV 函数
cv2.es.fromCvMatWithEsBackend(mat)	恢复 esMat 后端	esMat	仅能从 toCvMatWithEsBackend() 创建的数组恢复 esMat

视图转换 API 使用场景：

```
import cv2
import numpy as np

# 场景 1: 将 esMat 包装为 numpy 数组，用于标准 OpenCV 函数
frame = cv2.es.esMat()
```

```
ok, frame = reader.nextFrame(frame)
if not ok:
    raise RuntimeError("decode failed")

# 方法 1: 使用 toCvMatView (零拷贝, 保留 esMat 元数据)
mat_view = cv2.es.toCvMatView(frame)
print(type(mat_view)) # <class 'numpy.ndarray'>
# 可以用于标准 OpenCV 操作 (只读); 结果是普通 CPU 数组, 不再携带 esMat 元数据
gray = cv2.cvtColor(mat_view, cv2.COLOR_BGR2GRAY)

# 只能从原始视图恢复 esMat, 不能从 gray 等新生成的 CPU 数组恢复
esmat_back = cv2.es.fromCvMatView(mat_view)

# 方法 2: 使用 toCvMatWithEsBackend (创建 esMat 后端的 numpy)
mat_with_backend = cv2.es.toCvMatWithEsBackend(frame)
# 这个 numpy 数组底层由 esMat 支持, 可用于硬件加速和标准 OpenCV 之间桥接
esmat_back2 = cv2.es.fromCvMatWithEsBackend(mat_with_backend)
```

与 mapToCpuMat() 的区别:

特性	mapToCpuMat()	toCvMatView()	toCvMatWithEsBackend()
类型	成员方法	全局函数	全局函数
调用方式	arr = frame.mapToCpuMat()	arr = cv2.es.toCvMatView(frame)	arr = cv2.es.toCvMatWithEsBackend(frame)
返回值	纯 numpy 数组	带 esMat 元数据的 numpy 视图	带 esMat 后端的 numpy 数组
是否拷贝	DMA-BUF 路径否	否 (零拷贝视图)	否 (共享 esMat 后端)
能否恢复 esMat	不能	能 (fromCvMatView())	能 (fromCvMatWithEsBackend())
主要用途	读取少量数据、调试	esMat 与 numpy 互转、保留元数据	桥接硬件加速与标准 OpenCV
性能特点	CPU 访问延迟较高	CPU 访问延迟较高	CPU 访问延迟较高
典型场景	读取图像尺寸、个别像素值	需要在 esMat 和 numpy 间灵活切换	在标准 OpenCV 函数中使用 esMat 数据

使用建议:

1. 仅需读取元信息或少量数据 → 使用 mapToCpuMat()

```
cpu_view = frame.mapToCpuMat()
height, width = cpu_view.shape[:2] # 读取尺寸
pixel = cpu_view[100, 200] # 读取单个像素
```

2. 需要在 esMat 和 numpy 间双向转换 → 使用 toCvMatView() + fromCvMatView() (恢复对象必须是原始视图)

```
mat_view = cv2.es.toCvMatView(frame)
# 处理...
frame_back = cv2.es.fromCvMatView(mat_view) # 恢复 esMat
```

3. 需要把 esMat 数据传给标准 OpenCV 函数 → 使用 toCvMatWithEsBackend() (标准 OpenCV 新生成的结果通常是 CPU 数组)


```
mat_with_backend = cv2.es.toCvMatWithEsBackend(frame)
# 可以传递给标准 OpenCV 函数
gray = cv2.cvtColor(mat_with_backend, cv2.COLOR_BGR2GRAY)
frame_back = cv2.es.fromCvMatWithEsBackend(mat_with_backend)
```

关键区别总结：

- `mapToCpuMat()` 最简单直接，但无法恢复为 `esMat`
- `toCvMatView()` 提供双向转换能力，保留 `esMat` 元数据
- `toCvMatWithEsBackend()` 创建 `esMat` 后端支持的 `numpy`，可桥接标准 OpenCV

DMA-BUF 路径下，这三类视图/映射都避免了整帧拷贝，但 CPU 访问性能都较慢，且只有保留绑定元数据的原始对象才能恢复为 `esMat`。software AVFrame 路径以实现返回为准；如需密集 CPU 处理，仍建议使用 `download()`。

数据传输策略：

- **全硬件路径**（推荐）：`esMat` 在硬件单元间传递，无 CPU 参与
- **大量 CPU 处理**：使用 `download()` 后在 CPU 内存处理，完成后 `upload()` 回设备
- **少量数据读取**：DMA-BUF 路径下使用 `mapToCpuMat()` 避免拷贝，如读取图像元信息
- **避免频繁切换**：在 CPU/设备内存间反复拷贝会严重影响性能

mapToCpuMat 的性能特点：DMA-BUF 路径下虽然避免了内存拷贝，但 CPU 直接访问设备内存的性能低于访问 CPU 本地内存。software AVFrame 路径会返回 `packed CPU 缓冲`；设备内存配置针对硬件单元间的协作优化，不适合 CPU 密集访问。

底层工作原理：

```
# mapToCpuMat() 的 DMA-BUF 路径逻辑示意（伪代码）
def mapToCpuMat(self):
    # 1. 通过 mmap 将设备内存映射到 CPU 地址空间
    mapped_ptr = system_mmap(self.dmaFd, self.size)

    # 2. 返回 numpy 数组视图（零拷贝，共享物理内存）
    return numpy.ndarray(shape=(self.rows, self.cols),
                        dtype=..., buffer=mapped_ptr)
```

关键点：

- DMA-BUF 路径下，`mapToCpuMat()` 只是映射已有的 DMA-BUF，不改变内存属性；software AVFrame 路径会导出 `packed CPU 缓冲`
- 适合读取少量元数据，不适合密集 CPU 操作

4.3.4.4. ROI 与切片

Python API	功能	是否拷贝	示例
<code>row(y)</code>	获取单行	-	<code>row = frame.row(100)</code>
<code>col(x)</code>	获取单列	-	<code>col = frame.col(50)</code>
<code>rowRange(start, end)</code>	行范围	-	<code>top_half = frame.rowRange(0, 540)</code>
<code>colRange(start, end)</code>	列范围	-	<code>left_half = frame.colRange(0, 960)</code>
<code>reshape(cn, rows)</code>	重塑形状	-	<code>reshaped = frame.reshape(1, -1)</code>

Python API	功能	是否拷贝	示例
adjustROI(dtop, dbot, dleft, dright)	调整 ROI 边界	-	frame.adjustROI(10, -10, 20, -20)
locateROI()	定位 ROI 位置	-	size, offset = frame.locateROI()

4.3.4.5. 平面操作

Python API	功能	返回值	示例
getPlane(idx)	获取单个平面	esMat	y_plane = nv12_frame.getPlane(0)
planeCount()	平面数量	int	count = frame.planeCount() # NV12=2, I420=3
planeHeights()	各平面高度	list[int]	heights = frame.planeHeights()
planeWidths()	各平面宽度	list[int]	widths = frame.planeWidths()
planeStrides()	各平面 stride	list[int]	strides = frame.planeStrides()
planeOffsets()	各平面偏移	list[int]	offsets = frame.planeOffsets()

4.3.4.6. 属性查询

Python API	返回值	说明
empty()	bool	是否为空（未分配内存）
size()	tuple[int, int]	图像尺寸 (width, height)
rows	int	行数（属性）
cols	int	列数（属性）
type()	int	OpenCV 类型（如 CV_8UC3）
depth()	int	数据深度（CV_8U/CV_16U 等）
channels()	int	通道数（1/3/4 等）
elemSize()	int	单个像素字节数
elemSize1()	int	单通道元素字节数
step	int	行字节数（属性）
step1(i)	int	以元素为单位的 step
total()	int	元素总数（rows × cols）
dims()	int	维度数（通常为 2）
isContinuous()	bool	内存是否连续无 padding
pixelFormat()	int	像素格式（ES_PIXFMT_XXX）

Python API	返回值	说明
alignMode()	int	对齐模式
dmaFd()	int	DMA 文件描述符
offset()	int	数据偏移量
getStep()	int	获取 step（同 step 属性）

4.3.4.7. 内存管理

Python API	功能	说明
release()	释放内存	手动释放 DMA-BUF（通常由析构自动调用）
swap(b)	交换内容	交换两个 esMat 的内部指针（零拷贝）

4.3.4.8. 像素格式常量

esMat 支持丰富的像素格式，覆盖常见的 YUV 和 RGB 格式。

常量	说明	平面数	典型用途
cv2.es.ES_PIXFMT_I420	YUV420P planar	3	编解码默认格式，Y/U/V 独立平面
cv2.es.ES_PIXFMT_NV12	YUV420SP（UV 交织）	2	VPS 常用格式，Android 相机
cv2.es.ES_PIXFMT_NV21	YUV420SP（VU 交织）	2	NV12 的 UV 交换版本
cv2.es.ES_PIXFMT_B8G8R8	BGR24 packed	1	OpenCV 标准格式，对应 CV_8UC3
cv2.es.ES_PIXFMT_B8G8R8A8	BGRA32 packed	1	带 Alpha 通道，对应 CV_8UC4
cv2.es.ES_PIXFMT_GRAY8	灰度 8 位	1	单通道图像，对应 CV_8UC1

格式转换说明：

- upload/download 按 esMat 当前类型或源数组类型传输数据；BGR ↔ NV12 等格式转换请使用 `cv2.esimgproc.cvtColor()` / `process()`
- YUV 格式通过 `getPlane()` 访问各平面
- RGB 格式通过 numpy 标准索引访问：`array[y, x, channel]`

4.4. 硬件视频解码

本章介绍如何在 Python 中使用 `cv2.escodec` 的硬件加速解码能力，将视频解码为设备内存中的 `cv2.es.esMat`，并与 `cv2.esimgproc` / `cv2.escodec` 编码器组成零拷贝流水线。

与标准 OpenCV 的主要区别：

- 标准 `cv2.VideoCapture`：通常输出 `numpy.ndarray`（CPU 内存）
- EIC7700 `cv2.escodec.createVideoReader`：输出 `cv2.es.esMat`（设备内存），适合后续硬件处理

4.4.1. 功能概述

4.4.1.1. 推荐用法

下面是最常用的“解码→处理/显示/编码”的骨架：

```
import cv2

params = cv2.escodec.ESVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12 # 推荐：与 VPS/VENC 链路更匹配

reader = cv2.escodec.createVideoReader("input.mp4", params)
frame = cv2.es.esMat()

while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    # 方案 A：继续走硬件链路（零拷贝）
    # cv2.esimgproc.process(frame, dst, proc_params)
    # writer.write(frame)

    # 方案 B：下载到 CPU 做 NumPy/OpenCV 处理（会产生拷贝）
    # cpu_bgr = frame.download()
    # cpu_gray = cv2.cvtColor(cpu_bgr, cv2.COLOR_BGR2GRAY)
    pass
```

4.4.1.2. 典型场景

- 视频转码：解码（VDEC）→ 处理（VPS，可选）→ 编码（VENC）
- 智能分析：解码（输出 NV12）→ 预处理（letterbox/归一化）→ 推理 → 可视化叠加
- 监控接入：多路 RTSP 解码，按需丢帧降低延迟

4.4.2. 支持的解码格式

4.4.2.1. 硬件解码支持

EIC7700 SoC 的 VDEC 模块提供以下格式的硬件加速解码：

格式	解码支持	Profile	分辨率范围	备注
H.264	硬件加速	Baseline/Main/High	48x48 ~ 8192x8192	主流格式，最广泛支持
H.265	硬件加速	Main	144x144 ~ 8192x8192	高压缩率，4K/8K 推荐
MJPEG	硬件加速	-	48x48 ~ 32768x32768	帧内编码，监控常用

提示

表中的分辨率范围用于描述硬件能力的参考区间；不同 SDK/驱动/固件版本在边界条件（码流级别、参考帧数量、缓冲配置等）上可能存在差异，请以实际运行结果为准。

软件解码支持（无 VDEC 硬件加速，通过 FFmpeg 软件路径）：

- **VP8** - Google WebM 格式
- **VP9** - WebM 编码
- **AV1** - 开源编码
- **MPEG-2** - 传统 DVD 格式（已过时）
- **MPEG-4** - 传统视频格式（已过时）

提示

如果业务允许，优先选择 H.264/H.265/MJPEG 等硬件解码路径；其他格式会走 FFmpeg 软件解码，CPU 开销和实时性取决于具体码流与系统负载。

4.4.2.2. 容器格式支持

解码器基于 FFmpeg 实现，支持所有常见容器格式：

容器	文件扩展名	常见用途	备注
MP4	.mp4	存储/分发	兼容性通常最好
Matroska	.mkv	存储/归档	支持多音轨/字幕等特性
AVI	.avi	兼容旧系统	对现代编码特性支持有限
FLV	.flv	传统推流	更多用于历史系统/工具链
MPEG-TS	.ts	传输/直播	广播与流媒体常用封装
Raw	.h264/h265	性能/调试	无容器封装，便于测试码流

容器选择建议：

- **MP4**：首选，兼容性最好，适合存储和分发
- **MKV**：开源格式，功能丰富，支持多音轨/字幕
- **TS**：流媒体传输标准，适合 IPTV/HLS
- **Raw**：无容器封装，解析开销最低，适合性能测试

4.4.2.3. 解码器输出格式

解码器输出的 esMat 像素格式通过 ESVideoReaderInitParams.decodeSurfaceFormat 参数配置：

```
import cv2

# 配置输出为 NV12 格式（推荐）
params = cv2.escodec.ESVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12

reader = cv2.escodec.createVideoReader("input.mp4", params)
```

支持的输出格式：

格式	零拷贝支持	常量名	典型用途
YUV 格式			
I420	支持	cv2.es.ES_PIXFMT_I420	FFmpeg 标准，解码器默认格式
NV12	支持	cv2.es.ES_PIXFMT_NV12	推荐 ，VPS/编码器常用，Android 标准

格式	零拷贝支持	常量名	典型用途
NV21	支持	cv2.es.ES_PIXFMT_NV21	Android 相机输出格式
YUV420SP010LE	支持	cv2.es.ES_PIXFMT_YUV420SP010LE	10-bit NV12, HDR 内容专用
灰度格式			
GRAY8	支持	cv2.es.ES_PIXFMT_GRAY8	8 位灰度, 只输出 Y 分量
RGB 格式			
BGR	有开销	cv2.es.ES_PIXFMT_BGR8	24 位 BGR packed, 需解码器支持对应输出布局
RGB	有开销	cv2.es.ES_PIXFMT_RGB8	24 位 RGB packed, 需解码器支持对应输出布局
BGRA	有开销	cv2.es.ES_PIXFMT_BGRA8	32 位 BGRA, 需转换+Alpha 通道
RGBA	有开销	cv2.es.ES_PIXFMT_RGBA8	32 位 RGBA, 需转换+Alpha 通道
BGRx	有开销	cv2.es.ES_PIXFMT_BGRX8	32 位 BGR+填充字节
RGBx	有开销	cv2.es.ES_PIXFMT_RGBX8	32 位 RGB+填充字节

格式选择建议：

场景	推荐格式	原因
视频转码（解码→编码）	NV12	零拷贝, VPS/VENC 原生支持
推理前处理（可选）	NV12 或 BGR	NV12 需手动转换, BGR 自动转换但有开销
图像处理（VPS）	NV12	VPS 硬件原生格式
NumPy 处理（CPU）	BGR	与 OpenCV 标准一致, 但需拷贝
HDR 内容	YUV420SP010LE	保留 10-bit 色深
灰度分析	GRAY8	节省内存和带宽

性能关键点：

- **YUV 格式**：零拷贝性能最佳, 直接在设备内存处理
- **RGB/BGR packed 格式**：依赖解码器 `dec_pixfmt` 能力; `VideoReader` 本身不执行后处理色彩空间转换
- 默认格式为 **I420**, 如需与 VPS/VENC 链路匹配, 可显式设置为 **NV12**

格式转换示例：

```
# 场景 1: 零拷贝转码（最优性能）
params = cv2.escodec.ESVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12 # 与编码器输入一致

reader = cv2.escodec.createVideoReader("input.mp4", params)
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.pixelFormat = cv2.es.ES_PIXFMT_NV12
enc_params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("output.mp4", (1920, 1080), 30.0, enc_params)

frame = cv2.es.esMat()
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
```

```

break
writer.write(frame) # 全程设备内存，无拷贝

# 场景 2: 请求 BGR 布局用于 NumPy 处理 (有拷贝)
# 是否可直接输出 BGR 取决于解码器 dec_pixfmt 能力, VideoReader 不执行后处理色彩空间转换
params = cv2.escodec.ESSVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_B8G8R8

reader = cv2.escodec.createVideoReader("input.mp4", params)
frame = cv2.es.esMat()

while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    numpy_bgr = frame.download() # 下载到 CPU (输入 surface 为 BGR 布局时得到 BGR 数组)
    processed = cv2.cvtColor(numpy_bgr, cv2.COLOR_BGR2GRAY)
    # 进行 CPU 处理...

```

4.4.3. 创建解码器

使用 `createVideoReader()` 函数创建硬件解码器。

```

def createVideoReader(
    src: str,
    params: ESSVideoReaderInitParams = None
) -> VideoReader

```

参数:

- `src` (str): 视频文件路径或网络流 URL
 - 本地文件: `"/path/to/video.mp4"`
 - RTSP 流: `"rtsp://192.168.1.100:8554/stream"`
 - RTMP 流: `"rtmp://server/live/stream"`
- `params` (ESSVideoReaderInitParams, 可选): 解码参数

返回值:

- `VideoReader` 对象

基础用法:

```

import cv2

# 本地文件
reader = cv2.escodec.createVideoReader("video.mp4")

# RTSP 网络流
reader = cv2.escodec.createVideoReader("rtsp://camera_ip/live")

# 带参数
params = cv2.escodec.ESSVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12
reader = cv2.escodec.createVideoReader("video.mp4", params)

```

提示

创建失败通常会抛出 `cv2.error` 或返回不可用对象；业务代码应使用异常处理保护创建过程。

4.4.4. 解码参数配置

```
class ESVideoReaderInitParams:
    minNumDecodeSurfaces: int = 2      # 解码缓冲区数量 (DMA-BUF 数)
    allowFrameDrop: bool = False        # 是否允许丢帧 (实时流优化)
    udpSource: bool = False              # UDP 源优化 (减少缓冲延迟)
    strideAlign: int = 64                # YUV stride 对齐 (字节, 必须 ≥ 64)
    bufCacheMode: int = 1                # 解码 buffer 缓存模式
    decodeSurfaceFormat: int = cv2.es.ES_PIXFMT_I420 # 解码输出 surface 布局, 不执行后处理色彩空间转换
    enableHistogram: bool = False        # 启用直方图统计 (VDEC 硬件特性)
    firstFrameIdx: int = 0                # 起始帧索引 (用于跳过前 N 帧)
    rawMode: bool = False                 # 原始模式 (输出压缩码流而非解码帧)
```

参数详解：

参数	默认值	说明	适用场景
minNumDecodeSurfaces	2	解码器维护的 DMA-BUF 缓冲区数量。增加可提升流水线效率，但占用更多内存。	通常 2-4 即可，网络流可设为 4+
allowFrameDrop	False	True 时，解码器可丢弃帧以降低延迟。	实时流、监控场景
udpSource	False	True 时，优化 UDP 流的缓冲策略。	RTSP/RTP 等 UDP 传输协议
strideAlign	64	YUV 行对齐字节数，必须 ≥ 64 且为 2 的幂。VDEC 硬件要求对齐以优化 DMA 传输。	默认值适用绝大多数场景
decodeSurfaceFormat	ES_PIXFMT_I420	请求解码输出 surface 布局。 不执行后处理色彩空间转换 ，例如 YUV 到 RGB/BGR 转换应在解码后使用 esimgproc 处理。	I420: 默认布局 NV12: VPS/VENC 常用 P010LE: 10-bit 内容
enableHistogram	False	启用 VDEC 硬件直方图统计 (实验性功能)。	视频分析、自适应处理
firstFrameIdx	0	从第 N 帧开始解码，跳过前 N 帧。	视频截取、测试
rawMode	False	True 时，retrieve() 返回压缩码流 (AVPacket) 而非解码帧。	码流分析、转封装

配置示例：

```
# 示例 1: 低延迟实时流 (RTSP 监控)
params = cv2.escodec.ESVideoReaderInitParams()
params.minNumDecodeSurfaces = 1 # 最小缓冲, 降低延迟
params.allowFrameDrop = True    # 允许丢帧, 避免累积延迟
params.udpSource = True         # UDP 传输优化
```



```

reader = cv2.escodec.createVideoReader("rtsp://camera/live", params)

# 示例 2: NV12 输出格式 (VPS 处理常用)
params = cv2.escodec.ESSVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12 # NV12 布局
params.strideAlign = 64 # 默认对齐

reader = cv2.escodec.createVideoReader("video.mp4", params)

# 示例 3: 10-bit HDR 内容
params = cv2.escodec.ESSVideoReaderInitParams()
params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_YUV420SP010LE # P010LE (10-bit)

reader = cv2.escodec.createVideoReader("hdr_video.mp4", params)

# 示例 4: 从第 100 帧开始解码
params = cv2.escodec.ESSVideoReaderInitParams()
params.firstFrameIdx = 100

reader = cv2.escodec.createVideoReader("long_video.mp4", params)

```

4.4.5. 解码视频帧

4.4.5.1. 模式 1: nextFrame

```

frame = cv2.es.esMat()

while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    # 处理 frame (设备内存)
    process_frame(frame)

```

特点:

- 简洁易用
- 自动处理 EOF
- 适合大多数场景

4.4.5.2. 模式 2: grab + retrieve

```

frame = cv2.es.esMat()

while reader.grab():
    if some_condition:
        reader.retrieve(frame)
        process_frame(frame)

```

特点:

- 细粒度控制
- 可选择性解码
- 适合需要跳帧的场景

提示

esMat 是设备内存对象，若要使用 CPU 侧 OpenCV/NumPy 处理，请使用 `frame.download()`；若要继续走硬件链路（VPS/VENC），直接传递 `esMat` 即可。

4.4.6. 视频格式查询

```
fmt = reader.format()

print(f"宽度: {fmt.width}")
print(f"高度: {fmt.height}")
print(f"帧率: {fmt.avgFrameRate:.2f}")
print(f"编码格式: {fmt.codec}")
print(f"像素格式: {fmt.surfaceFormat}")
print(f"比特深度: {fmt.bitDepth}")
print(f"完整范围标志: {fmt.videoFullRangeFlag}")
```

ESFormatInfo 属性：

属性	类型	说明	典型值
width	int	视频宽度（像素）	1920, 3840
height	int	视频高度（像素）	1080, 2160
avgFrameRate	float	平均帧率（fps）	25.0, 30.0, 60.0
codec	int	编码格式（内部解码器 ID）	见下表
surfaceFormat	int	解码输出像素格式（ES_PIXFMT_*）	ES_PIXFMT_I420, ES_PIXFMT_NV12
bitDepth	int	比特深度	8, 10
videoFullRangeFlag	bool	色彩范围标志（Full/Limited）	True=Full(0-255), False=Limited(16-235)
enableHistogram	bool	直方图统计是否启用	True/False
nMaxHistogramBins	int	直方图最大 bin 数	通常为 256

codec 字段（ESCODEC_*常量）：

常量值	名称	说明
0	ESCODEC_H264	H.264/AVC
1	ESCODEC_H265	H.265/HEVC
2	ESCODEC_VP8	VP8
3	ESCODEC_VP9	VP9
4	ESCODEC_MPEG2	MPEG-2
5	ESCODEC_MJPEG	Motion JPEG
6	ESCODEC_MPEG4	MPEG-4 Part 2
7	ESCODEC_AV1	AV1
-1	ESCODEC_UNKNOWN	未知格式

提示

codec 字段返回的是内部解码器 ID (ESCODEC_)，这与编码器 API 中使用的 CODEC_常量 (仅 H264/H265/MJPEG) 不同。解码器支持更多格式 (VP8/VP9/AV1 等)，但编码器仅支持 H.264/H.265/MJPEG。

4.4.7. 解码器 API 参考

本节以“快速查阅”为目标，避免与前文教程重复。

4.4.7.1. 工厂函数

函数名	功能	参数	返回值
cv2.escodec.createVideoReader()	创建硬件解码器	src: str params: ESVideoReaderInitParams = None	VideoReader

4.4.7.2. VideoReader 方法

方法名	功能	说明
nextFrame(dst)	读取下一帧到 esMat	返回 (ok, dst)，推荐主循环使用
grab() / retrieve(dst)	分离抓取与解码	需要跳帧/预取时使用
format()	查询 ESFormatInfo	获取宽高、帧率、像素格式等
get(propId) / set(propId, value)	读写属性	用于 seek 等 (CAP_PROP_*)

4.4.7.3. ESVideoReaderInitParams 常用字段

字段	说明
decodeSurfaceFormat	输出像素格式 (建议 NV12 用于硬件链路)
minNumDecodeSurfaces	解码缓冲数量 (实时流可适当增加或减少以平衡延迟与吞吐)
allowFrameDrop	实时流低延迟场景可启用丢帧
udpSource	UDP 流 (RTSP/RTP) 可启用相应优化
firstFrameIdx	从指定帧开始解码

完整字段说明见[解码参数配置](#)。

4.4.7.4. ESFormatInfo 常用字段

字段	说明
width / height	分辨率
avgFrameRate	平均帧率
codec	解码器内部编码标识 (ESCODEC_*)

字段	说明
surfaceFormat	输出像素格式 (ES_PIXFMT_*)
bitDepth	8/10-bit

示例见[视频格式查询](#)。

4.5. 硬件视频编码

本章介绍如何在 Python 中使用 `cv2.escodec` 的硬件编码能力，将设备内存中的 `cv2.es.esMat` 编码为 H.264 / H.265 / MJPEG，并输出为文件或推流 URL。

与标准 OpenCV 的主要区别：

- 标准 `cv2.VideoWriter`：通常输入 `numpy.ndarray`（CPU 内存），多为软件编码路径
- `cv2.escodec.createESVideoWriter`：面向硬件编码，建议输入 `cv2.es.esMat`（设备内存）以减少 CPU 拷贝

4.5.1. 功能概述

4.5.1.1. 推荐用法

最常见用法是将解码器/图像处理输出的 `esMat` 直接送入编码器，构建“解码 →（可选处理）→ 编码”的设备侧流水线。

```
import cv2

# 解码器输出 NV12（常用于 VPS/VENC 链路）
decode_params = cv2.escodec.ESVideoReaderInitParams()
decode_params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12
reader = cv2.escodec.createVideoReader("input.mp4", decode_params)

# 编码器参数
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.rcMode = cv2.escodec.RC_VBR
enc_params.bitrate = 6000
enc_params.gopSize = 60
enc_params.pixelFormat = cv2.es.ES_PIXFMT_NV12
enc_params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "output.mp4", (1920, 1080), 30.0,
    enc_params
)

frame = cv2.es.esMat()
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame)

writer.release()
```

4.5.1.2. 支持的编码格式

格式	特性	典型应用
H.264 (AVC)	最广泛兼容，成熟稳定	网络直播、监控录像、通用视频
H.265 (HEVC)	高压缩率，支持 10-bit	4K 视频、HDR 内容、高效存储
MJPEG	简单快速，逐帧编码	图像序列、监控截图

4.5.1.3. 支持的输出目标

- 本地文件：MP4、MKV、AVI 等常见容器格式
- 流媒体推送：RTMP 直播推流
- 分辨率范围：取决于编码器实现、码流级别与系统资源配置
- 帧率范围：1 至 120 fps
- 码率控制：CBR（固定码率）、VBR（可变码率）、CQP（固定 QP）

4.5.1.4. 核心特性

硬件加速编码：

- 使用 VENC 硬件单元进行视频编码
- 支持从设备内存（esMat）直接输入，实现零拷贝编码
- 提供精细的码率控制（CBR/VBR/CQP）和 GOP 结构配置

与标准 OpenCV 对比：

- 标准 `cv2.VideoWriter`：CPU 软件编码，接受 `numpy` 数组（CPU 内存）
- EIC7700 `createESVideoWriter`：硬件编码，接受 `esMat`（设备内存），减少 CPU 开销

使用场景：

- 视频监控系统：多路 RTSP 流编码存储
- 直播应用：CBR 码率控制的 RTMP 推流
- 视频转码：解码 → 处理 → 重编码
- 高效存储：H.265 高压缩比编码

零拷贝输入的重要性：

硬件编码器支持硬件帧输入和软件帧输入。若输入为解码器或 VPS 输出的硬件 `esMat`，应开启硬件输入，并确保 `pixelFormat` 与输入 `esMat` 像素格式一致：

```
# 输入为硬件 esMat（解码器/VPS 输出）时：建议开启硬件输入
params = cv2.escodec.EncoderParams()
params.pixelFormat = cv2.es.ES_PIXFMT_NV12
params.useHardwareInput = True # 硬件 AVFrame 直接传递给编码器

# 输入为软件帧时，应使用 EncoderParams 重载并关闭硬件输入
params.useHardwareInput = False
```

模式	特点	适用场景
useHardwareInput=True	零拷贝硬件输入	硬件 esMat 输入（解码器/VPS 输出），且 pixelFormat 匹配
useHardwareInput=False	软件帧输入	输入不是硬件 AVFrame，或需要按软件帧路径处理

4.5.1.5. 编码格式详细规格

`createESVideoWriter` 硬件接口支持以下编码格式：

格式	编码支持	Profile	分辨率范围	对齐要求	备注
H.264	硬件加速	Baseline/-Main/High	144×128 ~ 8192×8192	2 像素对齐	主流格式，推荐
H.265	硬件加速	Main/Main10	136×128 ~ 8192×8192	2 像素对齐	高压缩率，推荐
MJPEG	硬件加速	-	32×32 ~ 32768×32768	2 像素对齐	帧内编码，最大分辨率

硬件编码器说明：

- H.264/H.265/MJPEG 使用 ESMPP 硬件编码器 (FFmpeg: `h264_esmpp_encoder` / `hevc_esmpp_encoder` / `mjpeg_esmpp_encoder`)
- 本章聚焦于 `cv2.escodec.createESVideoWriter` 的硬件编码路径；如需其他编码格式，通常需要走 FFmpeg 软件编码或其他工具链。

4.5.1.6. 容器格式详细支持

容器	文件扩展名	常见用途	备注
MP4	.mp4	存储/分发	通用兼容性好
Matroska	.mkv	存储/归档	功能丰富，支持多轨
AVI	.avi	兼容旧系统	对现代编码特性支持有限
FLV	.flv	传统推流	常用于历史 RTMP 链路
MPEG-TS	.ts	传输/直播	广播与流媒体常用封装
Raw	.h264/h265	性能/调试	无容器封装，便于测试码流

4.5.1.7. 编码器输入格式兼容性

编码器可从 `esMat` 直接输入。为了尽量走零拷贝链路，建议优先使用解码器/VPS 常见的 YUV 格式（例如 NV12/I420）。

格式	常量名	典型来源	备注
8-bit YUV 格式			
I420	<code>cv2.es.ES_PIXFMT_I420</code>	解码器默认输出	通用性好
NV12	<code>cv2.es.ES_PIXFMT_NV12</code>	解码器/VPS 输出	推荐用于完整硬件链路
NV21	<code>cv2.es.ES_PIXFMT_NV21</code>	VPS 输出，Android 常用	与 NV12 的 UV 顺序不同
YV12	<code>cv2.es.ES_PIXFMT_YV12</code>	软件处理	兼容性视链路而定
YUYV	<code>cv2.es.ES_PIXFMT_YUY2</code>	相机输入 (YUV422 packed)	需要与实际输入匹配
UYVY	<code>cv2.es.ES_PIXFMT_UYVY</code>	相机输入 (YUV422 packed)	需要与实际输入匹配
10-bit YUV 格式			

格式	常量名	典型来源	备注
P010LE	cv2.es.ES_PIXFMT_YUV420SP010LE	HDR 解码器 (10-bit NV12)	10-bit 链路
YUV420P010LE	cv2.es.ES_PIXFMT_YUV420P010LE	HDR 解码器 (10-bit I420)	10-bit 链路
灰度格式			
GRAY8	cv2.es.ES_PIXFMT_GRAY8	解码器，单通道	单通道输入

格式选择建议：

编码格式	推荐输入格式	原因
H.264/H.265	NV12 或 I420	最佳兼容性，零拷贝性能最优
MJPEG	I420	传统 JPEG 标准支持
HDR 内容 (10-bit)	P010LE 或 YUV420P010LE	保留 10-bit 色深，配合 H.265 Main10 Profile

输入格式配置示例：

```
# 示例 1: 从 NV12 解码器直接编码 (推荐)
decode_params = cv2.escodec.ESVideoReaderInitParams()
decode_params.decodeSurfaceFormat = cv2.es.ES_PIXFMT_NV12

reader = cv2.escodec.createVideoReader("input.mp4", decode_params)

encode_params = cv2.escodec.EncoderParams()
encode_params.pixelFormat = cv2.es.ES_PIXFMT_NV12 # 与解码器输出一致
encode_params.useHardwareInput = True           # 零拷贝

writer = cv2.escodec.createESVideoWriter(
    "output.mp4", (1920, 1080), 30.0, encode_params
)

# 零拷贝转码
frame = cv2.es.esMat()
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame)

# 示例 2: 10-bit HDR 编码
hdr_params = cv2.escodec.EncoderParams()
hdr_params.codec = cv2.escodec.CODEC_H265
hdr_params.pixelFormat = cv2.es.ES_PIXFMT_YUV420SP010LE # 10-bit P010LE
hdr_params.bitDepth = 10                                # 10-bit 输出
hdr_params.profile = 2                                   # Main10 Profile
hdr_params.useHardwareInput = True

hdr_writer = cv2.escodec.createESVideoWriter(
    "hdr_output.mp4", (3840, 2160), 30.0, hdr_params
)
```

4.5.2. 创建编码器

使用 `createESVideoWriter()` 函数创建硬件编码器。

命名说明 函数名为 `createESVideoWriter` (而非 `createVideoWriter`) 是为了与标准 OpenCV 的 `cv2.VideoWriter` 区分。后者使用 CPU 软件编码，而本函数提供硬件加速能力。

重载	参数	说明
<code>createESVideoWriter(filename, frameSize, fps, codec=CODEC_H265)</code>	filename: str, frameSize: tuple[int, int], fps: float, codec: int	简化接口，默认按硬件输入帧路径创建编码器
<code>createESVideoWriter(filename, frameSize, fps, params)</code>	filename: str, frameSize: tuple[int, int], fps: float, params: EncoderParams	推荐接口，可显式配置编码格式、码率、GOP、输入像素格式和硬件输入方式

在 Python 中，这两个重载通过第 4 个参数类型区分：传入 `int` 表示 `codec`；传入 `EncoderParams` 表示使用参数重载。若输入为软件帧，或需要显式配置 `pixelFormat`、码率、GOP 等参数，应使用 `EncoderParams` 重载。

参数：

- `filename` (str): 输出文件路径或流 URL
- `frameSize` (tuple): 视频尺寸 (width, height)
- `fps` (float): 帧率
- `codec` (int): 编码格式
 - `cv2.escodec.CODEC_H264` (0)
 - `cv2.escodec.CODEC_H265` (1)
 - `cv2.escodec.CODEC_MJPEG` (2)
- `params` (`EncoderParams`): 编码参数（此重载下编码格式等信息从 `params` 读取）

基础用法：

```
# H.264 编码到 MP4 (推荐使用 EncoderParams 重载)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(
    "output.mp4",
    (1920, 1080),
    30.0,
    params
)

# H.265 编码到 MKV
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H265
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(
    "output.mkv",
    (3840, 2160),
    25.0,
    params
)
```



```
# 编码循环
frame = cv2.es.esMat()
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame)

# 完成编码
writer.release()
```

4.5.3. 编码参数配置

EncoderParams 字段较多，建议把它当作“按需覆盖”的配置对象使用：只设置与你的业务相关的少量字段，其余使用默认值。

完整字段与默认值见 [EncoderParams 参数类](#)。

4.5.3.1. 常用参数说明

参数	默认值	说明	适用场景
codec	CODEC_H264	编码格式：H.264(0), H.265(1), MJPEG(2)	H.265: 4K/10-bit H.264: 1080p/兼容性 MJPEG: 图像序列
bitrate	2000	目标码率 (kbps), VBR/CBR 模式使用	参考范围: 1080p 2000-6000、720p 1000-3000、4K 10000-25000 (与内容复杂度/画质目标/网络带宽相关)
rcMode	RC_VBR	码率控制：CBR(0), VBR(1), CQP(2)	VBR: 文件存储 CBR: 直播推流 CQP: 固定质量
gopSize	75	I 帧间隔 (帧数)，通常为 1-4 秒	实时流: 30-60 文件: 60-120 低延迟: 15-30
qp	-1	固定 QP (CQP 模式)，18-28 高质量，23 默认	CQP 模式下使用，其他模式设为-1
useHardwareInput	False	关键! 输入为 esMat 走设备侧链路时通常应设为 True	如果输入来自 CPU 侧 numpy，先 upload 为 esMat 再设 True；否则应按实际输入类型选择
iqp / qp / bqp	30/32/32	各类型帧的基础 QP 值	精细调整质量时使用
qpMin / qpMax	-1	QP 范围限制 (VBR 模式)	VBR: qpMin=15, qpMax=40 CBR: 使用默认值
profile / level	-1/0	H.264/H.265 Profile/Level，-1=自动	通常使用自动，特殊需求才手动设置
bitDepth	8	色深 (8/10)	10-bit: HDR 内容，通常配合 H.265 Main10
enableLowLatency	False	低延迟模式 (减少编码缓冲)	实时通信、云游戏

参数	默认值	说明	适用场景
timestampMode	0	PTS 生成: 0=帧计数, 1=使用输入 PTS	0: 常规场景 1: VFR 视频、自定义时间戳

QP 值与质量关系:

- QP 18-23: 高质量, 接近无损
- QP 23-28: 中等质量, 视觉良好
- QP 28-35: 低质量, 可见压缩痕迹
- QP > 35: 质量较差

4.5.3.2. 配置示例

```
# 示例 1: 高质量 H.264 (文件存储)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.bitrate = 6000 # 6 Mbps
params.rcMode = cv2.escodec.RC_VBR
params.qpMin = 15
params.qpMax = 35
params.gopSize = 60
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True # 零拷贝

writer = cv2.escodec.createESVideoWriter(
    "hq_output.mp4", (1920, 1080), 30.0, params
)

# 示例 2: 低码率 CBR (网络直播)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.bitrate = 2000 # 2 Mbps
params.rcMode = cv2.escodec.RC_CBR
params.gopSize = 30
params.enableLowLatency = True
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "stream_output.mp4", (1280, 720), 30.0, params
)

# 示例 3: 固定 QP (恒定质量)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.qp = 23 # 固定 QP=23
params.rcMode = cv2.escodec.RC_CQP
params.gopSize = 60
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "fixqp_output.mp4", (1920, 1080), 30.0, params
)

# 示例 4: H.265 4K 编码
params = cv2.escodec.EncoderParams()
```

```
params.codec = cv2.escodec.CODEC_H265
params.bitrate = 15000 # 15 Mbps
params.rcMode = cv2.escodec.RC_VBR
params.gopSize = 120
params.qpMin = 20
params.qpMax = 40
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "4k_output.mp4", (3840, 2160), 30.0, params
)

# 示例 5: H.265 10-bit HDR 编码
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H265
params.bitDepth = 10 # 10-bit
params.bitrate = 20000 # 20 Mbps
params.rcMode = cv2.escodec.RC_VBR
params.profile = 2 # Main10 Profile
params.pixelFormat = cv2.es.ES_PIXFMT_YUV420SP010LE # P010LE
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "hdr_output.mp4", (3840, 2160), 30.0, params
)

# 示例 6: MJPEG 编码 (图像序列)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_MJPEG
params.qfactor = 30 # 质量因子 (较小=更高质量)
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "sequence.mjpeg", (1920, 1080), 30.0, params
)
```

编码推荐配置：

场景	分辨率	帧率	编码器	码率	GOP	说明
高清存档	1080p	30	H.264	6000 kbps	120	VBR, qpMin=15, qpMax=35
网络直播	720p	30	H.264	2000 kbps	30	CBR, enableLowLatency=True
4K 存储	4K	30	H.265	15000 kbps	120	VBR, qpMin=20, qpMax=40
HDR 内容	4K	30	H.265	20000 kbps	120	10-bit, Main10 Profile
低延迟	720p	30	H.264	2000 kbps	15	CBR, GOP ≤ 30, enableLowLatency

场景	分辨率	帧率	编码器	码率	GOP	说明
图像序列	任意	任意	MJPEG	-	1	qfactor=20-50

4.5.4. 高级特性

4.5.4.1. 自定义码流处理

对于需要自定义码流处理的场景（如实时网络传输、自定义封装），Python 用户可使用以下方法：

4.5.4.2. 方案 1：编码到文件后处理

```
# 先编码到临时文件
temp_file = "temp_output.mp4"
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(temp_file, (1920, 1080), 30.0, params)

# 编码
while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame)
writer.release()

# 后处理：提取码流/推送等
import subprocess
subprocess.run(["ffmpeg", "-i", temp_file, "-c", "copy", "-f", "flv", "rtmp://server/live"])
```

4.5.4.3. 方案 2：使用原始格式输出

```
# 编码为原始 H.264
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "output.h264", # 无容器
    (1920, 1080),
    30.0,
    params
)

# 编码后，码流直接在 output.h264 中，可以自行处理
```

4.5.4.4. 流媒体推送

```
# 方法 1: 直接推流到 RTMP 服务器
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
    "rtmp://server/live/stream",
    (1920, 1080),
    30.0,
    params
)

# 方法 2: 先编码再用 ffmpeg 推流
temp_file = "temp.mp4"
writer = cv2.escodec.createESVideoWriter(temp_file, (1920, 1080), 30.0, params)

# 编码...
writer.release()

# ffmpeg 推流
import subprocess
subprocess.Popen([
    "ffmpeg", "-re", "-i", temp_file,
    "-c", "copy", "-f", "flv",
    "rtmp://server/live/stream"
])
```

4.5.5. 编码器 API 参考

本节提供完整的硬件编码器 Python API 参考。

4.5.5.1. 工厂函数

函数名	功能	参数	返回值
cv2.escodec.createESVideoWriter()	创建硬件编码器	重载 1: filename, frame-Size, fps, codec: int = CODEC_H265 重载 2: filename, frame-Size, fps, params: EncoderParams	VideoWriter 对象

使用示例:

```
# 基本创建 (使用 codec 重载)
writer = cv2.escodec.createESVideoWriter(
    "output.mp4",
    (1920, 1080),
    30.0,
    cv2.escodec.CODEC_H264
)

# 带参数创建 (使用 params 重载; 编码格式从 params.codec 读取)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
```

```
params.bitrate = 6000
params.rcMode = cv2.escodec.RC_VBR
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(
    "output.mp4", (1920, 1080), 30.0, params
)
```

4.5.5.2. VideoWriter 类方法

方法名	功能	参数	返回值	说明
write()	写入一帧	frame: esMat	None	校验输入帧并将当前帧送入编码器；编码器内部可能缓存延迟帧
release()	释放编码器资源	无	None	flush 编码器内部缓存，写入文件尾并关闭资源
getParams()	获取编码参数	无	EncoderParams	返回当前编码器参数配置

方法使用示例：

```
import cv2

# 创建编码器
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(
    "output.mp4", (1920, 1080), 30.0, params
)

# 编码视频帧
reader = cv2.escodec.createVideoReader("input.mp4")
frame = cv2.es.esMat()

while True:
    ok, frame = reader.nextFrame(frame)
    if not ok:
        break
    writer.write(frame) # 零拷贝写入

# 查询当前编码参数
current_params = writer.getParams()
print(f"编码格式: {current_params.codec}")

# 完成编码
writer.release()
```

4.5.5.3. EncoderParams 参数类

基础参数：

参数名	类型	默认值	说明
codec	int	CODEC_H264	编码格式（CODEC_H264/ H265/ MJPEG）
bitrate	int	2000	目标码率（kbps），0=自动
rcMode	int	RC_VBR	码率控制（RC_CBR/VBR/CQP）
gopSize	int	75	GOP 大小（I 帧间隔）
qp	int	-1	固定 QP 值（-1=自动，仅 CQP 模式）
enableLowLatency	bool	False	低延迟模式

硬件输入配置：

参数名	类型	默认值	说明
useHardwareInput	bool	False	关键！ 使用 DRM_PRIME 零拷贝输入
pixelFormat	int	ES_PIXFMT_I420	输入像素格式
strideAlign	int	1	YUV stride 对齐

QP 控制参数：

参数名	类型	默认值	说明
iqp	int	30	I 帧 QP（0-51）
pqp	int	32	P 帧 QP（0-51）
bqp	int	32	B 帧 QP（0-51）
qpMax	int	-1	最大 QP（0-51，-1=自动）
qpMin	int	-1	最小 QP（0-51，-1=自动）
qpIMax	int	-1	I 帧最大 QP
qpIMin	int	-1	I 帧最小 QP
iQpDelta	int	2	I 帧 QP 偏移
ipQpDelta	int	2	I-P QP 偏移
spQpDelta	int	0	特殊 B 帧 QP 偏移

GOP 结构参数：

参数名	类型	默认值	说明
bFrmNum	int	2	B 帧数量（1-3）
gopMode	int	0	GOP 模式
sbInterval	int	0	特殊 B 帧间隔
bgInterval	int	-1	长期参考帧间隔
bgQpDelta	int	5	长期参考帧 QP 偏移

H.264/H.265 特性参数：

参数名	类型	默认值	说明
profile	int	-1	Profile (-1=自动)
level	int	0	Level (0=自动)
tier	int	0	HEVC Tier (0=Main, 1=High)
bitDepth	int	8	色深 (8/10)
enableCabac	int	0	H.264 CABAC (0/1)
enableDeblock	int	1	去块滤波器 (0/1)

码率控制高级参数：

参数名	类型	默认值	说明
iprop	int	-1	I 帧比例 (50-100, -1=自动)
statTime	int	1	码率统计时间 (1-60 秒)
cpbSize	int	-1	CPB 缓冲区大小 (kbps, -1=自动)

MJPEG 特性参数：

参数名	类型	默认值	说明
qfactor	int	50	质量因子 (1-99, 越小质量越好)
qfactorMin	int	-1	最小质量因子
qfactorMax	int	-1	最大质量因子

高级功能参数：

参数名	类型	默认值	说明
outFps	int	25	输出帧率覆盖 (≤ 0 使用 fps 参数)
timestampMode	int	0	PTS 生成模式 (0=帧计数, 1=使用输入 PTS)

参数配置示例：

```
import cv2

# 示例 1: 高质量 H.264 编码 (VBR)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.bitrate = 6000 # 6 Mbps
params.rcMode = cv2.escodec.RC_VBR
params.gopSize = 60
params.qpMin = 15
params.qpMax = 35
params.pixelFormat = cv2.es.ES_PIXFMT_I420
```



```
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("hq.mp4", (1920, 1080), 30.0, params)

# 示例 2: 低延迟 CBR 编码 (直播)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.bitrate = 2000 # 2 Mbps
params.rcMode = cv2.escodec.RC_CBR
params.gopSize = 30
params.enableLowLatency = True
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("stream.mp4", (1280, 720), 30.0, params)

# 示例 3: 固定 QP 编码 (CQP)
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.qp = 23 # 固定 QP=23
params.rcMode = cv2.escodec.RC_CQP
params.gopSize = 60
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("cqp.mp4", (1920, 1080), 30.0, params)

# 示例 4: H.265 4K 编码
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H265
params.bitrate = 15000 # 15 Mbps
params.rcMode = cv2.escodec.RC_VBR
params.gopSize = 120
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("4k.mp4", (3840, 2160), 30.0, params)

# 示例 5: 10-bit HDR 编码
params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H265
params.bitDepth = 10
params.profile = 2 # Main10 Profile
params.bitrate = 20000
params.pixelFormat = cv2.es.ES_PIXFMT_YUV420SP010LE
params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter("hdr.mp4", (3840, 2160), 30.0, params)
```

4.5.5.4. 编码常量

编码格式:

常量名	值	说明	硬件加速
cv2.es-codec.CODEC_H264	0	H.264/AVC	是 (ESMPP)
cv2.es-codec.CODEC_H265	1	H.265/HEVC	是 (ESMPP)
cv2.es-codec.CODEC_MJPEG	2	Motion JPEG	是 (ESMPP)

码率控制模式：

常量名	值	说明	适用场景
cv2.escodec.RC_CBR	0	固定码率	网络直播、实时传输
cv2.escodec.RC_VBR	1	可变码率	文件存储、离线处理
cv2.escodec.RC_CQP	2	固定 QP	固定质量、测试

编码器运行参数通过 `EncoderParams` 在创建时配置；`ESVideoWriter` 不提供 OpenCV 通用 `VideoWriter` 属性读写接口。创建后可通过 `getParams()` 获取当前编码参数。

4.5.5.5. 输入像素格式常量

8-bit YUV 格式（推荐，零拷贝）：

常量名	说明	典型来源
cv2.es.ES_PIXFMT_I420	YUV420 Planar	解码器默认输出
cv2.es.ES_PIXFMT_NV12	YUV420 Semi-Planar (UV 交织)	解码器/VPS 输出，推荐
cv2.es.ES_PIXFMT_NV21	YVU420 Semi-Planar (VU 交织)	VPS 输出
cv2.es.ES_PIXFMT_YV12	YVU420 Planar	软件处理
cv2.es.ES_PIXFMT_YUY2	YUV422 Packed	相机输入
cv2.es.ES_PIXFMT_UYVY	YUV422 Packed	相机输入

10-bit YUV 格式（HDR）：

常量名	说明	用途
cv2.es.ES_PIXFMT_YUV420SP010LE	10-bit NV12 (P010LE)	HDR 内容，H.265 Main10
cv2.es.ES_PIXFMT_YUV420P010LE	10-bit I420	HDR 内容

灰度格式：

常量名	说明
cv2.es.ES_PIXFMT_GRAY8	8-bit 灰度

4.5.5.6. 设计原则总结

- 1. **零拷贝优先**：输入为硬件 `esMat` 且像素格式匹配时设置 `useHardwareInput=True`；输入为软件帧时应关闭硬件输入
- 2. **格式匹配**：编码器 `pixelFormat` 应与解码器/VPS 输出格式一致（推荐 NV12）
- 3. **码率控制**：
 - 直播/实时：CBR + `enableLowLatency=True` + 短 GOP(30)
 - 存储/离线：VBR + `qpMin/qpMax` 范围控制 + 长 GOP(60-120)

- 固定质量：CQP + 合适的 qp 值 (18-28)
4. **分辨率对齐**：编码器要求 2 像素对齐（宽度和高度必须为偶数）
 5. **资源释放**：编码完成后必须调用 `writer.release()` 写入文件尾

4.6. 硬件图像处理

本章介绍如何使用 `cv2.esimgproc` 面向 `cv2.es.esMat` 完成图像缩放、格式转换、裁剪/Letterbox、归一化与 OSD 绘制。

与标准 OpenCV 不同，`cv2.esimgproc` 主要面向设备内存对象 `cv2.es.esMat`：

- 输入/输出通常都是 `esMat`，避免 CPU 拷贝
- 多数算子返回 `(ret, dst)`，其中 `ret==0` 表示成功
- 通过参数结构体（例如 `ProcessParams`）统一描述组合处理；启用归一化时，当前实现内部会按预处理与归一化分阶段执行

4.6.1. 功能概述

4.6.1.1. 图像处理后端

EIC7700 OpenCV 的图像处理接口主要围绕 `esMat` 设备内存设计。缩放、裁剪、格式转换、归一化、混合等路径通过 FFmpeg/libavfilter 与 ESMPP 后端构建硬件滤镜图；OSD 绘制优先使用 Native 快路径，条件不满足时回退到 FFmpeg 后端。主要功能包括：

- **图像缩放**：`resize()`
- **格式转换**：`cvtColor()`（按 `ES_PIXFMT_*` 指定输出格式）
- **裁剪/Letterbox**：`crop()` / `clip()` / `makeLetterboxClip()`
- **几何变换**：`rotate()` / `flip()`
- **前处理融合**：`process()`（可合并缩放/裁剪/Letterbox/格式转换/归一化）
- **图形绘制 (OSD)**：`drawBoxes()` / `drawTexts()` / `drawLines()` / `drawGrids()`

4.6.1.2. 适用场景

- **视频监控**：实时缩放、格式转换、OSD 叠加
- **AI 前处理**：图像归一化、letterbox 缩放
- **视频会议**：多分辨率输出、画面合成
- **智能分析**：ROI 提取、坐标变换

4.6.2. 图像缩放与格式转换

4.6.2.1. `resize`: 图像缩放

```
def resize(
    src: esMat,
    dst: esMat,
    dsize: tuple[int, int],
    fx: float = 0,
    fy: float = 0,
    interpolation: int = 0
) -> tuple[int, esMat]
```

参数：

- `src` (esMat): 源图像 (设备内存)
- `dst` (esMat): 目标图像 (设备内存)
- `dsize` (tuple): 目标尺寸 (width, height)
- `fx`, `fy` (float): 缩放因子 (`dsize` 为空时使用)
- `interpolation` (int): 兼容参数; 当前实现中该参数被忽略, 实际插值策略由硬件后端决定

返回值:

- (status, dst) - status 为 0 表示成功

基础缩放:

```
src = cv2.es.esMat()
ok, src = reader.nextFrame(src) # 1920x1080
if not ok:
    raise RuntimeError("decode failed")

dst = cv2.es.esMat()
ret, dst = cv2.esimgproc.resize(src, dst, (640, 480))

if ret == 0:
    print(f"输出尺寸: {dst.size()}") # (640, 480)
```

4.6.2.2. `cvtColor`: 格式转换

`cv2.esimgproc.cvtColor()` 用输出像素格式常量 (`cv2.es.ES_PIXFMT_*`) 描述目标格式, 返回 (ret, dst) :

```
def cvtColor(
    src: esMat,
    dst: esMat,
    dst_outputFormat: int,
) -> tuple[int, esMat]
```

示例 (NV12 → BGR):

```
nv12_frame = cv2.es.esMat()
ok, nv12_frame = reader.nextFrame(nv12_frame) # 典型为 NV12/I420
if not ok:
    raise RuntimeError("decode failed")

bgr = cv2.es.esMat()
ret, bgr = cv2.esimgproc.cvtColor(nv12_frame, bgr, int(cv2.es.ES_PIXFMT_B8G8R8))
if ret != 0:
    raise RuntimeError("esimgproc.cvtColor failed")
```

提示: 如果你的处理链路最终要回到编码器, 通常建议全程保持 YUV (例如 NV12), 只在确实需要 CPU 侧处理/可视化时才转到 RGB/BGR。

4.6.3. `process`: 通用前处理

`process` 是更通用的处理接口, 支持格式转换、归一化等。

```
def process(
    src: esMat,
    dst: esMat,
```

```
params: ProcessParams
) -> tuple[int, esMat]
```

ProcessParams:

```
class ProcessParams:
    common: CommonParams
    normalize_mode: NormalizeModeParams

class CommonParams:
    # 常用字段 (不止这些): dsize/fx/fy/crop_rect/flip_code/rotate_code 等
    clip_params: ClipParams # 裁剪/Letterbox 参数
    output_format: int # 输出像素格式 (ES_PIXFMT_*)

class NormalizeModeParams:
    enable: bool
    params: NormalizeParams
```

4.6.3.1. 用 makeLetterboxClip 生成 Letterbox 参数

Letterbox (保持宽高比并填充边) 通常通过 `process()` + `makeLetterboxClip()` 完成:

```
clip = cv2.esimgproc.makeLetterboxClip(
    src_size=(1920, 1080),
    target_size=(640, 640),
    fill="0x727272",
    alignment=cv2.esimgproc.LETTERBOX_ALIGN_CENTER,
)
```

`ClipParams` 结构体字段 (Python 侧可直接读写) 主要包含:

- `x, y, width, height`: 缩放后图像放到输出画布的位置与大小
- `o_w, o_h`: 输出画布大小 (目标尺寸)
- `fill`: 填充颜色 (例如 `"black"`、`"0x727272"`、`"black@0.7"`)

提示

YUV 输入在很多场景要求坐标/尺寸为偶数 (2 对齐)。如果 Letterbox 的 `x/y/width/height` 不是偶数, 可能会失败。

4.6.3.2. AI 预处理

```
params = cv2.esimgproc.ProcessParams()

# 1) Letterbox 到 640x640 (输出画布 640x640, 灰色填充)
common = params.common
common.clip_params = cv2.esimgproc.makeLetterboxClip(
    src_size=src.size(),
    target_size=(640, 640),
    fill="0x727272",
)
params.common = common
```

```
# 2) 归一化: 示例为 [0,255] -> [0,1]
norm_mode = params.normalize_mode
norm_mode.enable = True

norm = norm_mode.params
norm.mode = cv2.esimgproc.Z_SCORE
norm.mean_val = (0.0, 0.0, 0.0)
norm.std_reciprocal = (1/255.0, 1/255.0, 1/255.0)
norm.step_reciprocal = 1.0
norm.output_format = int(cv2.es.ES_PIXFMT_B8G8R8I1_PLANAR)
norm_mode.params = norm
params.normalize_mode = norm_mode

# 一次 process 调用描述 Letterbox + 归一化; 当前实现内部按预处理与归一化分阶段执行
preprocessed = cv2.es.esMat()
ret, preprocessed = cv2.esimgproc.process(src, preprocessed, params)
if ret != 0:
    raise RuntimeError("esimgproc.process failed")
```

如果你需要更“严格对齐”的 Letterbox 计算（例如确保所有坐标与尺寸满足 2 对齐），可以参考当前 Python 虚拟环境中由 `opencv_eswin_sample-1.0.0-py3-none-any.whl` 安装的示例模块：

- `~/opencv_env/lib64/python3.12/site-packages/opencv_eswin_sample/yolov5s/yolov5s.py`
- 运行方式：`python3 -m opencv_eswin_sample.yolov5s.yolov5s -h`

4.6.4. OSD 绘制

绘制接口会原地修改传入的 `esMat`，常用于把检测框/标签叠加到视频帧上；后端会在 Native 快路径与 FFmpeg 回退路径之间自动选择。

这些接口同样返回 `(ret, img)`，其中 `img` 一般仍是传入并被修改后的 `esMat`。

格式与后端说明（以当前实现为准）：

- `drawBoxes()`：对 YUV420P(I420)/NV12/NV21 有 Native 后端快路径；其它格式会回退到 FFmpeg 后端。
- `drawTexts()`：Native 后端支持更多格式，但在 `alpha/shadow/border` 等特性不可用或未编译 FreeType 时也会回退到 FFmpeg 后端。
- `drawLines()`：当前绘制直线接口仅支持水平线或垂直线。

4.6.4.1. drawBoxes: 绘制矩形框

```
def drawBoxes(
    img: esMat,
    params: DrawBoxesParams
) -> tuple[int, esMat]
```

DrawBoxesParams:

`DrawBoxesParams.boxes` 由 `Rectangle` 组成（Python 绑定里 `vector` 属性通常表现为 `tuple`，需要先构建 `list` 再整体赋值）。

绘制检测框：

```

frame = cv2.es.esMat()
ok, frame = reader.nextFrame(frame)
if not ok:
    raise RuntimeError("decode failed")

params = cv2.esimgproc.DrawBoxesParams()

boxes = []

rect1 = cv2.esimgproc.Rectangle()
rect1.x, rect1.y = 100, 100
rect1.width, rect1.height = 200, 150
rect1.color = "red"
rect1.thickness = 3
boxes.append(rect1)

rect2 = cv2.esimgproc.Rectangle()
rect2.x, rect2.y = 400, 300
rect2.width, rect2.height = 150, 100
rect2.color = "blue@0.5" # 半透明
rect2.thickness = -1 # 填充
boxes.append(rect2)

params.boxes = boxes
ret, frame = cv2.esimgproc.drawBoxes(frame, params)
if ret != 0:
    raise RuntimeError("esimgproc.drawBoxes failed")

```

4.6.4.2. drawTexts: 绘制文本

```

def drawTexts(
    img: esMat,
    params: DrawTextsParams
) -> tuple[int, esMat]

```

DrawTextsParams:

DrawTextsParams.labels 由 TextLabel 组成。

绘制标签:

```

params = cv2.esimgproc.DrawTextsParams()

labels = []
label = cv2.esimgproc.TextLabel()
label.text = "Person: 0.95"
label.x = 100
label.y = 80
label.fontfile = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
label.fontsize = 20
label.fontcolor = "white"
label.box = True
label.boxcolor = "black@0.7"
label.boxborderw = 5
labels.append(label)

params.labels = labels
ret, frame = cv2.esimgproc.drawTexts(frame, params)

```

```
if ret != 0:
    raise RuntimeError("esimgproc.drawTexts failed")
```

4.6.5. 图像处理 API 快速查阅

本节列出 `cv2.esimgproc` 的常用对外 API（以当前 Python 绑定导出的接口为准）。

4.6.5.1. 常用算子

函数名	功能	参数	返回值
<code>cv2.esimgproc.resize()</code>	缩放	src: esMat dst: esMat dsize: (w, h) fx/fy/interpolation	(ret, dst)
<code>cv2.esimgproc.cvtColor()</code>	像素格式转换	src: esMat dst: esMat dst_outputFormat: ES_PIXFMT_*	(ret, dst)
<code>cv2.esimgproc.crop()</code>	按矩形裁剪	src: esMat dst: esMat rect: (x, y, w, h)	(ret, dst)
<code>cv2.esimgproc.clip()</code>	按 ClipParams 裁剪/Letterbox	src: esMat dst: esMat params: ClipParams	(ret, dst)
<code>cv2.esimgproc.rotate()</code>	旋转	src: esMat dst: esMat rotateMode: ROTATE- MODE_*	(ret, dst)
<code>cv2.esimgproc.flip()</code>	翻转	src: esMat dst: esMat flipMode: FLIP_*	(ret, dst)
<code>cv2.esimgproc.normalize()</code>	归一化	src: esMat dst: esMat params: Normal- izeParams	(ret, dst)
<code>cv2.esimgproc.process()</code>	融合前处理	src: esMat dst: esMat params: ProcessParams	(ret, dst)
<code>cv2.esimgproc.processDualInput()</code>	双输入处理（叠加/融合）	src: esMat overlay: esMat dst: esMat params: DualInput- Params	(ret, dst)
<code>cv2.esimgproc.alphaBlend()</code>	两路混合	src: esMat overlay: esMat dst: esMat params: BlendParams	(ret, dst)
<code>cv2.esimgproc.snapshot()</code>	按间隔抓拍	src: esMat dst: esMat interval: int	(ret, dst)

4.6.5.2. 绘图函数

函数名	功能	参数	返回值
cv2.esimgproc.drawBoxes()	矩形框	img: esMat params: DrawBoxes-Params	(ret, img)
cv2.esimgproc.drawTexts()	文本标签	img: esMat params: DrawTextsParams	(ret, img)
cv2.esimgproc.drawLines()	直线	img: esMat params: DrawLines-Params	(ret, img)
cv2.esimgproc.drawGrids()	网格	img: esMat params: DrawGridsParams	(ret, img)

4.6.5.3. 辅助函数

函数名	功能	参数	返回值
cv2.esimgproc.makeLetterboxClip()	生成 Letterbox 参数	src_size: (w, h) target_size: (w, h) long_edge_fit: bool=True fill: str="black" alignment: LETTERBOX_ALIGN_* stride: int=2	ClipParams

4.6.5.4. 常用常量

- 状态码: `ESIMGPROC_SUCCESS` (0) 以及一组 `ESIMGPROC_ERR_*`
- Letterbox 对齐: `LETTERBOX_ALIGN_CENTER` / `LETTERBOX_ALIGN_TOP_LEFT` / ...
- 归一化模式: `MIN_MAX`、`Z_SCORE` (通过 `params.normalize_mode.enable` 控制是否启用)
- 说明: `processMultiInput()` / `stack()` 当前 Python 绑定未导出, 原因是 `std::vector<esMat>` 参数绑定受限; 多输入拼接/堆叠请使用 C++ 接口或先确认本地 Python 绑定已扩展支持。

4.7. 故障排查

本章参考 C++ 用户手册的结构, 给出 Python 使用中的最常见问题定位方法。

4.7.1. 日志系统

```
export OPENCV_LOG_LEVEL=DEBUG
export FFMPEG_LOG_LEVEL=debug
```

也可以在代码中设置 (需在创建编解码器前生效):

```
import os

os.environ["OPENCV_LOG_LEVEL"] = "DEBUG"
os.environ["FFMPEG_LOG_LEVEL"] = "debug"

import cv2
reader = cv2.VideoCapture("video.mp4")
```

4.7.2. 常见错误

4.7.2.1. 错误 1: 无法打开解码器

错误信息：

```
[ERROR] Unable to open decoder for file: input.mp4
```

可能原因：文件不存在/路径错误、格式不支持、驱动未加载。

排查：

```
ls -lh input.mp4
ffprobe input.mp4
```

4.7.2.2. 错误 2: 编码器创建失败

错误信息：

```
[ERROR] Failed to create encoder: No hardware encoder available
```

可能原因：驱动未加载、资源不足、分辨率/像素格式不满足约束。

排查要点（保守约束）：

- 对于 YUV420 类格式，宽高通常至少需要是 2 的倍数；不同编码格式可能要求更高对齐，失败时以返回码/日志为准。
- 目标板上检查驱动/状态：

```
cat /sys/kernel/debug/es_venc/status
```

4.7.2.3. 错误 3: 零拷贝路径不可用

错误信息：

```
[WARN] Zero-copy path not available, falling back to memcpy
```

常见原因：输入不是 esMat（例如传入了 NumPy 数组）、或格式不兼容。

建议：确保编解码与图像处理链路全程使用 esMat。

4.7.2.4. 错误 4：模块/方法名不匹配

- `ModuleNotFoundError: No module named 'cv2.escodec'`：通常是安装了标准 OpenCV 覆盖 ESWIN 版本。
- `'esMat' object has no attribute 'dmabuf_fd'`：当前 Python 绑定获取 DMA buffer fd 的方法名为 `dmaFd()`。

```
frame = cv2.es.esMat()
ok, frame = reader.nextFrame(frame)
if not ok:
    raise RuntimeError("decode failed")
print("dma fd:", frame.dmaFd())
```

4.7.3. 调试技巧

4.7.3.1. 验证数据路径

逐步验证“解码 →（可选）下载一帧检查 → 编码”，快速定位问题环节：

```
import cv2

reader = cv2.escodec.createVideoReader("input.mp4")
fmt = reader.format()
enc_params = cv2.escodec.EncoderParams()
enc_params.codec = cv2.escodec.CODEC_H264
enc_params.pixelFormat = cv2.es.ES_PIXFMT_I420
enc_params.useHardwareInput = True
writer = cv2.escodec.createESVideoWriter(
    "output.mp4", (fmt.width, fmt.height), fmt.avgFrameRate, enc_params)

frame = cv2.es.esMat()
ok, frame = reader.nextFrame(frame)
if not ok:
    raise RuntimeError("decode failed")

# 下载一帧到 CPU 做可视化校验（调试用）
cpu = frame.download()
cv2.imwrite("debug_frame.jpg", cpu)

writer.write(frame)
writer.release()
```

4.7.3.2. 检查系统信息

```
dmesg | grep -i "error\|fail"
cat /sys/kernel/debug/es_vdec/status
cat /sys/kernel/debug/es_venc/status
```

4.8. 附录

4.8.1. 完整示例代码

完整可运行的示例代码参见：

- **Python 示例模块：**当前 Python 环境的 `site-packages/opencv_eswin_sample/`
- **Python 示例运行方式：** `python3 -m opencv_eswin_sample.<module>`
- **数据文件：**当前 Python 环境的 `site-packages/opencv_eswin_sample/data/`

Python 示例由 `opencv_eswin_sample-1.0.0-py3-none-any.whl` 提供，默认使用 wheel 内置数据。安装 Python 示例 wheel 后，可通过 `python3 -m opencv_eswin_sample.<module> -h` 查看参数说明。

主要示例包括：

示例	功能	源文件
简单转码	零拷贝视频转码	<code>transcode.py</code>
推理示例（可选）	推理 Pipeline	<code>yolov5s/yolov5s.py</code>
水印叠加	图形绘制	<code>watermark/watermark.py</code>
多进程处理	多路并发	<code>multiprocess/process.py</code>

4.8.2. API 参考索引

模块	API 参考文档	位置
<code>cv2.es</code>	escore API 参考	本手册 <code>cv2.es</code> 章节
<code>cv2.escodec</code>	escodec API 参考	本手册 <code>cv2.escodec</code> 章节
<code>cv2.esimgproc</code>	esimgproc API 参考	本手册 <code>cv2.esimgproc</code> 章节
<code>es_py</code>	ESWIN SDK Python 文档	交付方提供的 ESSDK Python 文档

交互式文档查询：除了本手册外，您还可以在 Python 交互环境中使用 `help(cv2.escodec)` 等命令查看 API 的详细文档和函数签名。

4.8.3. 常用编码配置表

本节只提供“怎么找参数、怎么最小化配置”的方法，避免不同版本字段差异导致示例失效。

- 1) 查看当前 wheel 支持的参数字段（以你正在运行的环境为准）：

```
import cv2

params = cv2.escodec.EncoderParams()
print([name for name in dir(params) if not name.startswith("_")])
help(cv2.escodec.EncoderParams)
```

- 2) 最小可用配置示例（常见字段：`codec` / `bitrate` / `useHardwareInput`）：

```
import cv2

params = cv2.escodec.EncoderParams()
params.codec = cv2.escodec.CODEC_H264
params.bitrate = 4000 # kbps
params.pixelFormat = cv2.es.ES_PIXFMT_I420
params.useHardwareInput = True

writer = cv2.escodec.createESVideoWriter(
```

```
"output.mp4",
(1920, 1080),
30.0,
params,
)
```

4.8.4. Python API 设计说明

4.8.4.1. API 命名规范

EIC7700 OpenCV Python API 遵循以下设计原则：

功能	Python API	说明
创建解码器	cv2.escodec.createVideoReader()	驼峰命名，遵循 OpenCV 风格
创建编码器	cv2.escodec.createESVideoWriter()	注意是 ESVideoWriter（不是 VideoWriter）
esMat 构造	esMat(rows, cols, type)	与 OpenCV Mat 接口类似
尺寸表示	(width, height) 元组	Python 原生元组，不需要 Size 对象
矩形表示	(x, y, w, h) 元组	Python 原生元组，不需要 Rect 对象
参数列表	Python 列表 [...]	直接使用 Python list，不需要 vector
对象管理	自动垃圾回收	通常由 Python GC 管理；需要提前释放资源时可显式调用 release()
异常处理	cv2.error 异常	使用 try-except 捕获

4.8.4.2. Python 侧使用约定

1. NumPy 互操作

```
# esMat ↔ NumPy
cpu = frame.download()    # 设备 → CPU
frame2 = cv2.es.esMat()
frame2.upload(cpu)        # CPU → 设备
```

2. 返回值风格

许多硬件算子返回 (ret, dst)：

```
ret, dst = cv2.esimgproc.resize(src, dst, (640, 480)) # interpolation 当前为兼容参数，实际会被后端忽略
if ret != 0:
    raise RuntimeError(f"resize failed: {ret}")
```

3. Python 异常机制

```
try:
    reader = cv2.escodec.createVideoReader("video.mp4")
except cv2.error as e:
```

```
print(f"OpenCV 错误: {e}")  
except FileNotFoundError:  
    print("文件不存在")
```

4. 参数与字段以运行环境为准

不同 wheel 版本可能存在字段差异，建议优先用 `help(cv2.esccodec)` / `help(cv2.esimgproc)` / `.pyi` 签名作为准绳。